Seven Verbs of Coding

A coding-method taxonomy for 150 algorithms and 254 LeetCode problems

How to read this book
The taxonomy
- Seven verbs
- The tree
- How to remember the tree
- Choosing at a blank page
- What the tree shows
- How to practise

Sweep
- Fold · UpKeep
- Lookback · Nelolo
- Reach · Reacut
- Sorted sweep · Sostre
- Two passes · PreSuf
- Runs · RunPack

Hold
- Window · GroShriSco
- Ratchet · PoAnPu
- Nesting · Pupore
- Cursor stacks · Leri
- Time ring · Evadco

Squeeze
- Pincers · MeKeMo
- Chase and zip · Chazip
- Bisect stored order · First True
- Bisect the answer · First True
- Partition · Pacha
- Peel · Peel

Drain
- Wave · Poprexen
- Diver · Poprexpu
- Cheapest first · PoStaRePu
- Keep the best · Putri

Branch
- Enumerate · Rechexun
- Path search · Marexun
- Combine · Balerico
- Pass down · Bachepa
- Stamp · Grabla

Fill
- Chain · Tasefire
- State vector · Costep
- Knapsack · Tasefire
- 2-D table · Tasefire
- Interval · Tasefire
- Subsequence · Tasefire

Serve
- Map and order · OpMapSync
- Hash to slots · Haprop
- Roots · FiFiSaSu
- Trie · WaCreStep
- Tally · Cobuco

Index of problems
Index of book entries

How to read this book

You know the data structures and most of the algorithms. What an interview tests is the step before the code: reading a problem statement and seeing which loop to write, what it carries, and which property of the problem makes that loop correct. This book trains that step.

It sorts everything by **coding method**, not by topic. A method is a loop skeleton with named steps and a few dials. Its name is built from the first syllables of its steps, the way *Poprexen* is POP, PRocess, EXpand, ENqueue. There are 37 methods in seven families. Each method page has four parts:

- **Picture, invariant, use when.** The motion of the code on the data, the promise kept at the top of every iteration, and the words in a statement that call for the method.
- **General solution.** The anchor problem, the intuition behind its solution, and one runnable Python solution with every step labelled by its syllable. Every general solution in the book is executed against tests when the book is built.
- **Dials and traps.** What the problems change, and the classic mistakes.
- **Problems.** 254 LeetCode problems in all. Each gives the **problem** with a tiny example, the **intuition** of its solution, and what **differs** from the general solution. If you can predict the intuition and the “differs” line from the statement, you own the method.

Every method also lists its **book entries**: the entries of *150 Algorithms* whose code it is.

The taxonomy

Seven verbs

Every loop is driven by something. Name the driver and what the code keeps, and the method follows.

Family	What drives the loop	What the code keeps	The question it asks
Sweep	the input, one pass	a summary of the past	What single summary of the past do I carry?
Hold	the input, one pass	the live items themselves	Which live items do I keep, and when do they leave?
Squeeze	a shrinking range of candidates	what is left of the range	What can I throw away for good?
Drain	a bag of pending work	the bag	Who is next in line?
Branch	the call stack	arguments down, return values up	What do the smaller pieces say?
Fill	the dependency order of a table	answers to smaller inputs	What did the smaller inputs already answer?
Serve	the caller's method calls	an invariant between calls	What must stay true between calls?

A kitchen keeps them in order: **sweep** the floor, **hold** the tray, **squeeze** the lemon, **drain** the sink, **branch** the recipe, **fill** the table, **serve** the guests.

The tree

Each family splits into methods by its dial. The numbers are the book entries each method covers.

```
CODING METHODS
├─ SWEEP: one pass over the input; carry a summary of the past
│    ├─ Fold             UpKeep      69 86 102 131 135
│    ├─ Lookback         Nelolo      1 2 4 5 96 134
│    ├─ Reach            Reacut      67 68 72
│    ├─ Sorted sweep     Sostre      26 27 65 70 112 142
│    ├─ Two passes       PreSuf      3 89 111 124 127 129
│    └─ Runs             RunPack     88 121 122 123
├─ HOLD: one pass over the input; keep the live items themselves
│    ├─ Window           GroShriSco  11 12 13 53 54 120
│    ├─ Ratchet          PoAnPu      23 62 64 132 133
│    ├─ Nesting          Pupore      22 24 25 63
│    ├─ Cursor stacks    Leri        50 95 144
│    └─ Time ring        Evadco      45 99 110 138
├─ SQUEEZE: every look discards candidates for good
│    ├─ Pincers          MeKeMo      7 8 9 56 126
│    ├─ Chase and zip    Chazip      10 66 104 150
│    ├─ Bisect stored order First True  14 47 92 98 106
│    ├─ Bisect the answer First True  15 16 55 57 105
│    ├─ Partition        Pacha       18 52 60
│    └─ Peel             Peel        100 125 128 130 136 145
├─ DRAIN: a bag of pending work drives the loop
│    ├─ Wave             Poprexen    32 39 40 78 79 80 83 114
│    ├─ Diver            Poprexpu    37 38 49 77 84 115
│    ├─ Cheapest first   PoStaRePu   20 42 46 119 149
│    └─ Keep the best    Putri       19 21 59 94 109 140
├─ BRANCH: the call stack drives the loop
│    ├─ Enumerate        Rechexun    30 31 73 103 137
│    ├─ Path search      Marexun     74 85 113
│    ├─ Combine          Balerico    17 33 34 107 148
│    ├─ Pass down        Bachepa     35 36 75 76
│    └─ Stamp            Grabla      81 82 116
├─ FILL: answers to smaller inputs, in dependency order
│    ├─ Chain            Tasefire    28 90
│    ├─ State vector     Costep      29 117
│    ├─ Knapsack         Tasefire    (classic DP)
│    ├─ 2-D table        Tasefire    (classic DP)
│    ├─ Interval         Tasefire    87
│    └─ Subsequence      Tasefire    58
└─ SERVE: the caller drives the loop; an invariant survives every call
     ├─ Map and order    OpMapSync   6 44 91 93 141
     ├─ Hash to slots    Haprop      51 101 139 146 147
     ├─ Roots            FiFiSaSu    41 118
     ├─ Trie             WaCreStep   43 97 143
     └─ Tally            Cobuco      48 61 71 108
```

How to remember the tree

Walk through the kitchen. Each verb is one job, and each of its methods is a picture inside that job that moves the way the code moves.

- **Sweep the floor.** You **fold** the dust into one pile as you go, **look back** at the spots you have already passed, **reach** as far as the broom goes before you step, **sort** the debris before you heap it, go over the floor in **two passes**, and push it out in **runs**.
- **Hold the tray.** A **window** of glasses slides along it; a **ratchet** of plates, where each new plate knocks off the smaller ones waiting on top; **nested** bowls that close in order; two piles that meet at your thumb, the **cursor stacks**; and a **time ring** that throws out what has gone cold.
- **Squeeze the lemon.** Close the **pincers** from both ends; **chase** a pip with a fast finger and a slow one, or **zip** two halves together; **bisect** it along its stored segments, or bisect by tasting the **answer** (sour enough?); **partition** the juice from the pulp around one pip; and **peel** the rind one strip at a time.
- **Drain the sink.** The water leaves in a **wave**, ring by ring; a **diver** follows one pipe to the bottom first; the **cheapest**, lowest path empties **first**; and the strainer **keeps the best** few pieces.
- **Branch the recipe. Enumerate** every variation and cross out each choice on the way back; follow a **path** through the cookbook, marking pages and unmarking them; **combine** what the sub-recipes return; **pass down** the guests' allergies;

and **stamp** each step on entry and exit.

- **Fill the table.** A **chain** of plates, each set from the last two; a **state vector** of glasses (full, empty, resting) copied forward each round; the picnic **knapsack** and its capacity; a **2-D** seating grid filled from one corner; **intervals** of the table filled short to long; and a **subsequence** of courses, each better than the one before.
- **Serve the guests.** The waiter's pad keeps a **map** of the tables and the **order** they came in; the coat check **hashes** coats to **slots**; families sit together around their **roots**; the menu is a **trie** of dish names; and the kitchen keeps a **tally** of every order.

The syllable names then say the steps, so a name recalls its code: *Poprexen* is POP, PRocess, EXpand, ENqueue. One line per family:

Family	Its methods, by syllable name
Sweep	UpKeep · Nelolo · Reacut · Sostre · PreSuf · RunPack
Hold	GroShriSco · PoAnPu · Pupore · Leri · Evadco
Squeeze	MeKeMo · Chazip · First True (stored order) · First True (the answer) · Pacha · Peel
Drain	Poprexen · Poprexpu · PoStaRePu · Putri
Branch	Rechexun · Marexun · Balerico · Bachepa · Grabla
Fill	Tasefire, set six ways: chain · Costep · knapsack · 2-D table · interval · subsequence
Serve	OpMapSync · Haprop · FiFiSaSu · WaCreStep · Cobuco

To make it stick, rebuild the tree from memory on a blank page: the seven verbs in kitchen order, the methods under each, then each method's steps and its invariant. Check against the tree above and fill in only what you missed. Do it today, tomorrow, in three days and in a week; each time the gaps get smaller, and the gaps are exactly what to read again.

Choosing at a blank page

Ask these in order and stop at the first yes.

1. Will the code answer **method calls over time**? **Serve.**
2. Is the big answer built from **answers to overlapping smaller inputs**? **Fill.**
3. Does it break into **pieces solved by recursion**: choices, children, halves? **Branch.**
4. Is there a **bag of pending items**, where I always take the next one? **Drain.**
5. Can each look **discard part of the candidates for good**? **Squeeze.**
6. In one pass, do I need **the live items themselves**? **Hold.**
7. Otherwise a **summary of the past** is enough. **Sweep.**

Then make the rest of the choices, before writing code:

Choice	What it decides
The shape of the answer	one optimum, a count, yes or no, all solutions, one answer per element
What the constraints allow	$n \le 20$: exponential; $n \le 10^4$: n^2; $n \le 10^5$: n log n; values to 10^9: bisect the value
What would make it easy	sort first, hash first, build the graph first
The property that licenses the method	contiguous and monotone (window), monotone test (bisect), unit cost (wave), no negative edges (cheapest first), overlapping subproblems (fill)
The dials	the bag and when a node is marked, what a state is, what is recorded, the base cases
The invariant	one sentence, said out loud; then write the code

Statements are written so the method is not named. Seeing through the disguise is the skill:

The statement says	The hidden model	The method
a lock of four wheels; fewest turns to the target (LC 752)	every code is a node with eight neighbours; every turn costs one	**Wave**: node = the 4-digit string, deadends marked before the start
the slowest eating speed that finishes in h hours (LC 875)	a faster speed never hurts: the test is monotone	**Bisect the answer** over speeds 1 .. max(piles)
fewest taps that water the whole garden (LC 1326)	each tap covers an interval	**Reach**, after turning taps into reaches
count subarrays that sum to k, negatives allowed (LC 560)	a window looks right, but negatives break monotonicity	**Lookback**: prefix sums in a map; the window is ruled out

The last row is the important one: choosing is as much ruling out as picking.

What the tree shows

Sorting by method instead of by topic puts together things that topic lists keep apart.

- **Fast power, Euclid's gcd and base conversion are the same squeeze as binary search.** Each step keeps a smaller part of a number.
- **KMP is a fold.** Its running value is how much of the pattern matches, with a fallback rule.
- **The hit counter, the rate limiter and the snake game are windows over time.** Expired events leave at the front.
- **The convex hull is the monotonic stack.** A point waits on the stack until a turn the wrong way pops it.
- **Bellman-Ford and the stock problems with cooldown are one method.** Each step computes a new vector of states from a copy of the previous one.
- **Kahn's algorithm is breadth-first search with an admission rule.** A node enters the queue when its indegree reaches zero.
- **Dijkstra, Prim and the k-way merge are one drain with a heap.** Only the key differs.
- **Design problems are services.** Every method call locates, changes and repairs.

Three families relate in a way worth stating once. **Hold** and **Drain** both pop from a container, but in Hold the input drives the loop and the container is memory, while in Drain the container drives the loop. **Branch** has four kinds, told apart by which way the information flows: enumerate (shared state, undone on the way back), combine (return values come up), pass down (arguments carry constraints) and stamp (entry and exit marks). Add a memo to *combine* and you are in **Fill**: dynamic programming is recursion without repetition.

How to practise

Cover the code. Read a problem statement and give, in ninety seconds and without code: the family, the method, the property that licenses it, the dials, and the invariant. Then read the "differs" line and check yourself. Mix problems from different families in one session: choosing between methods is the skill, and it only gets trained when the choice is open.

Sweep

One pass over the input; carry a summary of the past.

The question. What single summary of the past do I carry?

Picture. A pen moves left to right across the input and keeps one ledger line up to date. It never goes back: everything it needs about `items[0..i-1]` is on that line.

Shared skeleton.

```
state = start                    # dial: what the summary is
for i, x in enumerate(items):
    answer = use(state, x)       # read the past before x joins it
    state = update(state, x)     # fold x into the summary
```

Dials. What the summary is. One number (a tank, a vote, a match length) makes a fold; a map of past keys makes a lookback; the farthest edge seen so far makes a reach. Sorting first lays a set of intervals along a line the pen can sweep, holding only the last interval kept. When one pass cannot see both sides, two passes each carry half the summary. When the input is made of pieces (runs, tokens, lines), an inner loop walks to the end of a piece and the pen jumps there.

Method	Mnemonic	What it sets	Book entries
Fold	UpKeep	one running value, with a reset or fallback rule	#69, #86, #102, #131, #135
Lookback	Nelolo	a map from past keys to counts or positions	#1, #2, #4, #5, #96, #134
Reach	Reacut	the farthest index reached so far, and where to cut	#67, #68, #72
Sorted sweep	Sostre	the sort key, and the last interval kept	#26, #27, #65, #70, #112, #142
Two passes	PreSuf	a summary from each side, or mark then apply	#3, #89, #111, #124, #127, #129
Runs	RunPack	where the current piece ends	#88, #121, #122, #123

A sweep keeps a *summary*; **Hold** keeps the live items themselves (a stack, a window, a heap) because some of them must leave later. If the summary at `i` needs the answers at many earlier indices, not one running value, the problem has become a table: **Fill**. The family-wide trap is the order of the two lines in the loop: use the past before `x` joins it, or `x` pairs with itself (Two Sum). The other two traps sit at the ends of the pass. At the start, the summary needs its seed: without `{0: 1}` the empty prefix goes uncounted. At the end, a run, group or chunk still open when the loop stops must be flushed after it.

Fold · UpKeep

Update the running state, Keep the best.

Picture. One number rides along the input like a fuel gauge: every item moves it, and a rule says when it resets, falls back or hands over to a new candidate.

Invariant. At the top of step `i` the running value is the right summary of `items[0..i-1]`: the fuel since `start`, the surviving vote, the length of the pattern matched so far.

Use when. one pass and O(1) extra space; a running balance that must not go negative; a value that appears more than n / 2 times; pick uniformly from a stream of unknown length; every element appears twice except one; find a pattern without re-reading the text.

Book entries. #69 Gas Station · #86 KMP String Matching · #102 Boyer-Moore Majority Vote · #131 Reservoir Sampling · #135 Single Number

GENERAL SOLUTION

Problem. Stations stand on a circle; station `i` gives `gas[i]` and the road to the next one costs `cost[i]`. Return the station from which a full lap with an empty starting tank is possible, or -1: `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]` gives 3.

Intuition. The tank is the running value. When it goes negative at `i`, no station from `start` to `i` can be the start, because each of them was reached with a non-negative tank and still fails by `i`; so the candidate jumps to `i + 1` with an empty tank. If the total gas covers the total cost, the last candidate standing completes the lap.

```
i   gas-cost   tank   start
0   -2         -2     reset to 1
1   -2         -2     reset to 2
2   -2         -2     reset to 3
3   +3          3     3
4   +3          6     3       answer 3
```

```python
def can_complete_circuit(gas, cost):
    if sum(gas) < sum(cost):          # no start can finish the lap
        return -1
    start = 0                         # the best candidate so far
    tank = 0                          # fuel left since leaving start
    for i in range(len(gas)):
        tank += gas[i] - cost[i]      # Up: fold station i into the tank
        if tank < 0:                  # Keep: start..i all fail
            start = i + 1             # the next candidate
            tank = 0
    return start
```

Example.

```python
print(can_complete_circuit([1, 2, 3, 4, 5], [3, 4, 5, 1, 2]))   # 3
```

Dials. What the running value is (a tank, a candidate and its count, a match length, the item picked, an XOR), the rule that resets or falls back, and whether a second pass verifies the result.

Traps. Trusting the scan without the total check: it can end on a start that cannot finish the lap. A vote that is not guaranteed to exist must be counted in a second pass. In KMP the fallback is a `while`, not an `if`: one shorter border may fail too.

PROBLEMS

LC 134 · Gas Station

Problem: Stations stand on a circle: station `i` gives `gas[i]`, the drive to the next costs `cost[i]`. Return the station (unique if it exists) from which a full lap on an empty tank is possible, or -1: `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]` gives 3.

Intuition: A failed stretch condemns every start inside it: each was reached with a tank of at least zero and still ran dry, and starting there on an empty tank does no better. One pass discards whole stretches, and once the total gas covers the total cost, the survivor makes the lap.

Differs: nothing: the general solution is this problem. The dial on show is the reset: a negative tank moves `start` to `i + 1` and empties the tank.

LC 169 · Majority Element

Problem: Given an array `nums` of size n, return the element that appears more than ⌊n / 2⌋ times; it always exists: `[2,2,1,1,1,2,2]` gives 2.

Intuition: Pair off votes of different values and cancel them. The majority has more votes than all the others together, so it cannot be cancelled completely: it is the candidate left standing.

Differs: the running value is a candidate and its count. A count of 0 adopts `x`; a match adds one; any other value cancels one vote. No second pass: the majority is promised to exist.

```python
if count == 0:
    candidate = x
count += 1 if x == candidate else -1
```

LC 229 · Majority Element II

Problem: Given an integer array of size n, return all elements that appear more than `⌊n / 3⌋` times, in any order: `[3,2,3]` gives `[3]`, `[1,2]` gives `[1,2]`.

Intuition: At most two values can exceed n / 3. Cancel three different votes at a time: those values cannot be cancelled away, so they are among the two survivors. Surviving proves nothing about the count, so a recount decides.

Differs: two candidates with two counts. Test a match with either candidate first, then an empty slot, else decrement both. A second pass keeps the survivors with more than `n // 3` votes.

LC 28 · Find the Index of the First Occurrence in a String

Problem: Return the index of the first occurrence of `needle` in `haystack`, or -1: `haystack = "sadbutsad"`, `needle = "sad"` gives 0; `"leetcode"`, `"leeto"` gives -1.

Intuition: Slide through the text once, carrying how much of the needle matches right now. On a mismatch the longest border of the matched part (`lps`) is the longest shorter match still alive, so the text is never re-read: O(n + m).

Differs: build `lps` first with the same fold, `needle` against itself. Then `j`, the matched length ending at `haystack[i]`, falls back with `j = lps[j - 1]` on a mismatch; at `j == len(needle)` return `i - j + 1`, and -1 after the loop.

```
while j > 0 and haystack[i] != needle[j]:
    j = lps[j - 1]                    # fall back to a shorter border
if haystack[i] == needle[j]:
    j += 1
```

LC 459 · Repeated Substring Pattern

Problem: Given a string `s`, return true if it is a substring repeated two or more times: `"abab"` gives true, `"aba"` false, `"abcabcabcabc"` true.

Intuition: A border of length b means `s` equals itself shifted by n - b. The longest border gives the smallest shift p, and `s` is two or more whole copies of `s[:p]` exactly when p < n and p divides n.

Differs: run only the `lps` fold, on `s` itself. Let `p = n - lps[-1]`; the answer is `lps[-1] > 0 and n % p == 0`.

LC 382 · Linked List Random Node

Problem: Build a class from the head of a singly linked list whose `getRandom()` returns a node value, every node equally likely: for the list 1 → 2 → 3, each of 1, 2, 3 with chance 1 / 3.

Intuition: Let the i-th node take over the pick with chance 1 / i. It keeps the pick past each later node j with chance (j - 1) / j; the product telescopes, so every node ends with chance 1 / n. The length is never needed, and the extra space is O(1).

Differs: reservoir with k = 1 inside `getRandom`: walk the list, and at the i-th node (1-based) replace the pick with that node's value when `random.randint(1, i) == 1`.

```
i, node = 0, self.head
while node:
    i += 1
    if random.randint(1, i) == 1:     # chance 1 / i
        pick = node.val
    node = node.next
```

LC 136 · Single Number

Problem: In a non-empty array every element appears twice except one; return that one in linear time and constant extra space: `[4,1,2,1,2]` gives 4.

Intuition: XOR is a switch per bit: `x ^ x == 0`, `x ^ 0 == x`, and the order does not matter. Folding every number into one XOR cancels the pairs, wherever they sit, and leaves the single one.

Differs: the running value starts at 0 and folds each number with `^=`; there is no keep or reset step.

Lookback · Nelolo

NEed, LOok, LOg: look before you log.

Picture. The pen keeps a map of everything it has passed. At each item it says which past key would complete it, looks that key up, and only then writes the item's own key into the map.

Invariant. At the top of step `i` the map holds exactly the keys of `items[0..i-1]` (plus the seed), so every hit pairs `i` with an earlier position and no item pairs with itself.

Use when. a pair or a subarray with a given sum, difference or residue; negatives allowed, so a window will not shrink correctly; group items that are the same up to order; count points on one line; totals per key that arrive in pieces.

Book entries. #1 Two Sum · #2 Prefix Sum + Hash Map · #4 Longest Consecutive Sequence · #5 Group Anagrams · #96 Design Underground System · #134 Max Points on a Line

GENERAL SOLUTION

Problem. Given an integer array `nums` (negatives allowed) and an integer `k`, count the contiguous subarrays whose sum is `k`: `nums = [1,1,1]`, `k = 2` gives 2.

Intuition. A subarray `i..j` sums to `P[j] - P[i-1]`, the difference of two prefix sums. So at each prefix the pen needs an earlier prefix equal to `prefix - k`, and a map counts how many of those it has passed. Looking before logging keeps every pair strictly earlier-then-later.

```
x   prefix   need   hits   seen after
-   0        -      -      {0: 1}
1   1        -1     0      {0: 1, 1: 1}
1   2        0      1      {0: 1, 1: 1, 2: 1}
1   3        1      1      total 2
```

```python
def subarray_sum(nums, k):
    seen = {0: 1}                    # the empty prefix, before index 0
    prefix = 0
    total = 0
    for x in nums:
        prefix += x
        need = prefix - k            # Need: the earlier prefix that pairs
        total += seen.get(need, 0)   # Look: before this prefix is logged
        seen[prefix] = seen.get(prefix, 0) + 1   # Log
    return total
```

Example.

```python
print(subarray_sum([1, 1, 1], 2))                          # 2
```

Dials. The key (a value, a prefix sum, a residue, a letter count, a reduced slope, a route), what the map stores for it (a count, the first index, a list, a running total), and the seed (`{0: 1}` for counts, `{0: -1}` for first indices, none for pairs of items).

Traps. Logging before looking lets an item pair with itself. Forgetting the seed misses every subarray that starts at index 0. When the map stores a first index, overwriting it with a later one shortens every length measured from it.

PROBLEMS

LC 1 · Two Sum

Problem: Given `nums` and `target`, return the indices of the two numbers that add up to `target`; exactly one solution exists and one element may not be used twice: `[2,7,11,15]`, `target = 9` gives `[0,1]`.

Intuition: Each number knows its partner: `target - x`. A map of the values already passed answers "have I seen my partner?" in O(1), and looking before logging means the partner is always an earlier index, never the number itself.

Differs: the key is the value itself and the map stores its index; no seed. Need is `target - x`; the first hit returns `[seen[need], i]`.

```python
need = target - nums[i]
if need in seen:
    return [seen[need], i]
seen[nums[i]] = i
```

LC 560 · Subarray Sum Equals K

Problem: Given `nums` (values from -1000 to 1000) and `k`, return the number of non-empty contiguous subarrays that sum to `k`: `[1,2,3]`, `k = 3` gives 2 (`[1,2]` and `[3]`).

Intuition: Negative values mean growing a window can lower its sum, so a window has no monotone shrink rule. The difference of two prefix sums covers every subarray, and a count per prefix finds all partners at once.

Differs: nothing: the general solution is this problem. The seed `{0: 1}` counts the subarrays that start at index 0.

LC 974 · Subarray Sums Divisible by K

Problem: Given `nums` and `k >= 2`, return the number of non-empty subarrays whose sum is divisible by `k`: `[4,5,0,-2,-3,1]`, `k = 5` gives 7.

Intuition: A subarray sum is divisible by `k` exactly when its two prefixes leave the same remainder. Bucket the prefixes by remainder; every earlier prefix in the same bucket closes one subarray. Python's `%` is non-negative, so negative prefixes land in the right bucket.

Differs: the key is `prefix % k`, and Need is the same residue: `total += seen.get(r, 0)`. The seed is still `{0: 1}`.

LC 525 · Contiguous Array

Problem: Given a binary array `nums`, return the length of the longest contiguous subarray with an equal number of 0s and 1s: `[0,1,0]` gives 2.

Intuition: Read 0 as -1: an equal count means a sum of zero, which means two equal prefix sums. The farthest pair is the earliest index of a prefix and the current one, so each prefix keeps its first index forever.

Differs: the map stores the *first* index of each prefix, seeded `{0: -1}`; Need is the same prefix, the length is `i - first[prefix]`, and a prefix is logged only if absent.

```
prefix += 1 if x == 1 else -1
if prefix in first:
    best = max(best, i - first[prefix])
else:
    first[prefix] = i
```

LC 128 · Longest Consecutive Sequence

Problem: Given an unsorted integer array, return the length of the longest run of consecutive values, in O(n) time: `[100,4,200,1,3,2]` gives 4 (1, 2, 3, 4).

Intuition: O(n) rules out sorting, so put every value in a set. A value whose predecessor is absent starts a run; walk up from run starts only, and each value is touched at most twice.

Differs: log every value into a set first, then look over the set (duplicates would repeat a walk): skip a value whose `num - 1` is in the set; from a run start, walk up while `num + length` is in the set.

LC 49 · Group Anagrams

Problem: Given an array of lowercase words, group the anagrams together, in any order: `["eat","tea","tan","ate","nat","bat"]` gives `[["bat"],["nat","tan"],["ate","eat","tea"]]`.

Intuition: Two words are anagrams exactly when their letter counts match, so the letter count is a name shared by the whole group. A map from that name to a list collects each group in one pass.

Differs: the key is a word's 26 letter counts as a tuple (or its sorted letters), and the map stores the list of words with that key. Need and Look drop out: every word is logged.

LC 149 · Max Points on a Line

Problem: Given unique points `[x, y]` on a plane, return the largest number of them that lie on one straight line: `[[1,1],[2,2],[3,3]]` gives 3.

Intuition: Fix an anchor: every point on a line through it shares one direction from it. Count directions in a map; exact reduced integer fractions keep nearly equal slopes apart, where floats can merge them.

Differs: a fresh map per anchor, keyed by the reduced slope to each later point: flip signs so `dx > 0`, then `(dy // g, dx // g)` with `g = gcd(dy, dx)`; vertical is `(1, 0)`. The answer is the largest count plus one.

Reach · Reacut

extend the REAch, CUT at the edge.

Picture. A shadow stretches ahead of the pen: every index it passes pushes the far edge out, and when the pen catches up with the edge it cuts: one more jump, one more part, one more chunk.

Invariant. At the top of step `i` every index up to `level_end` is covered by the cuts made so far, and `farthest` is the edge one more cut can reach.

Use when. jump lengths and the last index; the fewest jumps, clips or taps that cover a range; split a string into as many parts as possible so each letter is in one part; the most chunks that can be sorted on their own.

Book entries. #67 Jump Game · #68 Jump Game II · #72 Partition Labels

GENERAL SOLUTION

Problem. `nums[i]` is the longest jump allowed from index `i`. Return the fewest jumps from index 0 to the last index, which is promised reachable: `[2,3,1,1,4]` gives 2 (0 → 1 → 4).

Intuition. The indices whose fewest jumps is `j` form one interval, because a jump may be any length up to `nums[i]`. The pen walks through the current interval and stretches the edge of the next one; when it reaches the end of the current interval it cuts, and that cut is one jump. It is a wave whose levels are intervals.

```
i   i+nums[i]   farthest   level_end   jumps
0   2           2          0 -> 2      1
1   4           4          2           1
2   3           4          2 -> 4      2
3   4           4          4           2     answer 2
```

```python
def min_jumps(nums):
    jumps = 0
    level_end = 0                    # edge of the current level
    farthest = 0                     # edge of the next level
    for i in range(len(nums) - 1):   # never jump from the last index
        farthest = max(farthest, i + nums[i])   # Rea: extend the reach
        if i == level_end:           # Cut: this level is used up
            jumps += 1
            level_end = farthest     # the next level ends here
    return jumps
```

Example.

```python
print(min_jumps([2, 3, 1, 1, 4]))                          # 2
```

Dials. What pushes the reach (`i + nums[i]`, the end of an interval that starts at `i`, the last copy of a letter, the largest value so far), what a cut does (count a jump, emit a part, count a chunk) and what happens when the reach stalls (return `False` or -1).

Traps. Looping to the last index in Jump Game II counts a jump too many whenever a level ends exactly there. In the covering problems a cut with `farthest <= i` means a gap: return -1 there, or the loop keeps counting cuts that cover nothing. Partition Labels needs the last index of every letter before the sweep starts.

PROBLEMS

LC 55 · Jump Game

Problem: Standing at index 0, `nums[i]` is the longest jump from `i`. Return true if the last index can be reached: `[2,3,1,1,4]` gives true, `[3,2,1,0,4]` gives false.

Intuition: Any jump length up to `nums[i]` is allowed, so the reachable indices always form one block `[0, farthest]`. Walk inside the block and stretch it; the first index outside it is a wall nothing can cross.

Differs: no cuts: walk `i` only while `i <= farthest`, and return `True` once `farthest` reaches the last index; walking past `farthest` returns `False`.

```
i = farthest = 0
while i <= farthest:
    farthest = max(farthest, i + nums[i])
    if farthest >= len(nums) - 1:
        return True
    i += 1
```

LC 45 · Jump Game II

Problem: Standing at index 0, `nums[i]` is the longest forward jump from `i`. Return the fewest jumps that reach the last index; the test data guarantees it is reachable: `[2,3,1,1,4]` gives 2 (0 → 1 → 4), `[2,3,0,1,4]` gives 2.

Intuition: The indices reachable in `j` jumps form an interval, so counting jumps is counting levels of a wave over intervals. Reachability is promised, so the reach never stalls and no -1 check is needed.

Differs: nothing: the general solution is this problem. The cut at `i == level_end` is one jump.

LC 1024 · Video Stitching

Problem: Clips `[start, end]` of a sporting event may overlap. Return the fewest clips whose union covers `[0, time]`, or -1: clips `[[0,2],[4,6],[8,10],[1,9],[1,5],[5,9]]`, `time = 10` give 3.

Intuition: From a covered prefix `[0, level_end]`, the best next clip is the one that starts inside it and ends farthest, which is exactly a jump. Storing for each start the farthest end turns the clips into a Jump Game II whose reach may stall at a gap.

Differs: first turn clips into reaches: `reach[s] = max(reach[s], min(e, time))` for every clip with `s <= time`. Then sweep `i` over `range(time)` with `farthest = max(farthest, reach[i])`; at a cut, `farthest <= i` returns -1.

LC 1326 · Minimum Number of Taps to Open to Water a Garden

Problem: A garden spans `[0, n]`; tap `i` (for `i` in `0..n`) waters `[i - ranges[i], i + ranges[i]]`. Return the fewest taps that water the whole garden, or -1: `n = 5`, `ranges = [3,4,1,1,0,0]` gives 1.

Intuition: Each tap is an interval, and covering `[0, n]` with the fewest intervals is the fewest jumps once every left end stores the farthest right end that starts there. A stalled reach is a dry patch.

Differs: clip each tap to `[0, n]` and record it as a reach, then run the sweep of LC 1024 over `range(n)`, returning -1 on a stalled cut.

```
reach = [0] * (n + 1)
for i, r in enumerate(ranges):
    lo = max(0, i - r)
    reach[lo] = max(reach[lo], min(n, i + r))
```

LC 763 · Partition Labels

Problem: Split string `s` into as many parts as possible so that each letter appears in at most one part; return the part sizes: `"ababcbacadefegdehijhklij"` gives `[9,7,8]`.

Intuition: A part must hold every copy of every letter in it, so it stretches to the last copy of each letter it meets. Cutting at the first index that nothing inside the part reaches past gives the most parts.

Differs: a first pass records `last[ch]`. One edge does both jobs: over every `i`, `part_end = max(part_end, last[s[i]])`, and the cut at `i == part_end` appends `i - start + 1` and sets `start = i + 1`.

LC 769 · Max Chunks To Make Sorted

Problem: `arr` is a permutation of `0..n-1`. Split it into the most chunks such that sorting each chunk and joining them gives the sorted array: `[1,0,2,3,4]` gives 4, `[4,3,2,1,0]` gives 1.

Intuition: A prefix ending at `i` can stand alone exactly when it holds the values `0..i`, which for a permutation is when its largest value is `i`. The running maximum is the reach; meeting it is a cut.

Differs: the reach is the largest value so far, `top = max(top, arr[i])`; the cut at `top == i` counts one chunk.

Sorted sweep · Sostre

SOrt, then STREtch the last or start a new one.

Picture. Sorting lays the intervals along one line; the pen walks them left to right holding only the last one kept, and each new interval either stretches it or opens the next one.

Invariant. At the top of each step the output is sorted and disjoint, and only its last interval can still meet the intervals not yet seen.

Use when. intervals, meetings, bookings, balloons, ranges; merge the overlapping ones; the fewest to remove or the most to keep; how many rooms at once; split cards into consecutive groups; track which ranges are covered.

Book entries. #26 Merge Intervals · #27 Meeting Rooms II · #65 Insert Interval · #70 Interval Scheduling · #112 Hand of Straights · #142 Range Module

GENERAL SOLUTION

Problem. Given intervals `[start, end]`, merge every overlapping pair (touching counts) and return the disjoint intervals that cover the same points: `[[1,3],[2,6],[8,10],[15,18]]` gives `[[1,6],[8,10],[15,18]]`.

Intuition. Sort by start and lay the intervals along a line. Every later interval starts at or after the last merged one, so if it misses `merged[-1]` it misses everything kept before it. The pen compares with the last one only: overlap stretches it, a gap opens a new one.

```
next      test          merged
[1,3]     empty         [[1,3]]
[2,6]     2 <= 3        [[1,6]]              stretch
[8,10]    8 > 6         [[1,6],[8,10]]       new
[15,18]   15 > 10       [[1,6],[8,10],[15,18]]
```

```python
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):          # So: by start
        if merged and start <= merged[-1][1]:     # overlaps the last one
            merged[-1][1] = max(merged[-1][1], end)   # Stre: stretch it
        else:
            merged.append([start, end])           # a gap: start a new one
    return merged
```

Example.

```python
intervals = [[1, 3], [2, 6], [8, 10], [15, 18]]
print(merge_intervals(intervals))         # [[1, 6], [8, 10], [15, 18]]
```

Dials. The sort key: by start to merge and cover, by end to keep as many as fit. What is held instead of `merged[-1]`: the end of the last kept interval, a heap of end times, a count of cards. Whether touching counts as overlapping (`<=` or `<`).

Traps. `max` in the stretch: a contained interval like `[2, 3]` inside `[1, 4]` must not shrink it. Sorting by start for "keep the most" picks a long early interval that blocks many short ones. Read whether touching endpoints overlap; each problem says so.

PROBLEMS

LC 56 · Merge Intervals

Problem: Given `intervals[i] = [start, end]` in any order, merge all overlapping intervals and return the non-overlapping ones that cover the same points: `[[1,3],[2,6],[8,10],[15,18]]` gives `[[1,6],[8,10],[15,18]]`, and the touching pair `[[1,4],[4,5]]` gives `[[1,5]]`.

Intuition: Sorted by start, an interval can only meet the last merged one, so one comparison per interval decides stretch or new. The sort costs O(n log n); the sweep is linear.

Differs: nothing: the general solution is this problem. Touching intervals merge, so the test is `start <= merged[-1][1]`.

LC 57 · Insert Interval

Problem: `intervals` is sorted by start and non-overlapping. Insert `newInterval`, merging where needed, and return the result still sorted and disjoint: `[[1,3],[6,9]]` with `[2,5]` gives `[[1,5],[6,9]]`.

Intuition: The line is already sorted, so the intervals that meet the new one form one contiguous run. Copy the ones wholly before it, swallow the run into it, then copy the rest: one O(n) pass instead of a sort.

Differs: no sort. Copy the intervals that end before `newInterval` starts, absorb with `min`/`max` every one that starts at or before its end, append it once, then copy the rest.

```
while i < n and intervals[i][0] <= new_end:
    new_start = min(new_start, intervals[i][0])
    new_end = max(new_end, intervals[i][1])
    i += 1
```

LC 253 · Meeting Rooms II

Problem: Given meeting times `[start, end]`, return the fewest conference rooms needed so no two meetings in one room overlap: `[[0,30],[5,10],[15,20]]` gives 2, `[[7,10],[2,4]]` gives 1.

Intuition: Walk the meetings by start time. The room that frees up first is the only one worth reusing; a new room opens only when every room is busy at `start`, so the number of rooms is the largest number of meetings running at once.

Differs: the held item becomes a min-heap of end times, one per room. Sorted by start: if `ends[0] <= start`, pop it; always push `end`. The answer is `len(ends)`.

```
for start, end in sorted(intervals):
    if ends and ends[0] <= start:
        heapq.heappop(ends)
    heapq.heappush(ends, end)
```

LC 435 · Non-overlapping Intervals

Problem: Return the fewest intervals to remove so the rest do not overlap; intervals that only touch, like `[1,2]` and `[2,3]`, do not overlap: `[[1,2],[2,3],[3,4],[1,3]]` gives 1.

Intuition: Keep as many as possible. The interval that ends first leaves the most room for the rest, so keeping it is never worse than any other choice; repeat on what still fits.

Differs: sort by end and hold `last_end`; keep an interval when `start >= last_end` and move `last_end` to its end. Return `len(intervals) - kept`.

LC 452 · Minimum Number of Arrows to Burst Balloons

Problem: Balloons are ranges `[x_start, x_end]` on the x-axis; a vertical arrow at `x` bursts every balloon with `x_start <= x <= x_end`. Return the fewest arrows that burst all: `[[10,16],[2,8],[1,6],[7,12]]` gives 2.

Intuition: The earliest-ending balloon must be hit at or before its end, and shooting exactly at its end bursts every balloon already started. Endpoints are inclusive, so touching balloons share one arrow.

Differs: sort by end; the held value is the arrow, shot at the end of the first balloon. A balloon with `start > arrow` needs a new arrow at its end.

LC 846 · Hand of Straights

Problem: Return true if the cards `hand` can be split into groups of exactly `groupSize` cards with consecutive values, each card used once: `[1,2,3,6,2,3,4,7,8]` with `groupSize = 3` gives true (`[1,2,3]`, `[2,3,4]`, `[6,7,8]`).

Intuition: The smallest card left cannot sit in the middle of a group, because nothing smaller remains to precede it, so it must start one. Consuming from the smallest value up makes every choice forced.

Differs: count the cards, sort the distinct values, and consume smallest first: `count[card]` groups start at `card`, so each of the next `groupSize` values must have that many copies left.

LC 715 · Range Module

Problem: Design a class tracking half-open ranges `[left, right)`: `addRange`, `removeRange`, and `queryRange` (is every number in the range tracked?). After `addRange(10,20)` and `removeRange(14,16)`, `queryRange(10,14)` is true and `queryRange(13,15)` false.

Intuition: Keep the tracked ranges as one sorted, disjoint list between calls. An update then meets one contiguous run of the list, as in LC 57, and a query is answered by the single range that starts at or before `left`.

Differs: the held list persists across calls: `addRange` is LC 57 with touching ranges merged, `removeRange` keeps the pieces left and right of the cut, `queryRange` bisects for the last start `<= left` and checks its end `>= right`.

Two passes · PreSuf

PREfix pass, SUFfix pass: two easy passes beat one clever one.

Picture. The pen crosses the input twice: once left to right writing what the left side says, once back right to left folding in what the right side says. Or: one pass marks, the next applies.

Invariant. After the first pass every cell holds the summary of one side (or a mark that does not hide the old value); the second pass adds the other side without undoing the first.

Use when. every element needs the rest of the array, both sides (without division); a rule about both neighbours; all cells change at the same moment, in place; reverse the order of words; rotate in place; count the primes below n.

Book entries. #3 Product of Array Except Self · #89 Game of Life · #111 Candy · #124 Reverse Words in a String · #127 Rotate Image · #129 Sieve of Eratosthenes

GENERAL SOLUTION

Problem. Given `nums`, return `answer` where `answer[i]` is the product of every element except `nums[i]`, in O(n) and without division: `[1,2,3,4]` gives `[24,12,8,6]`.

Intuition. Everything except `nums[i]` is two pieces: what lies left of `i` and what lies right of it. One pass cannot see both sides at once, but each pass sees one side perfectly. The left pass writes the left product into `answer[i]`; the right pass multiplies in the right product.

```
i            0    1    2    3
nums         1    2    3    4
after Pre    1    1    2    6     product of the left side
suffix       24   12   4    1     product of the right side
after Suf    24   12   8    6
```

```python
def product_except_self(nums):
    n = len(nums)
    answer = [1] * n
    prefix = 1                          # product of nums[0..i-1]
    for i in range(n):                  # Pre: left to right
        answer[i] = prefix
        prefix *= nums[i]               # nums[i] joins after its use
    suffix = 1                          # product of nums[i+1..n-1]
    for i in range(n - 1, -1, -1):      # Suf: right to left
        answer[i] *= suffix
        suffix *= nums[i]
    return answer
```

Example.

```
print(product_except_self([1, 2, 3, 4]))                    # [24, 12, 8, 6]
```

Dials. What the two passes are: left then right (products, candies), mark then apply (next states in a spare bit, zero flags, crossed-out multiples), or flip all then flip each piece (reverse the string then every word; transpose then reverse every row).

Traps. Letting `nums[i]` into the prefix before writing `answer[i]` includes the element itself. Applying a change during the marking pass lets the change spread to cells it should not reach. In the second pass of Candy, overwriting instead of taking `max` breaks the left rule.

PROBLEMS

LC 238 · Product of Array Except Self

Problem: Given `nums`, return `answer` where `answer[i]` is the product of all elements except `nums[i]`, in O(n) time and without division: `[1,2,3,4]` gives `[24,12,8,6]` and `[-1,1,0,-3,3]` gives `[0,0,9,0,0]`. The follow-up asks for O(1) extra space besides the output.

Intuition: `answer[i]` is the left product times the right product, and each pass sees exactly one side. No division means a zero needs no special case.

Differs: nothing: the general solution is this problem. The output array holds the prefix products, so the extra space is O(1).

LC 135 · Candy

Problem: Children stand in a line with `ratings`. Each gets at least one candy, and a child rated higher than a neighbour gets more than that neighbour. Return the fewest candies: `[1,0,2]` gives 5 (2, 1, 2).

Intuition: The rule is two one-way rules: beat the left neighbour if rated higher, beat the right one if rated higher. A left pass settles the first, a right pass the second; taking `max` in the second keeps the first true, and every value stays the least possible.

Differs: start everyone at 1. Left pass: a higher rating than the left neighbour gets `candies[i - 1] + 1`. Right pass: a higher rating than the right neighbour gets `max(candies[i], candies[i + 1] + 1)`. Return the sum.

```
for i in range(n - 2, -1, -1):
    if ratings[i] > ratings[i + 1]:
        candies[i] = max(candies[i], candies[i + 1] + 1)
```

LC 289 · Game of Life

Problem: On a 0/1 board, a live cell with 2 or 3 live neighbours (of 8) lives, others die; a dead cell with exactly 3 comes alive. Update the board in place, all cells at once: `[[1,1],[1,0]]` becomes `[[1,1],[1,1]]`.

Intuition: All cells change at the same moment, so every count must see the old board. Write the next state into a spare bit: bit 0 stays the old state that neighbours read, bit 1 holds the new one, and a final pass promotes it.

Differs: mark then apply. Pass one counts live neighbours from bit 0 and writes the next state into bit 1 (`board[r][c] |= 2`); pass two shifts every cell right by one.

LC 73 · Set Matrix Zeroes

Problem: In an m x n matrix, if a cell is 0, set its whole row and column to 0, in place: `[[1,1,1],[1,0,1],[1,1,1]]` becomes `[[1,0,1],[0,0,0],[1,0,1]]`.

Intuition: Zeroing during the scan creates zeros that were not in the input, and they would wipe more rows and columns. Freeze the original zeros as marks first, then apply them in a second pass.

Differs: mark then apply. Pass one records the rows and columns that hold a zero; pass two zeroes every cell whose row or column is marked. For O(1) space, keep the marks in row 0 and column 0, plus one flag for column 0.

```
rows = {r for r in range(m) for c in range(n) if matrix[r][c] == 0}
cols = {c for r in range(m) for c in range(n) if matrix[r][c] == 0}
for r in range(m):
    for c in range(n):
        if r in rows or c in cols:
            matrix[r][c] = 0
```

LC 151 · Reverse Words in a String

Problem: Reverse the order of the words in `s`; the result has single spaces between words and none at the ends: `"the sky is blue"` gives `"blue is sky the"`, `"  hello world  "` gives `"world hello"`.

Intuition: Reversing the whole string puts the words in the right order but spells each one backwards; reversing each word again fixes the spelling. Two easy flips replace one hard rearrangement.

Differs: flip all then flip each: squeeze the spaces to single ones, reverse the whole character array, then reverse each word back. In Python, `" ".join(reversed(s.split()))` does the same.

LC 48 · Rotate Image

Problem: Rotate an n x n matrix 90 degrees clockwise, in place, without a second matrix: `[[1,2,3],[4,5,6],[7,8,9]]` becomes `[[7,4,1],[8,5,2],[9,6,3]]`.

Intuition: A clockwise turn sends `(r, c)` to `(c, n - 1 - r)`. That is two reflections: the transpose sends `(r, c)` to `(c, r)`, and reversing each row sends it on to `(c, n - 1 - r)`. Both are swaps, so the work stays in place.

Differs: flip all then flip each: transpose (swap `matrix[r][c]` with `matrix[c][r]` for `c > r`), then reverse every row.

LC 204 · Count Primes

Problem: Given an integer `n`, return the number of primes strictly less than `n`: `n = 10` gives 4 (2, 3, 5, 7).

Intuition: Every composite below `n` has a prime factor at most its square root, so crossing out the multiples of each small prime leaves exactly the primes. Multiples below `p * p` were already crossed by a smaller prime, so crossing starts at `p * p`.

Differs: mark then collect: for each `p` with `p * p < n` still uncrossed, cross out `p * p, p * p + p, ...` below `n`; then count the uncrossed values from 2 to `n - 1`.

```
crossed = [False] * n
for p in range(2, int(n ** 0.5) + 1):
    if not crossed[p]:
        for multiple in range(p * p, n, p):
            crossed[multiple] = True
```

Runs · RunPack

RUN to the end of the piece, PACK it, jump.

Picture. The pen stops at the start of a piece, an inner finger runs ahead to where the piece ends, the piece is packed into the output, and the pen jumps to where the finger stopped.

Invariant. At the top of the outer loop `read` stands on the first character of a piece, and everything before it has been packed exactly once.

Use when. runs of equal characters; tokens with a length or a separator; lines filled with as many words as fit; columns compared across strings; read a sequence aloud as counts of digits.

Book entries. #88 Encode and Decode Strings · #121 Longest Common Prefix · #122 Text Justification · #123 String Compression

GENERAL SOLUTION

Problem. Compress a list of characters in place: each run of equal characters becomes the character, followed by its count when the count is more than 1 (a count of 12 takes two cells); return the new length. `["a","a","b","b","c","c","c"]` gives 6, starting `["a","2","b","2","c","3"]`.

Intuition. The input is a row of pieces. A read finger runs from the start of a run to its end, the write finger packs the run behind it, and the outer loop jumps to where the read finger stopped. A run of `k > 1` cells packs into `1 + len(str(k)) <= k` cells, so writing never overtakes reading.

```
run     read     packed   write
"aa"    0 -> 2   a 2      2
"bb"    2 -> 4   b 2      4
"ccc"   4 -> 7   c 3      6     answer 6
```

```python
def compress(chars):
    write = 0
    read = 0
    while read < len(chars):
        start = read
        while read < len(chars) and chars[read] == chars[start]:
            read += 1                 # Run: walk to the end of the run
        chars[write] = chars[start]   # Pack: the character ...
        write += 1
        count = read - start
        if count > 1:
            for digit in str(count):  # ... and its count, digit by digit
                chars[write] = digit
                write += 1
    return write                      # jump: read already stands on the next
```

Example.

```python
chars = ["a", "a", "b", "b", "c", "c", "c"]
print(compress(chars))                    # 6
chars = ["a"] + ["b"] * 12
```

Dials. What ends a piece (a different character, a `#` after a length, a word that no longer fits, a mismatch in a column, the end of a digit run) and what packing writes (character and count, the payload, a padded line, the prefix so far).

Traps. Forgetting the last piece when the inner loop runs to the end of the input; with the outer `while read < len(...)` shape it is packed inside the loop, not after it. A count of 12 is two characters. Moving the outer index by one instead of jumping re-reads the piece.

PROBLEMS

LC 271 · Encode and Decode Strings

Problem: Design `encode`, which turns a list of strings into one string, and `decode`, which recovers the list exactly; the strings may hold any of the 256 ASCII characters: `["Hello","World"]` must come back as `["Hello","World"]`.

Intuition: Any separator could appear inside a string, so none is safe. A length prefix says where a piece ends without looking at its content: read the number up to `#`, take exactly that many characters, jump past them.

Differs: encode packs each string as `str(len(s)) + "#" + s`. Decode runs to the first `#` to read the length, packs exactly that many characters, and jumps `i = j + 1 + length`.

```python
j = i
while data[j] != "#":
    j += 1
length = int(data[i:j])
strs.append(data[j + 1:j + 1 + length])
i = j + 1 + length
```

LC 14 · Longest Common Prefix

Problem: Return the longest common prefix of an array of strings, or `""`: `["flower","flow","flight"]` gives `"fl"`, `["dog","racecar","car"]` gives `""`.

Intuition: Read the words column by column. The common prefix ends at the first column where some word differs or has run out, so the scan stops right after the answer and never reads further.

Differs: the piece is a column: for `col` in `range(len(strs[0]))`, run over every word and return `strs[0][:col]` at the first word that has ended or differs; return `strs[0]` if none does.

LC 68 · Text Justification

Problem: Pack `words` greedily into lines of exactly `maxWidth` characters, spreading spaces evenly, extra ones to the left gaps; the last line is left-justified. With `maxWidth = 16`, `["This","is","an","example","of","text","justification."]` gives `["This    is    an","example  of text","justification.  "]`.

Intuition: Greedy is the rule: each line takes as many words as fit with single spaces. Each word is at most `maxWidth` long, so every line takes at least one word and the jump always moves forward; packing a line only distributes its blanks.

Differs: the piece is a line: run `j` while `width + 1 + len(words[j]) <= maxWidth`. Pack `blanks = maxWidth - letters` as `blanks // gaps` per gap, plus one in the first `blanks % gaps`; the last line and one-word lines go left-justified, padded on the right.

LC 1592 · Rearrange Spaces Between Words

Problem: Rearrange the spaces of `text` (at least one word) so all gaps between words are equal and as large as possible, leftovers at the end, same total length: `"  this   is  a sentence "` gives `"this   is   a   sentence"`.

Intuition: Where the blanks sat does not matter, only how many there are. Divide them among the `len(words) - 1` gaps and put the remainder at the end. One word has no gaps, the one case that would divide by zero.

Differs: the pieces are the words, `text.split()`, and the blanks are counted with `text.count(" ")`. Pack them with `spaces // (len(words) - 1)` blanks between words and the remainder at the end; one word gets every blank after it.

```
if len(words) == 1:
    return words[0] + " " * spaces
gap, extra = divmod(spaces, len(words) - 1)
return (" " * gap).join(words) + " " * extra
```

LC 443 · String Compression

Problem: Compress `chars` in place with O(1) extra space: each run of equal characters becomes the character, then its length if above 1, one digit per cell; return the new length. `["a","a","b","b","c","c","c"]` gives 6 (`a 2 b 2 c 3`); `"a"` then twelve `"b"` gives 4 (`a b 1 2`).

Intuition: A run packs into no more cells than it occupies, so the write finger trails the read finger and the output can overwrite the input it has already read.

Differs: nothing: the general solution is this problem. Writing over `chars` while reading it is the dial on show.

LC 38 · Count and Say

Problem: `countAndSay(1) = "1"`; each later term reads the previous one aloud run by run, count before digit. Given `n` (1 to 30), return term `n`: `"1"`, `"11"`, `"21"`, `"1211"`, so `n = 4` gives `"1211"`.

Intuition: Each term is the run-length encoding of the previous one, so one round is one run-pack sweep that writes `count + digit` per run. Repeat it `n - 1` times; `n <= 30` keeps the rounds small.

Differs: start from `"1"` and repeat `n - 1` times: each run of equal digits packs as `str(count) + digit` into a new string, which becomes the input of the next round.

```
while i < len(say):                  # one round, out = [] and i = 0
    j = i
    while j < len(say) and say[j] == say[i]:
        j += 1                       # Run: to the end of the digit run
    out.append(str(j - i) + say[i])  # Pack: count, then digit
    i = j                            # jump
```

Hold

One pass over the input; keep the live items themselves.

The question. Which live items do I keep on hand, and when do they leave?

Picture. A tray rides along the input. Each new item is put on the tray, and the items it makes useless, or that have grown too old or been closed, are thrown out first. The answer is read off the tray, or off the items as they leave it.

Shared skeleton.

```
held = container()                      # dial: window, stack, deque, ring
for x in items:                         # the input drives the loop
    while held and must_leave(held, x): # dial: the leave rule
        settle(held.pop())              # a leaving item is answered or dropped
    held.add(x)                         # the newcomer waits its turn
    answer(held)                        # read the answer off the tray
```

Dials. The container (a span between two indices, a stack, a deque, two stacks, a ring of buckets) and the leave rule: the window breaks its condition or passes size k, a newcomer beats the item, a closer matches the opener, the cursor moves past it, the item is too old.

Method	Mnemonic	What it sets	Book entries
Window	GroShriSco	items between left and right; they leave when the rule breaks or k is passed	#11, #12, #13, #53, #54, #120
Ratchet	PoAnPu	a monotone stack or deque; items leave when a newcomer beats them	#23, #62, #64, #132, #133
Nesting	Pupore	a stack of open contexts; an opener leaves when its closer arrives	#22, #24, #25, #63
Cursor stacks	Leri	two stacks meeting at a cursor; moving pops one side and pushes the other	#50, #95, #144
Time ring	Evadco	a queue or ring in time order; events leave when they are too old	#45, #99, #110, #138

The invariant of every method is the same sentence: the container holds exactly the items still relevant. Sweep keeps one number about the past; Hold keeps the items, because a later item can still settle them or push them out. Drain also pops from a container, but there the container drives the loop and items are added as work is discovered; here the input drives the loop and the container is memory. The family-wide trap is the leave test: `if` where `while` belongs leaves stale items on the tray, and an item that leaves too early takes a right answer with it.

Window · GroShriSco

GROw right, SHRInk left, SCOre: both edges only move forward.

Picture. An elastic band slides along the array: the right edge takes one item per step, and the left edge follows only to repair the band, letting items fall out behind it.

Invariant. The counts describe exactly `s[left..right]`, and after Shrink it is the longest valid window that ends at `right`.

Use when. longest or shortest substring or subarray, contiguous, at most k distinct, at most k replacements or flips, sum at least target with positive numbers, every substring of length k, anagram or permutation of a pattern inside a string.

Book entries. #11 Longest Substring Without Repeating Characters · #12 Find All Anagrams in a String · #13 Minimum Window Substring · #53 Longest Repeating Character Replacement · #54 Minimum Size Subarray Sum · #120 Rabin-Karp

GENERAL SOLUTION

Problem. Given a string `s`, return the length of its longest substring with no repeated character: `"pwwkew"` gives 3, for `"wke"`.

Intuition. The right edge takes one character per step. When the newcomer repeats, the left edge walks forward until the earlier copy has fallen out, and then the band is the longest valid window ending at `right`. Shrinking is a safe repair because the rule survives it: every piece of a valid window is valid. Neither edge ever moves back, so the work is linear.

```
right  ch   window after Shrink   best
0      p    p                     1
1      w    pw                    2
2      w    w     (p, w dropped)  2
3      k    wk                    2
4      e    wke                   3
5      w    kew   (w dropped)     3
```

```python
def longest_window(s):
    count = {}                          # what the window holds
    left = 0
    best = 0
    for right in range(len(s)):
        ch = s[right]
        count[ch] = count.get(ch, 0) + 1  # Grow: s[right] joins on the right
        while count[ch] > 1:              # Shrink: while the rule is broken
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)  # Score: the window is valid here
    return best
```

Example.

```python
print(longest_window("pwwkew"))           # 3
```

Dials. What the window counts (characters, zeros, a sum, a top count), the broken rule, and the order. Longest: Grow, Shrink while broken, then Score. Shortest: Grow, then while valid Score and Shrink. Fixed size k: Grow at `right`, Shrink exactly `s[right - k]`, Score; no `while` at all, and the counts may be replaced by a rolling hash (#120: subtract the leading digit, multiply, add).

Traps. `if` instead of `while` on the shrink leaves a broken window scored. Scoring a longest window before the shrink records a broken one; scoring a shortest one after the shrink misses it. A window needs a rule that moves one way as it grows: with negative numbers it is ruled out.

PROBLEMS

LC 3 · Longest Substring Without Repeating Characters

Problem: Given a string `s` of letters, digits, symbols and spaces, return the length of the longest substring without repeating characters: `"abcabcbb"` gives 3, for `"abc"`.

Intuition: This is the general solution, with a faster repair. A repeat is fixed only by dropping its earlier copy and everything before it, and the index of that copy says where to cut. An index left of the window is stale and must not pull `left` back.

Differs: Shrink jumps instead of stepping: keep `last_seen[ch]`, and if it is at least `left`, set `left = last_seen[ch] + 1`. No counts.

```python
if ch in last_seen and last_seen[ch] >= left:
    left = last_seen[ch] + 1              # jump past the earlier copy
last_seen[ch] = right
```

LC 340 · Longest Substring with At Most K Distinct Characters

Problem: Given a string `s` and an integer `k`, return the length of the longest substring that contains at most `k` distinct characters: `s = "eceba"`, `k = 2` gives 3, for `"ece"`.

Intuition: A window with at most `k` distinct characters stays valid as it shrinks, so the band grows freely and the left edge walks forward whenever a `k + 1`-th character enters. The distinct count is the size of the count map, provided a character leaves the map when its last copy goes.

Differs: the broken rule is `len(count) > k`; on Shrink, delete a key when its count reaches 0.

```
while len(count) > k:
    count[s[left]] -= 1
    if count[s[left]] == 0:
        del count[s[left]]
    left += 1
```

LC 424 · Longest Repeating Character Replacement

Problem: Given a string `s` of uppercase letters and an integer `k`, you may change at most `k` characters. Return the length of the longest substring of one repeated letter you can get: `s = "AABABBA"`, `k = 1` gives 4.

Intuition: The cheapest repair keeps the most frequent letter of the window and replaces the rest, so a window is valid when its width minus its top count is at most `k`. A stale top count only holds the width; the answer grows only when a real count beats it.

Differs: count the 26 letters and keep `max_count`, the top count seen so far; the window is broken when `right - left + 1 - max_count > k`. `max_count` is never lowered.

LC 1004 · Max Consecutive Ones III

Problem: Given a binary array `nums` and an integer `k`, return the most consecutive 1s you can get by flipping at most `k` zeros: `[1,1,1,0,0,0,1,1,1,1,0]` with `k = 2` gives 6.

Intuition: Flipping a zero is the only cost, so a window with at most `k` zeros turns entirely into ones. Shrinking never adds a zero, so the longest-window order holds.

Differs: count only the zeros inside the window; it is broken when `zeros > k`.

LC 76 · Minimum Window Substring

Problem: Given strings `s` and `t`, return the shortest substring of `s` that contains every character of `t`, duplicates included, or `""` if none does: `s = "ADOBECODEBANC"`, `t = "ABC"` gives `"BANC"`.

Intuition: Covering `t` survives growing, not shrinking, so the order flips: grow until the window covers, then score and shrink while it still covers. A counter of characters still missing makes the cover test constant time; spare copies must not lower it.

Differs: shortest order. `need[ch]` counts copies still lacking and `missing` the characters of `t` uncovered; a character lowers `missing` only if its `need` was positive. Return `""` if nothing was scored.

```
while missing == 0:                      # valid: Score, then Shrink
    best = min(best, (right - left + 1, left))
    need[s[left]] += 1
    if need[s[left]] > 0:
        missing += 1                     # dropped a copy that was needed
    left += 1
```

LC 209 · Minimum Size Subarray Sum

Problem: Given an array of positive integers `nums` and a positive integer `target`, return the length of the shortest subarray whose sum is at least `target`, or 0 if none: `target = 7`, `nums = [2,3,1,2,4,3]` gives 2, for `[4,3]`.

Intuition: Every value is positive, so the sum rises as the window grows and falls as it shrinks. That one-way motion licenses the window: once a window reaches `target`, shrink it from the left as long as it still does, scoring each length.

Differs: shortest order with a running sum: add `nums[right]`; while the sum is at least `target`, Score the length, subtract `nums[left]`, advance `left`. Return 0 if nothing scored.

LC 438 · Find All Anagrams in a String

Problem: Given strings `s` and `p` of lowercase letters, return every start index of a substring of `s` that is an anagram of `p`: `s = "cbaebabacd"`, `p = "abc"` gives `[0,6]`.

Intuition: An anagram has exactly `len(p)` letters, so only one width qualifies and both edges move in lockstep. A slide changes two letter counts, and comparing 26 counts is constant time.

Differs: fixed size `m = len(p)`: Grow adds `s[right]`, Shrink removes `s[right - m]` once `right >= m`, Score compares the 26 counts with `p`'s and records `right - m + 1`.

```
window[ord(s[right]) - 97] += 1           # Grow
if right >= m:
    window[ord(s[right - m]) - 97] -= 1   # Shrink: fell past the width
if right >= m - 1 and window == need:     # Score
    starts.append(right - m + 1)
```

Ratchet · PoAnPu

POp what the newcomer settles, ANswer it, PUsh the newcomer.

Picture. Items wait on a staircase that only steps down. A newcomer knocks off every item it beats, each knocked-off item takes the newcomer as its answer, and the newcomer stands at the end.

Invariant. The stack holds, in input order, exactly the items no later item has beaten yet, so their values never rise from bottom to top.

Use when. next greater or smaller, previous greater, how many days until, span, nearest wall, largest rectangle, the maximum of every window of size k, a fence around points.

Book entries. #23 Monotonic Stack · #62 Largest Rectangle in Histogram · #64 Sliding Window Maximum · #132 Orientation and Segment Intersection · #133 Convex Hull

GENERAL SOLUTION

Problem. Given an array `nums`, return for every element the first strictly greater value to its right, or -1 if there is none: `[2, 1, 2, 4, 3]` gives `[4, 2, 4, -1, -1]`.

Intuition. An element waits until something bigger arrives, and the first bigger arrival is its answer. Waiting elements always form a staircase that steps down, because a newcomer clears every smaller one before it stands in line. Each element is pushed once and popped once, so the pass is linear. The stack holds indices, so a popped item knows where to write its answer.

```
i  x   settled by x        stack after (values)
0  2   -                   [2]
1  1   -                   [2, 1]
2  2   1 -> 2              [2, 2]
3  4   2 -> 4, 2 -> 4      [4]
4  3   -                   [4, 3]     (4 and 3 keep -1)
```

```
def next_greater(nums):
    answer = [-1] * len(nums)             # -1 stays for items nothing beats
    stack = []                            # waiting indices, values never rising
    for i in range(len(nums)):
        while stack and nums[stack[-1]] < nums[i]:
            j = stack.pop()               # Pop: the newcomer settles j
            answer[j] = nums[i]           # Answer: j's next greater value
        stack.append(i)                   # Push: i waits for its own answer
    return answer
```

Example.

```
print(next_greater([2, 1, 2, 4, 3]))      # [4, 2, 4, -1, -1]
```

Dials. The direction of the staircase (pop on `<` for next greater, on `>` for next smaller), what the answer is (a value, a distance, a width times a height), whether the answer goes to the popped item or to the newcomer, and the container: a deque adds a second exit at the front for items that expire. The convex hull pops on a turn instead of a comparison.

Traps. Pushing values instead of indices loses the distance and the expiry. `<=` instead of `<` answers an item with an equal one. Items still on the stack at the end never met their answer; leave the default or flush them with a sentinel.

PROBLEMS

LC 739 · Daily Temperatures

Problem: Given daily `temperatures`, return for each day how many days you wait for a warmer one, or 0 if none comes: `[73,74,75,71,69,72,76,73]` gives `[1,1,4,2,1,1,0,0]`.

Intuition: The general solution, asking "when" instead of "what". A cold day waits on the staircase until the first warmer day knocks it off, and the gap between their indices is the wait. An equally warm day is not warmer, so it does not settle anyone.

Differs: Answer writes the distance `i - j` instead of the value, and the default is 0.

LC 503 · Next Greater Element II

Problem: Given a circular array `nums`, where the element after the last is the first, return each element's next greater number in circular order, or -1: `[1,2,1]` gives `[2,-1,2]`.

Intuition: An element's next greater may sit before it, reached by wrapping around. A second lap shows every item still waiting the elements before it, so by its end each has seen the whole circle. Nothing is pushed in the second lap: every element already waits once.

Differs: walk `i` over `range(2 * n)` and read `nums[i % n]`; Push only while `i < n`.

```
for i in range(2 * n):
    while stack and nums[stack[-1]] < nums[i % n]:
        answer[stack.pop()] = nums[i % n]
    if i < n:
        stack.append(i)
```

LC 901 · Online Stock Span

Problem: Design `StockSpanner`: `next(price)` returns today's span, the number of consecutive days up to and including today with price at most today's. Prices 100, 80, 60, 70, 60, 75, 85 give spans 1, 1, 1, 2, 1, 4, 6.

Intuition: A day that today beats can never stop a later span that reaches today, so it folds into today's span and leaves. The top left standing is the previous higher price, where the span ends. Each day is pushed and popped once, so `next` is amortised constant.

Differs: a stream, and the answer goes to the newcomer: the stack holds `(price, span)`; pop while the top price is at most today's, adding each popped span to today's; push today.

```
def next(self, price):
    span = 1
    while self.stack and self.stack[-1][0] <= price:
        span += self.stack.pop()[1]
    self.stack.append((price, span))
    return span
```

LC 84 · Largest Rectangle in Histogram

Problem: Given `heights` of bars of width 1 standing side by side, return the area of the largest rectangle inside the histogram: `[2,1,5,6,2,3]` gives 10, the bars 5 and 6 at height 5.

Intuition: The best rectangle at a bar's height spreads until the first shorter bar on each side. When a shorter newcomer pops a bar, both walls are known: the newcomer on the right, the bar below it on the stack on the left. A final 0 bar is shorter than everything and flushes the stack.

Differs: heights never fall on the stack; pop while the top bar is taller than the newcomer, Answer with `height * (i - left)`, `left` one past the new top (0 if empty). Append a 0 sentinel.

```
while stack and bars[stack[-1]] > bars[i]:
    height = bars[stack.pop()]
    left = stack[-1] + 1 if stack else 0
    best = max(best, height * (i - left))
stack.append(i)
```

LC 85 · Maximal Rectangle

Problem: Given a `rows x cols` matrix of the characters "0" and "1", return the area of the largest rectangle containing only ones: `[["1","1","0"],["1","1","1"]]` gives 4.

Intuition: Every rectangle of ones rests on some bottom row. Seen from that row, the runs of ones above each column are the bars of a histogram, and the rectangle is a histogram rectangle. So one LC 84 per row covers every rectangle.

Differs: per row, `heights[c]` becomes `heights[c] + 1` when `matrix[r][c] == "1"`, else 0; run LC 84 on `heights` once per row and keep the best.

LC 239 · Sliding Window Maximum

Problem: Given `nums` and a window size `k`, return the maximum of every window of `k` consecutive elements as it slides from left to right: `[1,3,-1,-3,5,3,6,7]` with `k = 3` gives `[3,3,5,5,6,7]`.

Intuition: An older item no larger than a newcomer leaves the window first, so it can never be a maximum again and is thrown out at once. The survivors fall from front to back, so the front is the maximum; only the front can grow too old.

Differs: a deque of indices: pop the back while it is no larger than `nums[i]`, push `i`, drop the front if it is `<= i - k`, and from `i >= k - 1` Answer `nums[front]`.

```
while window and nums[window[-1]] <= nums[i]:
    window.pop()
window.append(i)
if window[0] <= i - k:
    window.popleft()
```

LC 587 · Erect the Fence

Problem: Given the points `trees[i] = [x, y]` of a garden, fence them all with the shortest rope and return the trees exactly on the fence, edges included: `[[1,2],[2,2],[4,2]]` gives all three.

Intuition: Walk the sorted trees; the hull is the staircase of left turns, and a right turn means the middle tree is inside, so the newcomer settles it. The cross product of #132 tells the turn. Trees on an edge make a straight turn and stay.

Differs: sort; build the lower chain, then the upper over the reversed list, popping only while `cross(chain[-2], chain[-1], p) < 0`, not `<= 0` as in #133. Return the set of both chains: their end trees, and every tree when all are collinear, appear in both.

```
while len(chain) >= 2 and cross(chain[-2], chain[-1], p) < 0:
    chain.pop()                          # a right turn: p settles it
chain.append(p)
```

Nesting · Pupore

PUsh on open, POp on close, REsolve: the last level opened is the first one closed.

Picture. Every opener leaves a bookmark on a stack; a closer always answers the newest bookmark, so the stack is the path from the outermost open level down to the innermost.

Invariant. The stack holds exactly the contexts opened and not yet closed, innermost on top, each with what its level needs to resume.

Use when. brackets, parentheses, valid, balanced, well-formed, nested, encoded `k[...]`, evaluate an expression, calculator, undo to the enclosing level, a stack that also reports its minimum.

Book entries. #22 Valid Parentheses · #24 Min Stack · #25 Decode String · #63 Basic Calculator

GENERAL SOLUTION

Problem. Given a string with brackets of three kinds, `()`, `[]` and `{}`, among other characters, return a map from each opener's index to its closer's index, or `None` if the brackets do not nest: `"(a[b]{c})"` gives `{0: 8, 2: 4, 5: 7}`.

Intuition. Brackets nest like boxes inside boxes: the box opened last must close first. So every opener waits on a stack, and a closer always belongs to the opener on top. A closer of the wrong kind, a closer with nothing open, or an opener still waiting at the end means the string does not nest.

```
i  ch   stack (indices)   resolved
0  (    [0]
2  [    [0, 2]
4  ]    [0]               2 -> 4
5  {    [0, 5]
7  }    [0]               5 -> 7
8  )    []                0 -> 8
```

```python
def match_brackets(s):
    partner = {")": "(", "]": "[", "}": "{"}  # closer -> its opener
    stack = []                            # indices of openers still waiting
    pairs = {}
    for i, ch in enumerate(s):
        if ch in "([{":
            stack.append(i)               # Push on open: the level waits
        elif ch in partner:
            if not stack or s[stack[-1]] != partner[ch]:
                return None               # nothing open, or the wrong kind
            j = stack.pop()               # Pop on close: the innermost level
            pairs[j] = i                  # Resolve: the pair is complete
    if stack:
        return None                       # openers never closed
    return pairs
```

Example.

```python
print(match_brackets("(a[b]{c})"))        # {0: 8, 2: 4, 5: 7}
```

Dials. What a frame holds (an opener, an index, the text and count before `[`, the total and sign before `(`, a running minimum), what Resolve does with it, and what happens on a closer with nothing open: fail, mark it, or start a new base.

Traps. Returning true without checking that the stack is empty accepts `"(("`. Reading the top of an empty stack on a leading closer crashes. One counter per kind cannot see the crossing in `"([)]"`; only the stack knows the order.

PROBLEMS

LC 20 · Valid Parentheses

Problem: Given a string `s` of the characters `()[]{}` only, return whether every bracket is closed by the same kind in the right order: `"()[]{}"` is valid, `"(]"` is not.

Intuition: The general solution with a yes-or-no answer. A closer needs only the kind of the innermost open bracket, so the stack holds characters, not indices. The string is valid when no closer fails and nothing is left open.

Differs: no pairs: the stack holds the opener characters, and Resolve is only the pop; return `True` or `False`.

LC 1249 · Minimum Remove to Make Valid Parentheses

Problem: Given a string `s` of `(`, `)` and lowercase letters, remove the fewest parentheses so that it is valid and return any such result: `"lee(t(c)o)de)"` gives `"lee(t(c)o)de"`.

Intuition: A `)` that finds nothing open can never be matched later, and a `(` still open at the end found no closer. Every other bracket has a partner, so removing exactly these strays is the minimum, and a stack of indices says where they are.

Differs: one kind, letters skipped; a `)` with nothing open is marked for removal instead of failing, and every index left on the stack at the end is marked too. Rebuild without the marks.

LC 32 · Longest Valid Parentheses

Problem: Given a string `s` of `(` and `)`, return the length of its longest valid (well-formed) parentheses substring: `")()())"` gives 4, for `"()()"`.

Intuition: Under the open `(`s lies a base: the last index no valid run can cross. A `)` pops its partner, and the new top, an open `(` or the base, is the nearest index outside the run ending at `i`. An unmatched `)` pops the base and becomes the new one.

Differs: seed the stack with -1, the base before the current run. On `)` pop; if the stack is now empty, push `i` as the new base, else score `i - stack[-1]`.

```
stack.pop()                             # a ")" pops its partner, or the base
if not stack:
    stack.append(i)                     # unmatched ")": the new base
else:
    best = max(best, i - stack[-1])     # the run right after the new top
```

LC 155 · Min Stack

Problem: Design a stack with `push(val)`, `pop()`, `top()` and `getMin()`, each in constant time: push -2, 0, -3; `getMin()` gives -3; after `pop()`, `top()` gives 0 and `getMin()` gives -2.

Intuition: A stack changes only at its top, so the minimum of everything below a frame cannot change while that frame is there. Each frame stores the minimum up to itself, resolved once at push time; a pop uncovers the old minimum for free.

Differs: no closers: every `push` opens a frame `(val, min(val, min below))`, `pop` drops it, and `getMin` reads the top frame's minimum.

```
below = self.entries[-1][1] if self.entries else val
self.entries.append((val, min(val, below)))
```

LC 394 · Decode String

Problem: Given an encoded string where `k[text]` means `text` repeated `k` times, groups nested inside groups, return the decoded string: `"3[a]2[bc]"` gives `"aaabcbc"`, and `"3[a2[c]]"` gives `"accaccacc"`.

Intuition: An inner group must be fully decoded before the outer level can repeat it. At each `[` the outer level parks its text so far and the count; at the matching `]` it comes back, repeats the finished inner text and appends it.

Differs: on `[` push `(text so far, count)` and start an empty level; on `]` pop and Resolve `current = before + current * count`. Digits build the count, which may have several digits.

```
if ch == "[":
    stack.append((current, count))      # Push: park the outer level
    current, count = "", 0
elif ch == "]":
    before, repeat = stack.pop()        # Pop and Resolve
    current = before + current * repeat
```

LC 224 · Basic Calculator

Problem: Evaluate a valid expression string of non-negative integers, `+`, `-`, parentheses and spaces, without `eval`; `-` may also be unary: `"(1+(4+5+2)-3)+(6+8)"` gives 23.

Intuition: With only `+` and `-` the value is a sum of signed terms, and a group in parentheses is one term, signed by what precedes its `(`. At `(` the outer total and that sign wait on the stack; at `)` the group folds in as one term. A unary `-` is an ordinary sign.

Differs: frames are `(total, sign)`: on `(` push and reset to `0, 1`; on `)` pop and fold the group in as one signed term. `+` and `-` add `sign * num` to `total` and set the next sign.

```
if ch == "(":
    stack.append((total, sign))         # Push: park the outer level
    total, sign = 0, 1
elif ch == ")":
    outer, outer_sign = stack.pop()     # Pop: the group is one term
    total, num = outer + outer_sign * (total + sign * num), 0
```

LC 150 · Evaluate Reverse Polish Notation

Problem: Given `tokens` of an expression in postfix order, with integers and `+ - * /`, return its value; division truncates toward zero: `["2","1","+","3","*"]` gives 9, `(2 + 1) * 3`.

Intuition: Postfix writes every operator after its operands, so the two newest values not yet used are exactly its arguments. Each operator pops them and pushes its result, which becomes an operand for a later operator.

Differs: no openers: numbers are pushed, and an operator pops two values (the right operand first), Resolves them and pushes the result. Division is `int(a / b)`: `//` rounds toward minus infinity.

```
b = stack.pop()
a = stack.pop()                          # pushed first: the left operand
stack.append(int(a / b) if token == "/" else ops[token](a, b))
```

Cursor stacks · Leri

A LEft stack and a RIght stack meet at the cursor.

Picture. The sequence is cut at the cursor into two stacks whose tops face each other; moving the cursor lifts an item off one top and sets it on the other, and editing touches only the tops.

Invariant. `left` read bottom to top, then `right` read top to bottom, is the whole sequence, and the cursor sits between the two tops.

Use when. a cursor, back and forward, undo and redo, backspace, type at the cursor, move k steps, rollback to the last begin, a queue built from stacks.

Book entries. #50 Key-Value Store with Transactions · #95 Design Browser History · #144 Design a Text Editor

GENERAL SOLUTION

Problem. Replay keystrokes on an empty line and return the text: `<` and `>` move the cursor one step, `#` deletes the character before the cursor, any other key is typed at the cursor, and a move or delete past an end of the line does nothing. `"abc<<x"` gives `"axbc"`.

Intuition. Cut the line at the cursor. Everything happens next to the cursor, so the two halves are stacks whose tops touch it: typing and backspace work on the top of the left one, and a step of the cursor moves one character from one top to the other. Every key costs at most one pop and one push.

```
key   left    right (top last)
abc   abc     -
<     ab      c
<     a       c b
x     ax      c b        text: "ax" + "bc"
```

```
def edit(keys):
    left = []                            # Le: text before the cursor
    right = []                           # Ri: text after it, nearest on top
    for key in keys:
        if key == "<":
            if left:
                right.append(left.pop())  # the cursor steps left
        elif key == ">":
            if right:
                left.append(right.pop())  # the cursor steps right
        elif key == "#":
            if left:
                left.pop()               # backspace deletes before the cursor
        else:
            left.append(key)             # typing pushes on the left
    return "".join(left) + "".join(reversed(right))
```

Example.

```
print(edit("abc<<x"))                    # axbc
```

Dials. What a move is (one step, k steps, a page of history), what an action discards (a new visit clears `right`), and what the stacks hold. An undo log is the same picture with time as the cursor: `begin` pushes an empty log, a write records the old value once per level, and `rollback` pops the log and writes it back (#50).

Traps. Popping an empty side on a move past either end. Forgetting that a new action throws away the redo side. Reading the right stack bottom to top: its top is the item next to the cursor.

PROBLEMS

LC 1472 · Design Browser History

Problem: Design `BrowserHistory(homepage)` with `visit(url)`, which clears the forward history, and `back(steps)` and `forward(steps)`, which move at most `steps` pages and return the current url. After visits a, b, c from home, `back(1)` gives b.

Intuition: The history is one sequence and the current page is the cursor. Back and forward move pages between the two stacks; a new visit makes the old forward pages unreachable, so they are thrown away. The home page never leaves the left stack.

Differs: `left` holds the history, current page on top. `visit` pushes and clears `right`; `back` moves pages to `right` but leaves one; `forward` moves them back; both return `left[-1]`. Entry #95 flattens both stacks into one array with a cursor and a `last` fence.

```
while steps > 0 and len(self.left) > 1:
    self.right.append(self.left.pop())
    steps -= 1
return self.left[-1]
```

LC 2296 · Design a Text Editor

Problem: Design `TextEditor`: `addText(text)` types at the cursor, `deleteText(k)` deletes up to `k` characters before it and returns how many, `cursorLeft(k)` and `cursorRight(k)` move up to `k` steps and return the last 10 characters before the cursor, or fewer: after `addText("leetcode")`, `deleteText(4)` gives 4, `cursorLeft(2)` gives `"le"`.

Intuition: The general solution with counts. The cursor moves only by steps, so every character moved or deleted is one pop and at most one push, and the top of `left` is exactly the text right before the cursor that the answers want.

Differs: calls carry k: `deleteText(k)` pops up to k from `left` and returns how many it popped; `cursorLeft(k)` and `cursorRight(k)` move up to k and return `"".join(left[-10:])`.

LC 1381 · Design a Stack With Increment Operation

Problem: Design `CustomStack(maxSize)`: `push(x)` is ignored when full, `pop()` returns the top or -1 if empty, and `increment(k, val)` adds `val` to the bottom `k` elements (all if fewer): push 1, 2, `increment(5, 100)`, `pop()` gives 102.

Intuition: An increment hits a bottom prefix, and a value is read only when it is popped. So the add waits on the top item of the prefix, and a pop hands it down to the item below, which is now in the prefix's top position. Every call is constant time.

Differs: one stack and a lazy array `inc`: `increment` adds `val` to `inc[min(k, size) - 1]` only, and does nothing on an empty stack; `pop` returns the top plus its `inc` and hands that `inc` down to the slot below.

```
i = len(self.stack) - 1
if i > 0:
    self.inc[i - 1] += self.inc[i]        # hand the pending add down
value = self.stack.pop() + self.inc[i]
self.inc[i] = 0
```

LC 844 · Backspace String Compare

Problem: Given strings `s` and `t` of lowercase letters and `#`, where `#` is a backspace, return whether they are equal once typed into empty editors; backspace on empty text does nothing: `"ab#c"` and `"ad#c"` both give `"ac"`, so true.

Intuition: The cursor never moves, so only the left stack is used. For constant space, read from the end instead: a `#` decides the characters before it, so count pending backspaces and skip that many characters.

Differs: only `left`: a letter pushes, `#` pops if `left` is not empty; build both strings and compare. For O(1) space, walk both strings from the end and skip one character per pending `#`.

LC 232 · Implement Queue using Stacks

Problem: Implement a first-in first-out queue with `push(x)`, `pop()`, `peek()` and `empty()` using only two stacks and their standard operations: push 1, push 2, then `peek()` gives 1 and `pop()` gives 1.

Intuition: Pouring one stack into another reverses it, so the oldest value lands on top of the out-stack. Pour only when the out-stack is empty; pouring onto values still waiting would bury older values under newer ones. Each value moves once, so every call is amortized constant time.

Differs: the two stacks are the back and the front of the queue: `push` goes on the in-stack; `pop` and `peek` read the out-stack, pouring the whole in-stack into it only when it is empty.

```
if not self.out_stack:
    while self.in_stack:
        self.out_stack.append(self.in_stack.pop())
return self.out_stack[-1]
```

Time ring · Evadco

EVict the expired, ADd the new, COunt what is held.

Picture. Events join a queue at the back in time order, and whatever has aged out drops off the front, so the queue is always the recent past; a fixed ring of buckets does the same with bounded memory.

Invariant. After Evict, the queue holds exactly the events inside the window, oldest at the front, and nothing behind the front can be older than it.

Use when. in the last N seconds, past 5 minutes, recent requests, at most k per window, rate limit, moving average of the last k values, fixed-size buffer, a body that grows at one end and shrinks at the other.

Book entries. #45 Design Hit Counter · #99 Design Snake Game · #110 Circular Queue · #138 Rate Limiter

GENERAL SOLUTION

Problem. Given the times of a stream of events, never decreasing, and a window length, return for each event at time `t` how many events so far fall in `(t - window, t]`: times `[1, 2, 3, 300, 301]` with window 300 give `[1, 2, 3, 4, 4]`.

Intuition. Events arrive in time order, so the queue is sorted by time and the oldest event is always at the front. An event that is too old now is too old for every later time, so it leaves for good, and only the front can expire. Each event enters once and leaves once.

```
t     evicted   held               count
1     -         [1]                1
2     -         [1, 2]             2
3     -         [1, 2, 3]          3
300   -         [1, 2, 3, 300]     4
301   1         [2, 3, 300, 301]   4
```

```
from collections import deque


def recent_counts(times, window):
    held = deque()                      # times still inside the window
    counts = []
    for t in times:                     # times never decrease
        while held and held[0] <= t - window:
            held.popleft()              # EVict: too old to count
        held.append(t)                  # ADd the new event at the back
        counts.append(len(held))        # COunt what is held
    return counts
```

Example.

```
print(recent_counts([1, 2, 3, 300, 301], 300))              # [1, 2, 3, 4, 4]
```

Dials. What expires the front (age in time, position past size k, a fixed capacity), what is counted (how many, a sum, the body cells), and the container: a deque when memory may follow the traffic, a ring of `window` buckets `t % window` when it must be bounded (#45), a fixed array with `head` and `count` for a queue (#110). A rate limiter adds before counting only if under the

limit (#138).

Traps. An off-by-one edge: `(t - 300, t]` and `[t - 3000, t]` evict on `<=` and `<`. A reused bucket must be reset when its second is stale. `if` instead of `while` leaves several expired events counted.

PROBLEMS

LC 362 · Design Hit Counter

Problem: Design `HitCounter` with `hit(timestamp)` and `getHits(timestamp)`, the number of hits in the past 300 seconds; timestamps are in seconds and never decrease: hits at 1, 2, 3, 300, then `getHits(300)` gives 4 and `getHits(301)` gives 3.

Intuition: The general solution split over two calls. A hit exactly 300 seconds old is outside the past 5 minutes. For heavy traffic, a ring of 300 buckets holds one second each, so memory stays at 300 slots whatever the hit rate; a bucket is reused when its second comes round again.

Differs: Add in `hit`, Evict and Count in `getHits`. Ring version: bucket `t % 300` stores its second and resets when stale; `getHits` sums the buckets with `t - time < 300`.

```
slot = timestamp % 300
if self.times[slot] != timestamp:         # stale bucket: start it again
    self.times[slot] = timestamp
    self.counts[slot] = 0
self.counts[slot] += 1
```

LC 359 · Logger Rate Limiter

Problem: Design `Logger`: `shouldPrintMessage(timestamp, message)`, called in time order, returns whether to print the message; a message printed at `t` blocks its copies until `t + 10`: "foo" at 1 is true, at 3 false, at 10 false, at 11 true.

Intuition: Only a printed message blocks its copies, so each message needs only the time it may next print. A dict holds that per message; an expired entry already compares as allowed, so it never has to be removed for the answer to be right.

Differs: held items are keyed by message: a dict message → the next time it may print. Print when `timestamp >= next_ok.get(message, 0)`, then set it to `timestamp + 10`; Evict is lazy.

```
if timestamp < self.next_ok.get(message, 0):
    return False                          # printed under 10 seconds ago
self.next_ok[message] = timestamp + 10
return True
```

LC 933 · Number of Recent Calls

Problem: Design `RecentCounter`: `ping(t)` records a call at `t` milliseconds and returns the number of calls in `[t - 3000, t]`; each `t` is larger than the last: pings 1, 100, 3001, 3002 give 1, 2, 3, 3.

Intuition: The general solution with a closed window: a call exactly 3000 ms old still counts. Times strictly increase, so the queue stays sorted and holds at most 3001 calls.

Differs: Evict while `held[0] < t - 3000`, not `<=`. Add, then Count `len(held)`.

LC 346 · Moving Average from Data Stream

Problem: Design `MovingAverage(size)`: `next(val)` adds a value and returns the average of the last `size` values (fewer at the start): with size 3, values 1, 10, 3, 5 give 1.0, 5.5, 4.66667, 6.0.

Intuition: "The last `size` values" expires by position, not by time: the oldest value leaves when a new one pushes the count past `size`. A running sum adds the newcomer and subtracts the leaver, so no call sums the window again.

Differs: the window is a count: Add `val`, Evict the front once `len(held) > size`, and keep a running sum that adds the new value and subtracts the evicted one; Count is `total / len(held)`.

```
self.held.append(val)
self.total += val
if len(self.held) > self.size:
    self.total -= self.held.popleft()     # the value that fell out
return self.total / len(self.held)
```

LC 622 · Design Circular Queue

Problem: Design `MyCircularQueue(k)` with `enQueue` and `deQueue` (returning success), `Front` and `Rear` (returning -1 when empty), `isEmpty` and `isFull`: with k = 3, the fourth `enQueue` fails, and after one `deQueue` it succeeds.

Intuition: The capacity is fixed, so the slots of an array of size k are reused round a ring: the front moves forward on every dequeue and the back wraps past the end. Counting the items tells full from empty, which a head equal to the tail alone cannot.

Differs: nothing expires by age: k slots, `head` and `count`. `enQueue` writes at `(head + count) % k` unless full, `deQueue` advances `head` round the ring; each updates `count`. `Rear` reads `(head + count - 1) % k`.

LC 353 · Design Snake Game

Problem: `SnakeGame(width, height, food)`: the snake starts at `(0, 0)`; the pieces `food[i] = [row, col]` appear one at a time, and each one eaten adds a cell. `move(direction)` returns the score, or -1 on hitting a wall or its body: width 3, height 2, food `[[1,2],[0,1]]`, moves RDRULU give 0, 0, 1, 1, 2, -1.

Intuition: One move changes only the two ends of the body: a new head in front, the tail leaving behind unless the snake eats. The tail leaves its cell in the same move, so the head may enter it. Food comes in list order, so the score is also the index of the next food.

Differs: the body is the held deque, head at the front, mirrored by a set. Test the walls; Evict the tail unless the new head is on `tuple(food[score])`; test the body after the tail leaves; then Add the head to both.

```
if self.score < len(self.food) and head == tuple(self.food[self.score]):
    self.score += 1                      # eat: the tail stays
else:
    self.occupied.remove(self.body.pop())  # Evict the tail first
if head in self.occupied:
    return -1
```

Squeeze

Every look discards candidates for good.

The question. What can I throw away for good?

Picture. Jaws close on the answer. Each look at one probe lets a whole side of the range fall away like the cut of a guillotine, and nothing that falls is ever looked at again.

Invariant. The answer is still inside `[lo, hi]`.

Shared skeleton.

```
def squeeze(lo, hi, pick, keeps_left):
    while lo < hi:                  # the range still holds two candidates
        probe = pick(lo, hi)        # dial: where to look
        if keeps_left(probe):       # dial: what one look proves
            hi = probe              # the right side cannot hold the answer
        else:
            lo = probe + 1          # the left side cannot hold the answer
    return lo                       # one candidate left: the answer
```

Dials. Where the probe sits and what one look discards. Pincers probe both ends and retire the weaker one. Chase and zip move two pointers the same way. The two bisects probe the middle, of stored data or of a yes/no test. Partition probes a pivot and keeps the side that holds k. Peel probes the low digit or bit and keeps the rest of the number.

Method	Mnemonic	What it sets	Book entries
Pincers	MeKeMo	probe both ends, retire the weaker end	#7, #8, #9, #56, #126
Chase and zip	Chazip	two pointers the same way: fast/slow, or one per list	#10, #66, #104, #150
Bisect stored order	First True	probe the middle of sorted data	#14, #47, #92, #98, #106
Bisect the answer	First True	probe the middle of a test that flips once	#15, #16, #55, #57, #105
Partition	Pacha	probe a pivot, keep the side that holds k	#18, #52, #60
Peel	Peel	probe the low digit or bit, keep the rest	#100, #125, #128, #130, #136, #145

A squeeze is licensed by an order: sorted values, a test that flips once, a pivot that splits, a number that shrinks. That is what tells it apart from a **sweep**, which visits every item and only summarises it; a squeeze proves most items irrelevant without looking at them. The family-wide trap is a look that discards the wrong part or nothing at all: moving the taller wall throws away a pair that may be the best, and `lo = mid` after `mid` failed keeps a range of two forever. Before writing the loop, say what one look proves, and on which side the answer must still be.

Pincers · MeKeMo

MEasure the pair at the two ends, KEep the best, MOve the weaker end.

Picture. Two jaws start at the ends of the line and close one step at a time; each step crosses off every pair that uses the end that moved.

Invariant. Every pair with an end outside `[left, right]` has been measured or proved no better than `best`.

Use when. a sorted array and a pair (or triplet) with a target sum; a pair scored by its distance and its smaller end; water held between walls; pair the heaviest with the lightest; a matrix sorted along rows and columns; read a matrix ring by ring.

Book entries. #7 3Sum · #8 Container With Most Water · #9 Trapping Rain Water · #56 Search a 2D Matrix · #126 Spiral Matrix

GENERAL SOLUTION

Problem. Given wall heights `height`, pick two walls `i < j`; they hold `min(height[i], height[j]) * (j - i)` units of water. Return the most water any pair holds: `[1,8,6,2,5,4,8,3,7]` gives 49, from the walls of height 8 and 7 at indices 1 and 8.

Intuition. Start with the widest pair. The shorter wall caps the water of every pair it is part of, and every other pair it forms inside the jaws is narrower, so none of them can beat the one just measured: the shorter wall retires for good. Each step retires one wall, so `n - 1` measurements cover all pairs.

```
left right  walls  area  best  move
0    8      1, 7   8     8     left (1 < 7)
1    8      8, 7   49    49    right (7 < 8)
1    7      8, 3   18    49    right
1    6      8, 8   40    49    right (tie)
```

```python
def max_area(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        width = right - left
        area = min(height[left], height[right]) * width   # Measure
        best = max(best, area)                            # Keep the best
        if height[left] < height[right]:                  # Move the weaker
            left += 1          # every pair with this left wall is no better
        else:
            right -= 1         # shorter or tied: the right wall is spent
    return best
```

Example.

```python
print(max_area([1, 8, 6, 2, 5, 4, 8, 3, 7]))               # 49
```

Dials. What Measure computes (a sum, an area, the water of one column), what Keep does with it (return on a hit, keep a maximum, add to a total, count a boat), and which end is weaker (the smaller sum, the shorter wall, the lower running maximum). The jaws can also be the walls of a matrix: a corner, or four bounds.

Traps. Moving the stronger end: it keeps the same cap and only loses width, so the best pair is skipped. Pincers need an order that makes one end provably useless; on an unsorted array with negatives, they prove nothing.

PROBLEMS

LC 167 · Two Sum II - Input Array Is Sorted

Problem: Given a 1-indexed array `numbers` sorted in non-decreasing order and a `target`, return the indices `[i, j]`, `i < j`, of the one pair that adds up to it (exactly one exists), using constant extra space: `[2,7,11,15]` with target 9 gives `[1,2]`.

Intuition: The array is sorted, so a sum below `target` means `numbers[left]` is too small even with the largest partner left: no pair with it can hit, and it retires. A sum above retires `numbers[right]` the same way. Exactly one solution exists, so the jaws meet it.

Differs: Measure is `numbers[left] + numbers[right]`; Keep returns `[left + 1, right + 1]` on a hit; a sum below `target` moves `left`, above moves `right`.

```python
total = numbers[left] + numbers[right]
if total < target:
    left += 1
elif total > target:
    right -= 1
```

LC 15 · 3Sum

Problem: Given an integer array `nums`, return every distinct triplet of values at three different indices that sums to zero: `[-1,0,1,2,-1,-4]` gives `[[-1,-1,2],[-1,0,1]]`.

Intuition: Fix the smallest value of the triplet; the other two are a Two Sum II on the sorted suffix. Sorting puts equal values side by side, so skipping a repeat at the same position skips exactly the triplets already found, and a positive smallest value can never reach zero.

Differs: sort, fix `nums[i]` in an outer loop and run Two Sum II on the suffix for `-nums[i]`. Skip `i` when `i > 0` and `nums[i] == nums[i - 1]`, stop when `nums[i] > 0`; after a hit move both ends and step `left` past copies.

```python
while left < right and nums[left] == nums[left - 1]:
    left += 1
```

LC 11 · Container With Most Water

Problem: Given wall heights `height`, return the most water two walls can hold, `min` of the two heights times their distance: `[1,8,6,2,5,4,8,3,7]` gives 49.

Intuition: The water is capped by the shorter wall. Every other pair inside the jaws that keeps that wall is narrower and no taller, so none beats the pair just measured, and the shorter wall retires for good after one measurement.

Differs: nothing: this is the general solution.

LC 42 · Trapping Rain Water

Problem: Given bar heights `height`, each bar one unit wide, return how much rain water is trapped between the bars after it rains: `[0,1,0,2,1,0,1,3,2,1,2,1]` gives 6.

Intuition: A column fills to the lower of the tallest walls on its two sides. If `left_max < right_max`, the true right wall of column `left` is at least `right_max`, so its lower wall is `left_max`: that column is settled and the left jaw moves.

Differs: carry `left_max` and `right_max`; Measure is the water over one column, Keep adds it to a total, and the side with the lower running maximum moves. Raise the maximum before adding.

```
left_max = max(left_max, height[left])
right_max = max(right_max, height[right])
if left_max < right_max:
    water += left_max - height[left]
    left += 1
```

LC 881 · Boats to Save People

Problem: `people[i]` is a weight, each at most `limit`. A boat carries at most two people whose weights sum to at most `limit`. Return the fewest boats that carry everyone: `[3,2,2,1]` with limit 3 gives 3.

Intuition: Sort the weights. The heaviest person must ride; if the lightest cannot join them, nobody can, so they ride alone. If the lightest can, pairing them is never worse than pairing anyone heavier. Each boat retires the heaviest end.

Differs: sort; each step the heaviest at `right` takes a boat and `left` moves too when `people[left] + people[right] <= limit`. Keep counts boats; the loop runs while `left <= right`.

```
if people[left] + people[right] <= limit:
    left += 1
right -= 1
boats += 1
```

LC 240 · Search a 2D Matrix II

Problem: In an `m x n` matrix every row ascends left to right and every column ascends top to bottom. Return whether `target` is in it: `[[1,4],[2,5]]` with target 2 gives true.

Intuition: The top-right corner is the largest of its row and the smallest of its column. One comparison there rules out a whole line: a value above `target` kills its column, below kills its row. The walk takes at most `m + n` steps.

Differs: the two ends are a row and a column meeting at the top-right corner. A value above `target` drops the column (`col -= 1`), below drops the row (`row += 1`); stop on a hit or when the walk leaves the matrix.

```
if matrix[row][col] > target:
    col -= 1
else:
    row += 1
```

LC 54 · Spiral Matrix

Problem: Given an `m x n` matrix, return all its elements in spiral order, clockwise from the top left: `[[1,2,3],[4,5,6],[7,8,9]]` gives `[1,2,3,6,9,8,7,4,5]`.

Intuition: The unread cells always form a rectangle that four bounds describe. Read its top row, right column, bottom row and left column, and each side read moves its wall one step inward. When the last ring is a single row or column, the unguarded sides would read it again backwards.

Differs: four walls instead of two, each moved inward after its side is read. Guard the bottom row with `top <= bottom` and the left column with `left <= right`.

Chase and zip · Chazip

CHAse: a fast pointer and a slow one on one path. ZIP: one pointer per sorted list, the smaller moves.

Picture. Two runners on one track, one twice as fast, until they meet in the loop or the fast one runs off the end; or two fingers walking down two sorted lists side by side.

Invariant. Everything behind the pointers is settled: fast has walked exactly twice as far as slow, or every value behind `i` and `j` has been matched or ruled out.

Use when. a linked list or a "next" rule (`i → nums[i]`, a number → its digit squares) and a loop, its start, or the middle, in `O(1)` space; two sorted lists to merge, intersect or compare.

Book entries. #10 Floyd's Tortoise and Hare · #66 Interval List Intersections · #104 Reorder List · #150 Inverted Index

GENERAL SOLUTION

Problem. Chase: a path runs `start, step(start), step(step(start)), ...`, where `step` returns `None` at the end; return the node where the path enters its loop, or `None` if it has none. On 1 → 2 → 3 → 4 → 5 → 3 the entry is 3. Zip: return the values two sorted lists share; `[1,3,4,6]` and `[2,3,4,5,6]` give `[3,4,6]`.

Intuition. In a loop the fast runner gains one node per step, so it closes the gap one node at a time and lands on the slow one instead of jumping over it. Fast has then walked slow's distance twice and stands on the same node, so slow's walk is a whole number of laps; with a tail of `a` nodes, a runner restarted at the start and one left at the meeting, both at speed 1, meet after `a` steps: at the entry. In a zip the smaller value cannot appear later in the other sorted list, so it retires.

```
phase   slow  fast
chase   1     1
chase   2     3
chase   3     5
chase   4     4     met inside the loop
walk    1     4     slow restarts
walk    2     5
walk    3     3     both at the entry
```

```python
def cycle_entry(start, step):
    slow = fast = start
    while fast is not None and step(fast) is not None:
        slow, fast = step(slow), step(step(fast))   # CHAse: 1 step vs 2
        if slow == fast:                            # met inside the loop
            slow = start                            # one runner restarts
            while slow != fast:                     # both at speed 1
                slow, fast = step(slow), step(fast)
            return slow                             # they meet at the entry
    return None                                     # ran off the end


def zip_common(a, b):
    i = j = 0
    common = []
    while i < len(a) and j < len(b):
        if a[i] == b[j]:                            # in both: keep it
            common.append(a[i])
            i, j = i + 1, j + 1
        elif a[i] < b[j]:                           # ZIP: the smaller moves,
            i += 1                                  # it is not in the other
        else:
            j += 1
    return common
```

Example.

```python
links = {1: 2, 2: 3, 3: 4, 4: 5, 5: 3}    # 1 -> 2 -> 3 -> 4 -> 5 -> back to 3
print(cycle_entry(1, links.get))          # 3
print(zip_common([1, 3, 4, 6], [2, 3, 4, 5, 6]))            # [3, 4, 6]
```

Dials. Chase: what `step` is (a `next` pointer, `nums[i]`, a digit rule), and whether to stop at the meeting, at the entry, or when fast runs out (the middle). Zip: what the pointers do on equal values, whether the walk runs from the back, and what is written.

Traps. Testing `slow == fast` before the first move, where both sit on the start; comparing node values instead of nodes. In a zip, forgetting the tail of the list that did not run out.

PROBLEMS

LC 141 · Linked List Cycle

Problem: Given the `head` of a linked list, return `true` if some node can be reached again by following `next`, else `false`: the list 3 → 2 → 0 → -4 with -4 pointing back to 2 gives `true`.

Intuition: Two runners on one track. Inside a loop fast gains exactly one node per step, so the gap shrinks by one and closes on a meeting instead of jumping over it; without a loop fast reaches the end of the list.

Differs: phase one only, with `step = node.next`: return `True` at the meeting and `False` when fast runs off the end.

LC 142 · Linked List Cycle II

Problem: Given the `head` of a linked list, return the node where its cycle begins, or `null` if there is none, without modifying the list: 3 → 2 → 0 → -4 with -4 pointing back to 2 gives the node 2.

Intuition: With a tail of `a` nodes and a meeting `b` steps into a loop of `c`, `a + b` is a whole number of laps. So `a` more steps from the meeting land on the entry, exactly as `a` steps from the head do: restart one runner and walk both at speed 1.

Differs: nothing: the general solution with `step = node.next`, returning the entry node or `None`.

LC 287 · Find the Duplicate Number

Problem: `nums` holds `n + 1` integers, each in `[1, n]`, and exactly one value appears two or more times. Return it without modifying `nums`, in constant extra space: `[1,3,4,2,2]` gives 2.

Intuition: Read index `i` as a node with an arrow to `nums[i]`. No value is 0, so nothing points at index 0 and the walk from it starts on a tail. The loop's entry gets one arrow from the tail and one from inside the loop, and only the repeated value has more than one arrow in.

Differs: the path is over indices: `step = lambda i: nums[i]`, starting at index 0; the entry of the loop is the duplicate. No list is built and `nums` is untouched.

LC 202 · Happy Number

Problem: Replace a positive integer `n` by the sum of the squares of its digits, again and again. `n` is happy if this reaches 1; return whether it is: 19 → 82 → 68 → 100 → 1 gives `true`, and 2 gives `false`.

Intuition: A number of four or more digits maps to a smaller one, and every number below 1000 maps below 244, so the walk sees finitely many values and falls into a loop. The number is happy exactly when that loop is the fixed point 1 → 1, and the runners always meet inside the loop.

Differs: `step` is the digit-square sum and never returns `None`; stop at the meeting and return `slow == 1`.

```
def squares(n):
    return sum(int(d) ** 2 for d in str(n))
```

LC 876 · Middle of the Linked List

Problem: Given the `head` of a linked list, return its middle node; with two middles, return the second: `[1,2,3,4,5]` gives the node 3, and `[1,2,3,4,5,6]` gives the node 4.

Intuition: Fast walks twice as far as slow, so when fast can no longer take a double step slow is halfway. This stop rule lands on the second middle of an even-length list, which is the one asked for.

Differs: phase one with no meeting test; loop `while fast and fast.next` and return `slow`.

LC 143 · Reorder List

Problem: Reorder the list L0 → L1 → … → Ln into L0 → Ln → L1 → Ln-1 → … in place, changing links, not values: `[1,2,3,4]` becomes `[1,4,2,3]`.

Intuition: The order reads the list from both ends, and a singly linked list reads only forward. Reverse the back half and it reads forward too; the answer is the two halves woven together, one node from each, relinked in place.

Differs: three moves: chase to the first half's last node (`while fast.next and fast.next.next`), cut there and reverse the second half, then zip by alternation instead of by comparison.

```
nxt_first, nxt_second = first.next, second.next
first.next, second.next = second, nxt_first
first, second = nxt_first, nxt_second
```

LC 88 · Merge Sorted Array

Problem: `nums1` holds `m` sorted values followed by `n` zeros, and `nums2` holds `n` sorted values. Merge `nums2` into `nums1` in place, sorted: `[1,2,3,0,0,0]`, `m = 3`, `[2,5,6]`, `n = 3` give `[1,2,2,3,5,6]`.

Intuition: The free slots are at the back of `nums1`, so fill it from the back with the largest value first. The write slot `i + j + 1` stays ahead of `i` while `nums2` has values left, so no unread value of `nums1` is overwritten.

Differs: zip from the back: `i = m - 1`, `j = n - 1`, and `write = m + n - 1` receives the larger of `nums1[i]` and `nums2[j]`. Loop while `j >= 0`: what is left of `nums1` is already in place. Return nothing.

```
if i >= 0 and nums1[i] > nums2[j]:
    nums1[write] = nums1[i]
    i -= 1
else:
    nums1[write] = nums2[j]
    j -= 1
```

Bisect stored order · First True

Find the FIRST index where the test is TRUE. True: hi sits on mid. False: lo jumps past.

Picture. The stored data reads ✗ ✗ ✗ ✓ ✓ ✓ under the test; each probe at the middle cuts away half of the unknown stretch until the cut lands on the first ✓.

Invariant. Every index left of `lo` is ✗ and every index from `hi` on is ✓, with the end of the range counting as ✓.

Use when. sorted, non-decreasing, timestamps strictly increasing, snapshot ids; first, last, how many, where would it go, the latest value at or before a time; `O(log n)` required.

Book entries. #14 Binary Search · #47 Snapshot Array · #92 Time-Based Key-Value Store · #98 My Calendar I and II · #106 Random Pick with Weight

GENERAL SOLUTION

Problem. Given a sorted list `nums` and a `target`, return the first index whose value is at least `target`, or `len(nums)` if there is none (the lower bound): `[1,3,3,5,8,9]` with target 4 gives 3, and target 3 gives 1.

Intuition. Under the test `nums[i] >= target` sorted data reads ✗ ✗ ✗ ✓ ✓ ✓: the test flips once. One probe at the middle tells on which side of `mid` the flip is, so half of the unknown stretch is discarded for good. A ✓ at `mid` may be the first one, so `hi` keeps it; a ✗ is out.

```
lo  hi  mid  nums[mid] >= 4
0   6   3    5: True,  hi = 3
0   3   1    3: False, lo = 2
2   3   2    3: False, lo = 3
3   3        stop: the first True is 3
```

```python
def first_true(lo, hi, is_true):
    while lo < hi:                  # [lo, hi) still holds unknown indices
        mid = (lo + hi) // 2        # probe the middle
        if is_true(mid):
            hi = mid                # True: hi sits on mid, it may be first
        else:
            lo = mid + 1            # False: lo jumps past, mid is out
    return lo                       # lo == hi: the first True


def lower_bound(nums, target):
    return first_true(0, len(nums), lambda i: nums[i] >= target)
```

Example.

```python
nums = [1, 3, 3, 5, 8, 9]
print(lower_bound(nums, 4))               # 3
print(first_true(1, 10, lambda v: v * v >= 30))            # 6
```

Dials. The test (`>=` for the first copy, `>` for one past the last), the range (`[0, n)`, versions `1 .. n`, flat cells `0 .. m*n - 1`), how a probe reads the data (fold a flat index into a cell, read a prefix sum), and what is returned (the index, a check of it, the slot before it).

Traps. `hi = mid - 1` in this half-open loop can jump past the answer; `lo = mid` loops forever on a range of two. Reading `nums[lo]` after the loop without checking `lo < len(nums)`.

PROBLEMS

LC 704 · Binary Search

Problem: Given distinct integers `nums` sorted ascending and a `target`, return its index or -1, in `O(log n)`: `[-1,0,3,5,9,12]` with target 9 gives 4, and with target 2 gives -1.

Intuition: The test `nums[i] >= target` flips once on sorted data. A present target sits exactly at the first ✓; an absent one leaves a larger value, or the end of the list, there instead.

Differs: after the lower bound, check it: return `lo` if `lo < len(nums)` and `nums[lo] == target`, else `-1`.

LC 35 · Search Insert Position

Problem: Given distinct integers `nums` sorted ascending and a `target`, return its index if present, else the index where it would be inserted in order, in `O(log n)`: `[1,3,5,6]` gives 2 for target 5, 1 for target 2, 4 for target 7.

Intuition: The first index whose value is at least the target is where the target is, or where it would go to keep the order; `len(nums)` means after the end. That is the lower bound itself.

Differs: return the lower bound as it is, with no check.

LC 34 · Find First and Last Position of Element in Sorted Array

Problem: Given `nums` sorted in non-decreasing order and a `target`, return the first and last index of `target`, or `[-1, -1]`, in `O(log n)`: `[5,7,7,8,8,10]` gives `[3,4]` for target 8 and `[-1,-1]` for target 6.

Intuition: The copies of the target form one run. The test `>=` finds its first index; the test `>` counts equal values as ×, so its first ✓ is one past the run. Two searches keep `O(log n)`.

Differs: two searches: the first with `nums[i] >= target`, the second with `nums[i] > target`; the last position is the second minus one. Return `[-1, -1]` when the first misses.

```python
first = first_true(0, n, lambda i: nums[i] >= target)
after = first_true(0, n, lambda i: nums[i] > target)
```

LC 278 · First Bad Version

Problem: Versions `1 .. n` are built in order and every version after a bad one is bad. Given an API `isBadVersion(v)`, return the first bad version with the fewest calls: `n = 5` with first bad 4 gives 4.

Intuition: The calls read × × ✓ ✓ along the versions: the test flips once. Halving the unknown stretch makes about `log2 n` calls, the fewest possible.

Differs: the test is the API call: `first_true(1, n, isBadVersion)`. The range is the versions `1 .. n`, and `hi = n` is safe because a bad version is guaranteed.

LC 74 · Search a 2D Matrix

Problem: Each row of an `m x n` matrix is sorted and starts above the last value of the row before. Return whether `target` is in it, in `O(log(m * n))`: `[[1,3,5,7],[10,11,16,20],[23,30,34,60]]` with target 3 gives true.

Intuition: Laid end to end, the rows are one sorted array of `m * n` values. One lower bound over that flat array finds the target, and a flat index folds back into a row and a column.

Differs: search the flat positions `0 .. m*n - 1`; the probe reads `matrix[p // n][p % n]`, dividing by the row length `n`. Check the cell at `lo` afterwards.

```
cell = lambda p: matrix[p // n][p % n]
lo = first_true(0, m * n, lambda p: cell(p) >= target)
```

LC 981 · Time Based Key-Value Store

Problem: Design `set(key, value, timestamp)`, called with strictly increasing timestamps, and `get(key, timestamp)`, which returns the value set for `key` at the largest time at or before `timestamp`, or `""`: after `set("foo","bar",1)` and `set("foo","bar2",4)`, `get("foo",3)` gives "bar" and `get("foo",4)` gives "bar2".

Intuition: Timestamps arrive in increasing order, so appending keeps every key's list sorted with no sort. The wanted version is the last time at or before `timestamp`: the slot just left of the first time that is larger.

Differs: each key keeps parallel lists `times` and `values`; `get` finds the first time `> timestamp` and returns the value one slot before it, or `""` when that slot is 0.

```
k = first_true(0, len(times), lambda i: times[i] > timestamp)
answer = values[k - 1] if k > 0 else ""
```

LC 528 · Random Pick with Weight

Problem: Given positive weights `w`, implement `pickIndex()`, which returns index `i` with probability `w[i] / sum(w)`: with `w = [1,3]`, index 0 comes up a quarter of the time and index 1 three quarters.

Intuition: Lay the weights end to end on a line of length `total`; index `i` owns a stretch of `w[i]` units. The prefix sums strictly increase, so the owner of a random point is the first prefix that reaches it.

Differs: build prefix sums once; `pickIndex` draws `target = randint(1, total)` and returns the first `i` with `prefix[i] >= target`.

```
target = random.randint(1, total)
pick = first_true(0, len(prefix), lambda i: prefix[i] >= target)
```

Bisect the answer · First True

Find the FIRST candidate where a yes/no test is TRUE. True: hi sits on mid. False: lo jumps past.

Picture. Nothing is sorted, but the candidate answers on a number line read ✗ ✗ ✗ ✓ ✓ ✓ under a test you can run; the cut halves the line of answers, not the input.

Invariant. Every candidate left of `lo` fails the test and `hi` passes it, so the smallest passing answer lies in `[lo, hi]`.

Use when. minimum capacity, smallest speed, minimize the largest, maximize the minimum, within `h` hours or `d` days; values up to 10^9; rotated sorted; a median of two sorted arrays in `O(log)`; any peak.

Book entries. #15 Binary Search on the Answer · #16 Search in a Rotated Sorted Array · #55 Median of Two Sorted Arrays · #57 Find Peak Element · #105 Median of K Sorted Arrays

GENERAL SOLUTION

Problem. Koko Eating Bananas: `piles[i]` bananas lie in pile `i`. Each hour Koko picks one pile and eats `k` bananas from it, or all of it if fewer remain. Return the smallest integer speed `k` that finishes every pile within `h` hours: `[3,6,7,11]` with `h = 8` gives 4.

Intuition. Nothing in the input is sorted, but the answers are: a faster speed never takes more hours, so the test "finishes within `h`" reads ✗ ✗ ✗ ✓ ✓ ✓ along the speeds `1 .. max(piles)`. Each probe runs a one-pass simulation at one fixed speed and discards half of the speeds.

```
lo  hi  speed  hours             result
1   11  6      1+1+2+2 = 6       True,  hi = 6
1   6   3      1+2+3+4 = 10      False, lo = 4
4   6   5      1+2+2+3 = 8       True,  hi = 5
4   5   4      1+2+2+3 = 8       True,  hi = 4
4   4          stop: speed 4
```

```
def first_true(lo, hi, is_true):
    while lo < hi:
        mid = (lo + hi) // 2
        if is_true(mid):
            hi = mid                # True: hi sits on mid
        else:
            lo = mid + 1            # False: lo jumps past
    return lo


def min_eating_speed(piles, h):
    def finishes(speed):            # the test: simulate one fixed answer
        hours = sum((pile + speed - 1) // speed for pile in piles)
        return hours <= h           # a partial hour costs a full one
    return first_true(1, max(piles), finishes)  # max(piles) always works
```

Example.

```
print(min_eating_speed([3, 6, 7, 11], 8))                  # 4
print(first_true(0, 10, lambda x: x * x >= 50))            # 8
```

Dials. The range (smallest conceivable answer, a value that surely works), the test (a greedy simulation, a comparison with the last element, a cross-check of a cut, the slope) and whether "largest that works" is searched as "first that fails, minus one".

Traps. `pile // speed` rounds down and forgets the partial hour; a range starting at 0 divides by zero; a `lo` below the true minimum (ship capacity below the heaviest package) makes the test lie.

PROBLEMS

LC 875 · Koko Eating Bananas

Problem: Each hour Koko eats `k` bananas from one pile, or the whole pile if it has fewer. Return the smallest integer `k` that eats all `piles` within `h` hours, where `len(piles) <= h`: `[3,6,7,11]` with `h = 8` gives 4.

Intuition: At speed `k` the hours `sum(ceil(p / k))` never grow as `k` grows, so "finishes within `h`" flips once along the speeds. One pile per hour at speed `max(piles)` takes `len(piles) <= h` hours, so the top of the range always passes.

Differs: nothing: the general solution, range `[1, max(piles)]`.

LC 1011 · Capacity To Ship Packages Within D Days

Problem: A ship carries packages `weights` in the given order, loading each day up to its capacity. Return the smallest capacity that ships all of them within `days` days: `[1,2,3,1,1]` with `days = 4` gives 3.

Intuition: A bigger ship never needs more days, so the test flips once along the capacities. For one fixed capacity, filling each day until the next package would overflow needs the fewest days: the packages keep their order, and a day that stops earlier only leaves more for later.

Differs: the range is `[max(weights), sum(weights)]`; the test loads packages greedily, starts a new day on overflow, and passes if the days needed are at most `days`.

```
need, load = 1, 0
for w in weights:
    if load + w > cap:
        need, load = need + 1, 0
    load += w
```

LC 410 · Split Array Largest Sum

Problem: Split `nums` into `k` non-empty contiguous pieces so that the largest piece sum is as small as possible, and return that sum: `[7,2,5,10,8]` with `k = 2` gives 18, from `[7,2,5]` and `[10,8]`.

Intuition: This is LC 1011 in other words: a cap on the piece sum is a ship's capacity. A larger cap never needs more pieces, and fewer than `k` pieces still counts as a yes, because `k <= len(nums)` lets any piece be split again without raising the largest sum.

Differs: the same test and range as LC 1011, with `k` in place of `days`: the smallest cap whose greedy split uses at most `k` pieces.

LC 33 · Search in Rotated Sorted Array

Problem: Distinct integers sorted ascending were rotated at an unknown index. Return the index of `target` or -1, in `O(log n)`: `[4,5,6,7,0,1,2]` with target 0 gives 4, and with target 3 gives -1.

Intuition: The array is two ascending ramps, so at least one half of any window lies on one ramp and is sorted. Two comparisons with that half's ends prove the target in it or out of it, and the other half takes the rest.

Differs: an exact search in a closed window `[lo, hi]`, returning `mid` on a hit. The probe asks which half is sorted (`nums[lo] <= nums[mid]`) and whether the target lies within its ends.

```
if nums[lo] <= nums[mid]:
    if nums[lo] <= target < nums[mid]:
        hi = mid - 1
    else:
        lo = mid + 1
```

LC 153 · Find Minimum in Rotated Sorted Array

Problem: A sorted array of unique values was rotated between 1 and `n` times. Return its minimum in `O(log n)`: `[3,4,5,1,2]` gives 1.

Intuition: With unique values every value on the left ramp is above `nums[-1]` and every value on the right ramp is at most it, so the test `nums[i] <= nums[-1]` reads × then ✓. Its first ✓ is the bottom of the cliff.

Differs: the test is `nums[i] <= nums[-1]` over `[0, n - 1]`; return `nums[lo]`.

```
lo = first_true(0, n - 1, lambda i: nums[i] <= nums[-1])
```

LC 4 · Median of Two Sorted Arrays

Problem: Given sorted arrays `nums1` and `nums2` of sizes `m` and `n`, return the median of all their values in `O(log(m + n))`: `[1,3]` and `[2]` give 2.0, and `[1,2]` and `[3,4]` give 2.5.

Intuition: The median splits the union into two halves. Bisect how many left-half values come from the shorter array: as that cut `i` grows, `a[i]` rises and `b[j - 1]` falls, so `a[i] >= b[j - 1]` flips once. Its first ✓ puts every left value at or below every right one.

Differs: the candidate is the cut `i` in the shorter array `a` over `[0, m]`, with `j = half - i` in `b`; read `-inf` and `inf` off the edges. The median comes from the four values around the cut.

```
half = (m + n + 1) // 2
i = first_true(0, m, lambda i: at(a, i) >= at(b, half - i - 1))
```

LC 162 · Find Peak Element

Problem: In `nums`, neighbours differ and both outside ends count as `-inf`. Return the index of any element larger than its neighbours, in `O(log n)`: `[1,2,3,1]` gives 2.

Intuition: The slope need not flip only once, but a climb from a rising point must stop at a peak on its right before it falls off the `-inf` end. So the window always enters uphill and leaves downhill, and some peak stays inside it.

Differs: the test is the slope, `nums[i] > nums[i + 1]`, over `[0, n - 1]`: falling keeps `mid`, rising jumps past it. Return `lo`.

Partition · Pacha

PArtition the range around a pivot, CHAse only the side that holds k.

Picture. A wall slides right through the range: smaller values are swapped behind it, the pivot drops onto the wall at its final place, and the side without index k is cut away.

Invariant. `nums[lo .. store - 1]` holds only values below the pivot, and the index `target` of the answer never leaves `[lo, hi]`.

Use when. kth largest or smallest, k closest, top k in any order; split in place into two or three classes (colours, parity, around a value); sort in place without a library.

Book entries. #18 Quick Sort · #52 Dutch National Flag · #60 Quickselect

GENERAL SOLUTION

Problem. Given an unsorted integer array `nums` and `k`, return the kth largest value, copies counted separately: `[3,2,1,5,6,4]` with `k = 2` gives 5.

Intuition. In ascending order the answer sits at index `n - k`. One partition drops the pivot onto its final index `p`, with every smaller value to its left and none to its right, so only the side that holds `n - k` can still hold the answer; the other side is cut away unsorted. Below, the random pick happens to choose the last value each time.

```
range  pivot  after partition    p  vs target 4
0..5   4      [3,2,1,4,6,5]      3  p < 4: lo = 4
4..5   5      [3,2,1,4,5,6]      4  hit: return 5
```

```python
import random


def partition(nums, lo, hi):
    pick = random.randint(lo, hi)
    nums[pick], nums[hi] = nums[hi], nums[pick]   # park the pivot at the end
    pivot, store = nums[hi], lo                   # nums[lo..store-1] < pivot
    for i in range(lo, hi):
        if nums[i] < pivot:                       # a small value goes behind
            nums[store], nums[i] = nums[i], nums[store]
            store += 1
    nums[store], nums[hi] = nums[hi], nums[store] # pivot onto the wall
    return store


def kth_largest(nums, k):
    target = len(nums) - k
    lo, hi = 0, len(nums) - 1
    while True:
        p = partition(nums, lo, hi)               # PArtition: p is final
        if p == target:
            return nums[p]
        if p < target:                            # CHAse the side with k
            lo = p + 1
        else:
            hi = p - 1
```

Example.

```python
print(kth_largest([3, 2, 1, 5, 6, 4], 2))                  # 5
row = [9, 4, 7, 1, 8, 4]
p = partition(row, 0, 5)                  # the pivot lands on its final index
```

Dials. The test that sends a value behind the wall (`< pivot`, even, `== 0`), how many regions (two with Lomuto, three with low/mid/high), the key compared (a value, a squared distance), and which sides are chased: one (select), both (sort), none (a single pass).

Traps. A fixed last-element pivot on sorted input peels one value per round: $O(n^2)$. Many equal values all land on one side of `<`; the three-way partition groups them and finishes them at once. In Dutch flag, advancing `mid` after a swap with `high` skips an unseen value.

PROBLEMS

LC 75 · Sort Colors

Problem: `nums` holds only 0, 1 and 2 (red, white, blue). Sort it in place so equal colours are adjacent in the order 0, 1, 2, without the library sort: `[2,0,2,1,1,0]` becomes `[0,0,1,1,2,2]`.

Intuition: Only three values exist, so the pivot is fixed at 1 and one pass with three walls sorts the array: 0s grow from the left, 2s from the right, and the unseen stretch shrinks between them. The value swapped back from `high` is unseen, so it must be looked at next.

Differs: three regions and no chase: `low`, `mid`, `high` split the array into 0s, 1s, unseen and 2s. A 0 swaps to `low` and both advance; a 1 advances `mid`; a 2 swaps to `high`, which retreats while `mid` stays.

```
if nums[mid] == 2:
    nums[mid], nums[high] = nums[high], nums[mid]
    high -= 1
```

LC 905 · Sort Array By Parity

Problem: Given an integer array `nums`, return an array with every even value before every odd one, in any order within each group: `[3,1,2,4]` may give `[2,4,3,1]`.

Intuition: The answer is a two-class split with no order inside the classes, so one partition pass with a class test in place of a pivot comparison does it, and an unstable swap is fine.

Differs: one partition pass over all of `nums` with the test `nums[i] % 2 == 0` in place of `< pivot`; no pivot is parked or placed, and nothing is chased. Return the array.

LC 2161 · Partition Array According to Given Pivot

Problem: Rearrange `nums` so values below `pivot` come first, then those equal to it, then those above, keeping the original order inside the below and above groups: `[9,12,5,10,14,3,10]` with pivot 10 gives `[9,5,3,10,10,12,14]`.

Intuition: Swaps scramble the order inside a group, and here that order must stay. Three lists filled in one left-to-right pass keep it, and the problem returns a new array, so the `O(n)` extra space is allowed.

Differs: no swaps: one pass sends each value to a `less`, `equal` or `greater` list, and the answer is their concatenation.

```
less = [x for x in nums if x < pivot]
equal = [x for x in nums if x == pivot]
greater = [x for x in nums if x > pivot]
```

LC 215 · Kth Largest Element in an Array

Problem: Given an integer array `nums` (up to 10^5 values in $[-10^4, 10^4]$) and `k`, return the kth largest element in sorted order, not the kth distinct one, ideally without sorting: `[3,2,1,5,6,4]` with `k = 2` gives 5.

Intuition: The pivot's index after a partition is final, so the side without `n - k` is never needed again. On distinct values a random pivot shrinks the window by a constant fraction on average, so the passes add up to expected `O(n)`. Here values must repeat, and a long run of equal values defeats the `<` test.

Differs: the general solution aimed at sorted index `n - k`, with a three-way partition as in LC 912: return once `n - k` falls inside the block equal to the pivot, else chase the side that holds it.

```
lt, gt = three_way(nums, lo, hi)    # nums[lt..gt] all equal the pivot
if lt <= target <= gt:
    return nums[target]
```

LC 973 · K Closest Points to Origin

Problem: Given points `[x, y]` and `k`, return the `k` points closest to the origin by Euclidean distance, in any order: `[[1,3], [-2,2]]` with `k = 1` gives `[[-2,2]]`.

Intuition: The `k` closest need not be sorted among themselves: once a pivot lands at index `k - 1`, everything before it is closer. Squared distances keep the order of distances, so no square root is needed.

Differs: compare points by the key `x * x + y * y`, aim at index `k - 1`, and return the first `k` points of the array.

LC 912 · Sort an Array

Problem: Sort `nums` ascending without built-in sort functions, in `O(n log n)` time and as little space as possible; up to `5·10⁴` values in `[-5·10⁴, 5·10⁴]`: `[5,2,3,1]` gives `[1,2,3,5]`.

Intuition: Quicksort is partition with both sides chased: every pivot lands at its final place, and a random pivot gives `O(n log n)` on average, in place. Long runs of equal values are possible, and a two-way split peels them one at a time; a three-way split finishes the whole equal block.

Differs: chase both sides: recurse into `[lo, lt - 1]` and `[gt + 1, hi]` after a three-way partition around the random pivot, so the block equal to it is final and skipped. A range of fewer than two values returns at once.

```
def quick_sort(nums, lo, hi):
    if lo < hi:
        lt, gt = three_way(nums, lo, hi)
        quick_sort(nums, lo, lt - 1)
        quick_sort(nums, gt + 1, hi)
```

Peel · Peel

PEEL the low digit or bit off a number, use it, and shrink the rest.

Picture. A number loses its last digit at every step like an onion its outer skin; the peeled digits pile up on one side while what is left shrinks toward zero.

Invariant. What is peeled and what is left still make the original: `n · base^k` plus the value of the `k` digits peeled so far, or `result · x^n` for a power.

Use when. convert to or from base b, column titles, short codes; numbers given as strings or digit arrays, “without converting to integers”; a huge exponent, modulo `10⁹ + 7`; set bits, powers of two; gcd.

Book entries. #100 URL Shortener · #125 Multiply Strings · #128 GCD and LCM · #130 Fast Exponentiation · #136 Count Set Bits · #145 Unique ID Generator

GENERAL SOLUTION

Problem. Write a non-negative integer `n` in a base: 44 in base 2 is “101100”. And raise `x` to a power `n >= 0` with few multiplications: `power(3, 13)` gives 1594323.

Intuition. `n % base` is the last digit and `n // base` is the number with that digit cut off, so each step peels one digit, uses it, and keeps a smaller number; the loop ends when nothing is left. Fast power peels the bits of `n`: 13 is 1101 in binary, so `x^13 = x^8 · x^4 · x^1`, and squaring `x` each step supplies those factors. `n` halves every step, like a binary search.

```
n   low bit  result            x after squaring
13  1        3                 9
6   0        3                 81
3   1        243 = 3^5         6561
1   1        1594323 = 3^13    (n becomes 0)
```

```python
DIGITS = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"


def to_base(n, base):
    if n == 0:
        return DIGITS[0]
    out = []
    while n > 0:
        low = n % base              # Peel: the low digit
        out.append(DIGITS[low])     # use it
        n = n // base               # shrink the rest
    return "".join(reversed(out))   # peeled lowest first


def power(x, n):
    result = 1
    while n > 0:
        if n % 2 == 1:              # Peel: the low bit of n
            result = result * x     # use it: this square counts
        x = x * x                   # x, x^2, x^4, ...
        n = n // 2                  # shrink the rest
    return result
```

Example.

```python
print(to_base(44, 2))                    # 101100
print(power(3, 13))                      # 1594323
```

Dials. The base (10, 2, 26, 62), what a peeled digit does (becomes a character, adds into a slot with a carry, multiplies a square in, counts a bit), how the rest shrinks (`// base`, `n & (n - 1)`, Euclid's `(a, b) → (b, a % b)`), and whether two numbers are peeled side by side.

Traps. Forgetting `n == 0`, which peels nothing and returns an empty string; forgetting to reverse, since the low digit comes out first; dropping the last carry after the loop.

PROBLEMS

LC 168 · Excel Sheet Column Title

Problem: Given an integer `columnNumber`, `1 <= columnNumber <= 2^31 - 1`, return its Excel column title, where A is 1, Z is 26, AA is 27: 28 gives "AB", and 701 gives "ZY".

Intuition: Columns count in bijective base 26: the digits A..Z stand for 1..26 and there is no zero, so 26 is "Z" and 27 is "AA". Shifting each digit down by one before it is peeled turns it into ordinary base 26 with digits 0..25.

Differs: subtract one before each peel: `n -= 1`, then the letter is `chr(ord("A") + n % 26)` and `n //= 26`. No zero case is needed.

```python
while n > 0:
    n -= 1
    out.append(chr(ord("A") + n % 26))
    n //= 26
```

LC 43 · Multiply Strings

Problem: Given two non-negative integers as strings `num1` and `num2`, return their product as a string, without converting the inputs to integers or using a big-integer library: "123" and "456" give "56088".

Intuition: Long multiplication. Digit `i` times digit `j` has the place value of slot `i + j + 1` in an array of `m + n` slots, the most digits an `m`-digit times an `n`-digit product can have. Each partial product adds into its slot and carries left.

Differs: peel the digits of both strings from the right in a double loop; the product adds into slot `i + j + 1`, its tens carry into slot `i + j`. Return `"0"` early; drop one leading zero.

```python
total = digits[i + j + 1] + int(num1[i]) * int(num2[j])
digits[i + j + 1] = total % 10
digits[i + j] += total // 10
```

LC 415 · Add Strings

Problem: Given two non-negative integers as strings, return their sum as a string, without converting the inputs to integers or using a big-integer library: “11” and “123” give “134”.

Intuition: Column addition. A column’s sum depends only on its two digits and the carry from the column to its right, so one right-to-left pass peels a digit from each string and writes one digit of the answer.

Differs: peel one digit from the right of each string, 0 once a string runs out; add both and the carry, keep `total % 10`, carry `total // 10`. Append a last carry, then reverse.

```
total = carry
total += int(num1[i]) if i >= 0 else 0
total += int(num2[j]) if j >= 0 else 0
out.append(str(total % 10))
carry = total // 10
```

LC 67 · Add Binary

Problem: Given two binary strings `a` and `b`, return their sum as a binary string: “11” and “1” give “100”, and “1010” and “1011” give “10101”.

Intuition: The same column addition in base 2: each column holds two bits and a carry, so the column sum is 0 to 3, writing `sum % 2` and carrying `sum // 2`.

Differs: the loop of LC 415 in base 2: keep `total % 2` and carry `total // 2`.

LC 50 · Pow(x, n)

Problem: Given a float `x` and an integer `n` in `[-2^31, 2^31 - 1]`, return `x` raised to `n`: `x = 2.0, n = 10` gives 1024.0, and `x = 2.0, n = -2` gives 0.25.

Intuition: `n` multiplications are too slow for `n` near `2^31`. `|n|` has at most 32 bits, so `x^n` is a product of at most 32 of the squares `x, x^2, x^4, ...`. A negative power is the positive power of `1 / x`, and Python’s integers do not overflow on `-n`.

Differs: for a negative `n`, set `x = 1 / x` and `n = -n` first; then the general `power`.

LC 191 · Number of 1 Bits

Problem: Given a positive integer `n` below `2^31`, return the number of 1 bits in its binary form: 11 is 1011, which gives 3.

Intuition: `n - 1` flips the lowest 1 and the zeros below it, so `n & (n - 1)` wipes exactly the lowest set bit. Peel set bits instead of all bits, and the loop runs once per 1.

Differs: peel the lowest set bit, not the lowest bit: `n &= n - 1`, and count the steps until `n` is 0.

```
while n:
    n &= n - 1
    count += 1
```

LC 338 · Counting Bits

Problem: Given `n >= 0`, return a list `ans` of length `n + 1` where `ans[i]` is the number of 1 bits of `i`, ideally in one `O(n)` pass: `n = 5` gives `[0,1,1,2,1,2]`.

Intuition: Peeling the low bit of `i` leaves `i >> 1`, a smaller number whose count is already in the table. Each answer is one peel and one lookup: overlapping subproblems filled smallest first.

Differs: no loop per number: `bits[i] = bits[i >> 1] + (i & 1)` fills `0 .. n` in one pass.

Drain

A bag of pending work drives the loop.

The question. Who is next in line?

Picture. A waiting room with a ticket machine. Work arrives in the room, the machine calls one item at a time, and serving it may send new items into the room; the loop ends when the room is empty. The machine is the whole method: oldest ticket first, newest first, or cheapest first.

Invariant. Everything taken out of the bag is final, or is recognised as stale and skipped.

Shared skeleton.

```
bag = make_bag(seeds)               # dial: queue, stack or heap; the seeds
while bag:
    node = take(bag)                # the machine calls the next ticket
    if is_stale(node):              # dial: claimed at take, skip old copies
        continue
    process(node)                   # node is final here
    for nxt in expand(node):
        if admit(nxt):              # dial: unseen, cheaper, indegree zero
            put(bag, nxt)           # claimed at put: mark it right here
```

Dials. The bag: a queue serves the oldest (rings of equal cost), a stack the newest (dive deep, back up), a heap the cheapest. When a node is claimed: at put, when the first discovery is final (wave, and a diver that only needs reachability); at take, with a stale check, when a later discovery can beat an earlier one (cheapest first, an exact depth-first order). The admit rule: unseen, a strictly better key, an indegree that reached zero. The seeds: one source, every source, every unvisited node in turn. Keep the best turns the bag around: the heap holds the answer and the item taken out is the one least worth keeping.

Method	Mnemonic	What it sets	Book entries
Wave	Poprexen	queue, claim at put, rounds when levels matter	#32, #39, #40, #78, #79, #80, #83, #114
Diver	Poprexpu	stack or recursion, a restart loop over all starts	#37, #38, #49, #77, #84, #115
Cheapest first	PoStaRePu	heap keyed by cost, claim at take, skip stale	#20, #42, #46, #119, #149
Keep the best	Putri	heap as bounded memory: push, trim to k	#19, #21, #59, #94, #109, #140

Drain and Hold both pop from a container, but in Hold the input drives the loop and the container is memory, while here the container drives: the loop runs until the bag is empty, however long that takes. The family-wide trap is claiming at the wrong moment. Claim at put with a heap and the first route found to a node freezes, not the cheapest one; claim at take with a queue and forget the stale check, and one node is processed many times.

Wave · Poprexen

POP, PRocess, EXpand, ENqueue: the queue serves the oldest first.

Picture. A ripple spreads from the sources one ring at a time; every node is reached by the first ring that touches it.

Invariant. The queue holds the frontier in order of distance, at most two distances deep, and every node in `dist` already has its final distance.

Use when. fewest steps, fewest moves, minimum number of transformations, nearest, level by level, minutes until everything spreads; every step costs the same.

Book entries. #32 Binary Tree Level-Order Traversal · #39 BFS Shortest Path · #40 Kahn's Algorithm · #78 Multi-Source BFS · #79 Word Ladder · #80 Clone Graph · #83 Alien Dictionary · #114 Bipartite Check

GENERAL SOLUTION

Problem. Given an unweighted graph as an adjacency list and a list of source nodes, return the fewest edges from the nearest source to every reachable node. With edges 0–1, 0–2, 1–3, 2–3, 3–4 and source 0 the answer is `{0: 0, 1: 1, 2: 1, 3: 2, 4: 3}`.

Intuition. Drop a stone at every source. The ripple reaches every node at distance 1, then every node at distance 2, because a queue serves nodes in the order they were found. The first ring that reaches a node is the nearest one, so a node is final the moment it is put into the queue.

```
pop   queue after   written
0     [1, 2]        dist[1] = 1, dist[2] = 1
1     [2, 3]        dist[3] = 2
2     [3]           (3 is already seen)
3     [4]           dist[4] = 3
```

```python
from collections import deque


def wave(graph, sources):
    dist = {s: 0 for s in sources}        # claim at put: seen means final
    queue = deque(sources)
    while queue:
        node = queue.popleft()            # Pop: the oldest waiting node
        # Process: dist[node] is final here; record or test the node
        for nxt in graph[node]:           # Expand: every neighbour
            if nxt not in dist:           # admit only a first sighting
                dist[nxt] = dist[node] + 1
                queue.append(nxt)         # Enqueue: joins the back
    return dist
```

Example.

```python
graph = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]}
print(wave(graph, [0]))                   # {0: 0, 1: 1, 2: 1, 3: 2, 4: 3}
print(wave(graph, [4, 0]))                # {4: 0, 0: 0, 3: 1, 1: 1, 2: 1}
```

Dials. What a node is (a tree node, a cell, a word, a lock code, a course), how its neighbours are generated, the seeds (one or many), the admission rule (unseen, indegree zero, uncoloured), and whether rounds matter (freeze `len(queue)` at the top of each round).

Traps. Writing the distance at put but checking `seen` at pop lets a later, longer route overwrite a right answer; claim and record at the same moment. Forgetting to mark the seeds lets them re-enter. `list.pop(0)` makes every pop linear; use `deque`.

PROBLEMS

LC 102 · Binary Tree Level Order Traversal

Problem: Given the root of a binary tree, return its values level by level, each level from left to right: `[3,9,20,null,null,15,7]` gives `[[3],[9,20],[15,7]]`.

Intuition: Peel the tree one horizontal layer at a time. At the start of a round the queue holds exactly one whole level; the children those nodes add wait behind them as the next level. A tree has no second route into a node, so nothing needs a seen mark.

Differs: rounds: freeze `size = len(queue)` at the top of each round and pop exactly `size` nodes into one level list. No `dist`, no seen set.

```python
for _ in range(len(queue)):               # one round is one level
    node = queue.popleft()
    level.append(node.val)
```

LC 199 · Binary Tree Right Side View

Problem: Given the root of a binary tree, return the values seen by someone standing on its right, top to bottom: `[1,2,3,null,5,null,4]` gives `[1,3,4]`.

Intuition: From the right you see one node per level: the rightmost. Rounds hand you each level left to right, because every parent enqueues its left child before its right one, so the last node of a round is the visible one.

Differs: rounds as in LC 102, but keep only the value of the last node popped in each round.

LC 994 · Rotting Oranges

Problem: In a grid, 0 is empty, 1 a fresh orange and 2 a rotten one. Each minute every fresh orange next to a rotten one (up, down, left, right) rots. Return the minutes until none is fresh, or -1 if some never rot: `[[2,1,1],[1,1,0],[0,1,1]]` gives 4.

Intuition: Rot spreads from all rotten oranges at once and at the same speed, like several ripples started together. One queue seeded with every rotten orange gives each fresh one its distance to the nearest source; the number of rounds is the minutes.

Differs: seed every rotten cell, count the fresh ones, run rounds over the four neighbours; rotting a cell is the mark. Stop when no fresh orange is left; if some remain, return -1.

LC 127 · Word Ladder

Problem: Turn `beginWord` into `endWord` by changing one letter at a time, every word on the way in `wordList`. Return the number of words in the shortest such ladder, or 0: "hit" to "cog" through hot, dot, dog gives 5.

Intuition: Words are nodes and a one-letter change is an edge of cost one, so the shortest ladder is the first time the ripple reaches `endWord`. The ladder counts words, which is edges plus one.

Differs: Expand tries every one-letter change of the word and keeps those still in the dictionary set, removing each from the set as it is enqueued. Return the distance plus one.

```
for i in range(len(word)):
    for ch in "abcdefghijklmnopqrstuvwxyz":
        cand = word[:i] + ch + word[i + 1:]
```

LC 752 · Open the Lock

Problem: A lock has four wheels of digits 0 to 9 and starts at "0000"; a move turns one wheel one step, and 9 wraps to 0. Some codes are deadends. Return the fewest moves to reach `target`, or -1: target "0202" with deadends 0201, 0101, 0102, 1212, 2002 gives 6.

Intuition: Each of the 10,000 codes is a node with eight neighbours, one step up or down on each wheel, and every move costs one, so the ripple's distance is the fewest moves. A deadend is simply a node the ripple may not enter.

Differs: nodes are 4-digit strings with eight generated neighbours. Put every deadend into the seen set before starting; if "0000" is a deadend, return -1.

LC 133 · Clone Graph

Problem: Given one node of a connected undirected graph, where each node has a value and a list of neighbours, return a deep copy of the whole graph.

Intuition: Walk the graph once and make each copy the first time you meet its original. A map from original to copy is both the visited mark and the lookup that wires copies to copies, so every node is copied once and every edge is rebuilt.

Differs: the seen map stores *old node → its copy*. On a first sighting create the copy and enqueue the node; on every edge append `copies[nxt]` to `copies[node].neighbors`.

LC 210 · Course Schedule II

Problem: There are `numCourses` courses; a pair `[a, b]` means b must be taken before a. Return an order that takes them all, or `[]` if that is impossible: 2 courses with `[[1,0]]` give `[0,1]`.

Intuition: Free the courses with no prerequisite first. Taking a course removes one requirement from each course that waits on it, and a course enters the queue when its last requirement is gone. Courses on a cycle wait on each other and never become free.

Differs: Kahn's admission rule: seed every course with indegree 0; when a course is popped, decrement each successor and enqueue it when its count reaches 0. Return the pop order, or `[]` if fewer than `numCourses` were popped.

```
for nxt in graph[course]:
    indegree[nxt] -= 1
    if indegree[nxt] == 0:
        queue.append(nxt)
```

LC 785 · Is Graph Bipartite?

Problem: An undirected graph is given as `graph[u]`, the list of u's neighbours. Return true if its nodes split into two groups with every edge between the groups: `[[1,3],[0,2],[1,3],[0,2]]` is bipartite.

Intuition: Paint the ripple's rings in alternating colours. An edge that joins two nodes of the same colour closes an odd cycle, and a graph can be split exactly when it has none.

Differs: the mark is a colour: a new neighbour gets the colour opposite to the node's, and a seen neighbour with the same colour returns `False`. Restart from every uncoloured node, because the graph may be disconnected.

Diver · Poprexpu

POP, PRocess, EXpand, PUsh: the stack serves the newest first.

Picture. A diver follows one corridor to its end, then backs up to the last junction that still has an unopened door; a restart from every untouched node floods the components one by one.

Invariant. Every node in `seen` has been processed exactly once; the stack holds the doors still to open, the newest on top, and a copy of a node already in `seen` is skipped when it surfaces.

Use when. how many islands, groups, provinces or components; can I reach, is there a path; flood fill, recolour a region, everything connected to the border; iterate lazily over a nested or tree structure, one value per call. Order of discovery does not matter, or must be depth-first.

Book entries. #37 Depth-First Search on a Graph · #38 Number of Islands · #49 Flatten Nested List Iterator · #77 BST Iterator · #84 Connected Components · #115 Cycle Detection in an Undirected Graph

GENERAL SOLUTION

Problem. Given an undirected graph as an adjacency list, return its connected components, each listed in depth-first order. With edges 0–1, 0–2, 1–3 and 4–5 the answer is `[[0, 1, 3, 2], [4, 5]]`.

Intuition. The stack always serves the newest door, so the walk follows one corridor to its end before it backs up to the last junction. A node is claimed when it is popped and an older copy of it is skipped, so the order matches the recursion exactly. Reachability in an undirected graph is symmetric, so one dive floods one whole component and every restart finds a new one.

```
pop   stack after   order
0     [2, 1]        [0]
1     [2, 3]        [0, 1]
3     [2]           [0, 1, 3]
2     []            [0, 1, 3, 2]
4     [5]           [4]            (restart: 4 untouched)
5     []            [4, 5]
```

```python
def dive(graph, start, seen):
    order = []
    stack = [start]
    while stack:
        node = stack.pop()                # Pop: the newest waiting node
        if node in seen:                  # stale copy: reached another way
            continue
        seen.add(node)                    # claim at take
        order.append(node)                # Process: record or test the node
        for nxt in reversed(graph[node]): # Expand: first neighbour on top
            if nxt not in seen:
                stack.append(nxt)         # Push
    return order


def components(graph):
    seen = set()
    groups = []
    for start in graph:                   # restart from every untouched node
        if start not in seen:
            groups.append(dive(graph, start, seen))
    return groups
```

Example.

```
graph = {0: [1, 2], 1: [0, 3], 2: [0], 3: [1], 4: [5], 5: [4]}
print(components(graph))                # [[0, 1, 3, 2], [4, 5]]
```

Dials. What a node is (a cell, a room, a nested list, a tree node), the mark (a `seen` set, or overwriting the cell), the seeds (one start, every untouched node, the border), what Process records (a count, an area, a label), and whether the dive pauses after each pop to answer a caller. Carrying the parent with each node turns the dive into an undirected cycle test: a seen neighbour that is not the parent closes a cycle.

Traps. Recursion on a 300 × 300 grid of land goes 90,000 calls deep, far past Python's limit of 1000; use the explicit stack. Dropping the `row < 0` test lets `grid[-1]` read the last row. Marking at push is fine for reachability but gives a different order than depth-first.

PROBLEMS

LC 200 · Number of Islands

Problem: Given an `m x n` grid of `"1"` (land) and `"0"` (water), return the number of islands: groups of land joined up, down, left or right. `[["1","1","0"],["0","0","1"]]` gives 2.

Intuition: An island is a connected component of land. A dive sinks the whole island it starts on, so the scan meets dry land again only on an island no earlier dive reached, and every restart counts a new island.

Differs: nodes are cells; Expand pushes the four neighbours and the claim at take skips cells off the map or not `"1"`. Sinking a cell to `"0"` is the mark. The restart loop scans every cell.

```
row, col = stack.pop()
if 0 <= row < rows and 0 <= col < cols and grid[row][col] == "1":
    grid[row][col] = "0"                # sinking is the claim
    stack += [(row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)]
```

LC 695 · Max Area of Island

Problem: Given an `m x n` grid of 0 and 1, return the area (number of cells) of the largest island of 1s joined up, down, left or right, or 0 if there is no land: `[[1,1,0],[0,0,1]]` gives 2.

Intuition: Each land cell is claimed exactly once, by the dive of its own island, so the number of cells a dive sinks is the area of that island. The answer is the largest such count.

Differs: as LC 200 on integers, but Process counts each cell it sinks; the dive returns the count and the restart loop keeps the maximum, starting from 0.

LC 130 · Surrounded Regions

Problem: Given an `m x n` board of `'X'` and `'O'`, flip in place every region of `'O'` that does not touch the border into `'X'`: in `[["X","X","X"],["X","O","X"],["X","X","O"]]` the centre `'O'` flips and the corner `'O'` stays (diagonals do not join).

Intuition: A region survives exactly when one of its cells sits on the border. Flood from the border inward and mark what is reached as safe; every `'O'` left unmarked is enclosed. A dive from an inner region would learn only at its end whether to flip.

Differs: the seeds are every `'O'` on the border; the dive marks what it reaches `'S'`. A final sweep turns each remaining `'O'` into `'X'` and each `'S'` back into `'O'`.

LC 841 · Keys and Rooms

Problem: Rooms `0..n-1` are locked except room 0; `rooms[i]` lists the keys found in room i, each opening one room. Return true if every room can be visited: `[[1],[2],[3],[]]` gives true, `[[1,3],[3,0,1],[2],[0]]` gives false.

Intuition: A key opens its room for good and the order of visits does not matter, so the rooms you can visit are exactly the nodes reachable from room 0 along key edges.

Differs: one dive from room 0 and no restart loop; the neighbours of a room are the keys it holds. Return `len(seen) == len(rooms)`.

LC 1971 · Find if Path Exists in Graph

Problem: Given `n` vertices `0..n-1`, a list of bidirectional `edges`, a `source` and a `destination`, return true if a path joins them: `n = 3`, `edges = [[0,1],[1,2],[2,0]]`, 0 to 2 gives true.

Intuition: The edges go both ways, so a path exists exactly when `destination` lies in the component of `source`. The dive can stop the moment the target surfaces; the rest of the component does not matter.

Differs: build adjacency lists from `edges` in both directions, dive from `source`, and return `True` as soon as `destination` is popped; `False` when the stack empties.

LC 341 · Flatten Nested List Iterator

Problem: Given a nested list of integers, where each item is an integer or a list, build an iterator with `next()` and `hasNext()` that yields the integers in order: `[[1,1],2,[1,1]]` yields 1, 1, 2, 1, 1.

Intuition: The flattening is a depth-first walk that pauses each time an integer surfaces. The stack holds the items still to go, the next one on top, so opening a list means pushing its items reversed. Empty lists vanish inside `hasNext`, so it never promises an integer that is not there.

Differs: the dive pauses. The stack starts as `nestedList` reversed; `hasNext` pops lists from the top and pushes their items reversed until an integer is on top; `next` pops it and returns `getInteger()`.

```
while self.stack and not self.stack[-1].isInteger():
    items = self.stack.pop().getList()    # open the list on top
    self.stack.extend(reversed(items))    # its first item ends on top
```

LC 173 · Binary Search Tree Iterator

Problem: Build an iterator over a binary search tree that returns its values in ascending order: `next()` gives the next value, `hasNext()` whether one is left. For `[7,3,15,null,null,9,20]`, `next()` returns 3, 7, 9, 15, 20. Aim for `O(h)` memory.

Intuition: In-order visits a node right after its whole left subtree. The stack holds the pending nodes of one root-to-leaf path, each under the nodes of its left subtree: the top is the smallest value left, and memory is `O(h)`. Each node is pushed and popped once: `O(1)` amortized.

Differs: the stack starts as the left spine of the root. `next` pops the smallest waiting node, then pushes the left spine of its right child; `hasNext` is `len(self.stack) > 0`.

```
node = self.stack.pop()                  # Pop: the smallest still waiting
child = node.right
while child:                             # Push: the left spine on the right
    self.stack.append(child)
    child = child.left
```

Cheapest first · PoStaRePu

POp, skip STAle, RElax, PUsh: the heap serves the cheapest first.

Picture. A heap of offers, one per cheaper route found to a node; the cheapest is always taken next, its node is settled at that price, and the dearer copies that surface later are thrown away.

Invariant. Every settled node has its final cost, no smaller than any cost taken before it; the heap holds offers for the rest, some of them stale.

Use when. cheapest, shortest weighted path, minimum time, cost or effort, maximum probability, connect everything at minimum cost, merge k sorted sources, the k smallest sums, the 10 most recent; every step costs something, and no step makes a route cheaper.

Book entries. #20 Merge K Sorted Arrays · #42 Dijkstra's Algorithm · #46 Design Twitter · #119 Prim's Minimum Spanning Tree · #149 Job Scheduler

GENERAL SOLUTION

Problem. Given a directed graph with non-negative edge weights and a source, return the cheapest total cost from the source to every reachable node. With edges 1→2 (4), 1→3 (1), 3→2 (2), 2→4 (1), 3→4 (5) and source 1 the answer is `{1: 0, 3: 1, 2: 3, 4: 4}`.

Intuition. No weight is negative, so a route only gets dearer as it grows. When the cheapest offer in the heap names a node, no offer still waiting can reach it cheaper: the node is settled at that price. A cheaper route found later pushes a new offer, so a node may have several copies; the dearer ones surface after it is settled and are skipped.

```
pop      result       pushed
(0, 1)   1 = 0        (4, 2), (1, 3)
(1, 3)   3 = 1        (3, 2), (6, 4)
(3, 2)   2 = 3        (4, 4)
(4, 2)   stale        2 was settled at 3
(4, 4)   4 = 4
(6, 4)   stale        4 was settled at 4
```

```python
import heapq


def cheapest_first(graph, source):
    dist = {source: 0}                    # best offer so far, final at take
    heap = [(0, source)]
    while heap:
        d, node = heapq.heappop(heap)     # Pop: the cheapest offer
        if d > dist[node]:                # skip Stale: settled cheaper
            continue
        for nxt, weight in graph[node]:   # Relax every edge out of node
            new = d + weight
            if nxt not in dist or new < dist[nxt]:
                dist[nxt] = new
                heapq.heappush(heap, (new, nxt))   # Push the better offer
    return dist
```

Example.

```python
graph = {1: [(2, 4), (3, 1)], 2: [(4, 1)], 3: [(2, 2), (4, 5)], 4: []}
print(cheapest_first(graph, 1))           # {1: 0, 2: 3, 3: 1, 4: 4}
```

Dials. The key: a summed distance (Dijkstra), a product popped largest first, the largest step on the path, the weight of the single edge into a tree (Prim), the front value of each sorted list (k-way merge), a run time (job scheduler). The stale test: a worse cost, already in the tree, or a cancelled job. The seeds: one source, the head of every list, the whole border. The stop: when the target is popped, after k pops, or when the heap is empty.

Traps. Claiming at push, as in a wave, freezes the first route found instead of the cheapest. A negative edge breaks the method: a settled node can get cheaper later; use Bellman-Ford. Pushing `(value, node)` with list nodes raises `TypeError` when two values tie; put an index in the middle.

PROBLEMS

LC 23 · Merge k Sorted Lists

Problem: Given an array of k linked lists, each sorted ascending, merge them into one sorted linked list and return it: `[[1,4,5],[1,3,4],[2,6]]` gives `[1,1,2,3,4,4,5,6]`.

Intuition: Every list is sorted, so the smallest value not yet merged is the front of some list. The heap holds at most k fronts, one per list, and the smallest comes out next: `O(N log k)` for N nodes in all.

Differs: the key is the front value of each list: seed `(head.val, i, head)` for every non-empty list; each pop links the node to the tail and pushes its `next`. No stale test: each node enters once. `i` breaks ties before nodes are compared.

```python
while heap:
    val, i, node = heapq.heappop(heap)    # Pop: the smallest front
    tail.next = node
    tail = node
    if node.next:                         # Push: the successor in list i
        heapq.heappush(heap, (node.next.val, i, node.next))
```

LC 373 · Find K Pairs with Smallest Sums

Problem: Given two arrays sorted ascending and an integer k, return the k pairs `[u, v]`, u from `nums1` and v from `nums2`, with the smallest sums: `[1,7,11]`, `[2,4,6]`, `k = 3` gives `[[1,2],[1,4],[1,6]]`.

Intuition: Fix `nums1[i]`: its pairs with `nums2[0], nums2[1], ...` form a sorted list of sums. That makes one sorted list per row, and the answer is the first k items of their merge. Only the next pair in a row can become the cheapest after a pop.

Differs: a node is an index pair `(i, j)` keyed by `nums1[i] + nums2[j]`. Seed `(i, 0)` for the first `min(k, len(nums1))` rows, pop k times, and after `(i, j)` push only `(i, j + 1)`.

```
for _ in range(k):
    total, i, j = heapq.heappop(heap)
    pairs.append([nums1[i], nums2[j]])
    if j + 1 < len(nums2):
        heapq.heappush(heap, (nums1[i] + nums2[j + 1], i, j + 1))
```

LC 743 · Network Delay Time

Problem: Nodes are labelled 1 to n; `times[i] = (u, v, w)` is a directed edge taking w time. A signal leaves node k; return the time until all n nodes receive it, or -1: `[[2,1,1],[2,3,1],[3,4,1]]`, `n = 4`, `k = 2` gives 2.

Intuition: The signal reaches each node along its cheapest path, so the last node to hear it is the one with the largest shortest distance. No `w` is negative, so cheapest first settles each node at its true arrival time.

Differs: the general solution from `k` over the directed `times`; the answer is the largest value in `dist`, or -1 when fewer than n nodes are in it.

LC 1514 · Path with Maximum Probability

Problem: An undirected graph of n nodes has `edges[i]` succeeding with probability `succProb[i]`. Return the highest success probability of a path from `start_node` to `end_node`, or 0: edges `[[0,1],[1,2],[0,2]]`, probabilities `[0.5,0.5,0.2]`, 0 to 2 gives 0.25.

Intuition: A path's value is the product of its edges, and every probability is at most 1, so extending a path never raises its value. That is the mirror of non-negative weights: take the most likely offer first, and its node is settled at that value.

Differs: the key is the product, popped largest first: push `(-prob, node)`, relax when `prob * p` beats the best so far, return on popping `end_node`, and 0 if it never surfaces.

LC 1631 · Path With Minimum Effort

Problem: Walk a grid `heights` from the top-left to the bottom-right cell, moving up, down, left or right. A route's effort is its largest absolute height difference between consecutive cells; return the least effort: `[[1,2,2],[3,8,2],[5,3,5]]` gives 2.

Intuition: The cost of a route is its largest step, and `max` with a new step never lowers it, so cost only grows along a path. That is all cheapest first needs: the first time the corner is popped, no waiting route can reach it with less effort.

Differs: nodes are cells; relax with `new = max(d, abs(heights[r][c] - heights[nr][nc]))` instead of a sum. Return `d` when the bottom-right cell is popped.

LC 1584 · Min Cost to Connect All Points

Problem: Given points `[x, y]`, the cost to connect two is their Manhattan distance `|x1 - x2| + |y1 - y2|`. Return the minimum cost to connect them all: `[[3,12],[-2,5],[-4,1]]` gives 18.

Intuition: Grow one tree from any point. The cheapest edge leaving the tree belongs to some minimum spanning tree (the cut property), so always join the point that is cheapest to attach. A cheaper edge to a point can turn up after the first one, so claim at take.

Differs: Prim: the key is the distance of the single edge into the tree, not a path sum; the stale test is "already in the tree"; every other point is a neighbour; stop after n claims.

```
cost, node = heapq.heappop(heap)
if node not in tree:                    # skip Stale: joined cheaper
    tree.add(node)
    total += cost                       # key: one edge, not a path
```

LC 355 · Design Twitter

Problem: Design `postTweet(userId, tweetId)`, `follow`, `unfollow` and `getNewsFeed(userId)`, which returns the 10 most recent tweet ids posted by the user or anyone they follow, newest first: user 1 posts 5, follows 2, 2 posts 6; the feed of 1 is `[6, 5]`.

Intuition: Each user's tweets are already in time order, so the feed is a merge of sorted lists cut after 10 items. The heap holds the newest unread tweet of each source, so the merge reads one tweet per source plus at most 10 more, however long the histories are.

Differs: `getNewsFeed` is the k-way merge keyed by a global post counter, newest first: seed the newest tweet of the user and of each followee as `(-time, ...)`, push that source's next older tweet after each pop, stop after 10.

LC 407 · Trapping Rain Water II

Problem: Given an `m x n` map of cell heights, return the volume of water it traps after rain: `[[3,3,3],[3,1,3],[3,3,3]]` gives 2, since the centre fills up to the wall at height 3.

Intuition: Water in a cell escapes over the lowest point of the wall around it. Keep the wall as a heap of boundary cells and breach it at its lowest point: every escape crosses the wall, so a cell first reached there fills up to that level, if lower, and joins the wall.

Differs: the seeds are all border cells keyed by height, marked at push. Pop the lowest boundary cell; each unvisited neighbour holds `max(0, level - h)` water and enters with key `max(level, h)`.

```
level, r, c = heapq.heappop(heap)         # the lowest point of the wall
for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
    if 0 <= nr < m and 0 <= nc < n and (nr, nc) not in seen:
        seen.add((nr, nc))
        water += max(0, level - heightMap[nr][nc])
        heapq.heappush(heap, (max(level, heightMap[nr][nc]), nr, nc))
```

Keep the best · Putri

PUsh, TRIm: a heap as bounded memory.

Picture. A room with k seats and a bouncer at the door: every newcomer is let in, and when the room overflows, the weakest person inside is shown out. The heap top is always that weakest one.

Invariant. The heap holds the k best items seen so far, and its top is the weakest of them: the k-th best.

Use when. k-th largest, top k, k most frequent, the running median, at most k projects, smash the two heaviest, the maximum and minimum under corrections; the data arrives over time or is too large to sort, and only a few ranks are ever asked for.

Book entries. #19 Top-K Elements · #21 Find Median from Data Stream · #59 Heapify · #94 Cache with Expiring Keys · #109 Heap Sort · #140 Stock Price Tracker

GENERAL SOLUTION

Problem. Given a stream of numbers and k, report after every arrival the k-th largest value so far, or `None` while fewer than k have arrived. The stream 3, 1, 5, 12, 2, 11 with `k = 3` gives `[None, None, 1, 3, 3, 5]`.

Intuition. The k-th largest is the smallest of the k largest. A min-heap with k seats keeps the k largest so far with the weakest on top: every newcomer enters, and when the room overflows the top is shown out. A value shown out is beaten by k values already seen, so it can never be among the top k again.

```
value   heap after trim   answer
3       [3]               None
1       [1, 3]            None
5       [1, 3, 5]         1
12      [3, 5, 12]        3      (1 shown out)
2       [3, 5, 12]        3      (2 shown out at once)
11      [5, 11, 12]       5      (3 shown out)
```

```python
import heapq


def keep_best(stream, k):
    heap = []                               # min-heap: the weakest on top
    answers = []
    for value in stream:
        heapq.heappush(heap, value)         # PUsh the newcomer
        if len(heap) > k:
            heapq.heappop(heap)             # TRIm: show the weakest out
        if len(heap) == k:
            answers.append(heap[0])         # the k-th largest so far
        else:
            answers.append(None)            # fewer than k values yet
    return answers
```

Example.

```python
print(keep_best([3, 1, 5, 12, 2, 11], 3))     # [None, None, 1, 3, 3, 5]
```

Dials. The bound: k seats, or half of the data each for two heaps balanced at the median. The direction: a min-heap keeps the largest, a max-heap (negated values) keeps the smallest. The order: tuples, negation, or a custom `__lt__`. Removals: lazy deletion, where a map or a count of departed values says which tops are stale (the cache of #94, the tracker of #140). The feed: a second order that decides when an item may enter (IPO). Underneath every heap is sift-down (#59): heapify builds one in `O(n)`, and heap sort (#109) moves the top to the end n - 1 times.

Traps. A size-k max-heap for "the k largest" evicts exactly the values to keep. With negated values, dropping one minus sign mixes the halves silently. `//` in an even median turns 1.5 into 1. With lazy deletion, the sizes that balance the heaps must count live values only.

PROBLEMS

LC 703 · Kth Largest Element in a Stream

Problem: Build `KthLargest(k, nums)` whose `add(val)` adds a score and returns the k-th largest score so far: `k = 3`, `nums = [4,5,8,2]`, then `add(3)` returns 4, `add(5)` 5, `add(10)` 5, `add(9)` 8.

Intuition: This is the general solution as a class. The k-th largest is the top of a size-k min-heap, and values only arrive, so a value trimmed out can never climb back into the top k.

Differs: the constructor pushes `nums` with the trim; `add(val)` pushes, trims to k and returns `heap[0]`.

LC 295 · Find Median from Data Stream

Problem: Build `MedianFinder` with `addNum(num)` and `findMedian()`, the middle value or the mean of the two middle values: add 1 and 2, the median is 1.5; add 3, it is 2.0.

Intuition: The median needs only the one or two middle values. Split the data into a lower and an upper half, each a heap with its middle value on top, and keep the sizes equal or the lower half one ahead: `O(log n)` per add, `O(1)` per query.

Differs: two heaps, a max-heap `low` (negated) and a min-heap `high`. `addNum` passes the value through `low` into `high`, then moves the top of `high` back when `high` is larger. `findMedian` reads the tops.

```python
heapq.heappush(self.low, -num)
heapq.heappush(self.high, -heapq.heappop(self.low))   # largest of low
if len(self.high) > len(self.low):                    # low may lead by one
    heapq.heappush(self.low, -heapq.heappop(self.high))
```

LC 480 · Sliding Window Median

Problem: Given `nums` and a window size k, return the median of every window of k consecutive values as the window slides right by one: `[1,3,-1,-3,5,3,6,7]`, `k = 3` gives `[1,-1,-1,3,5,6]`.

Intuition: These are the two heaps of LC 295, but values also leave. `heapq` cannot delete from the middle, and a departed value only misleads when it sits at a top, so note it and drop it when it surfaces; each is dropped once, so the whole slide is `O(n log n)`.

Differs: lazy deletion: a value leaving the window is counted in `delayed[value]`, a balance counter tracks live sizes, and delayed tops are popped before any top is read.

```
while self.low and self.delayed[-self.low[0]] > 0:   # departed, on top
    self.delayed[-self.low[0]] -= 1
    heapq.heappop(self.low)
```

LC 502 · IPO

Problem: Start with capital `w`; project i needs `capital[i]` to start and adds `profits[i]`. Finish at most k distinct projects, one after another, and return the largest final capital: `k = 2`, `w = 0`, `profits = [1,2,3]`, `capital = [0,1,1]` gives 4.

Intuition: Capital never falls, so a project affordable now stays affordable: the pool of choices only fills. Taking the largest profit in the pool now can never block a better pick later, because it raises `w` the most and so opens the most projects.

Differs: the feed: sort projects by capital; before each of the k picks, push the profit of every project with `capital <= w` into a max-heap, then pop the largest profit into `w`. Stop early when the heap is empty.

```
while i < len(projects) and projects[i][0] <= w:
    heapq.heappush(pool, -projects[i][1])       # affordable for good
    i += 1
if pool:
    w -= heapq.heappop(pool)                    # add the largest profit
```

LC 1046 · Last Stone Weight

Problem: Each turn smash the two heaviest stones `x <= y`: equal ones both vanish, otherwise a stone of `y - x` remains. Return the weight of the last stone, or 0: `[2,7,4,1,8,1]` gives 1.

Intuition: Every turn needs the two heaviest of a pile that keeps changing, and a smash adds at most one stone back. A max-heap hands over the two heaviest and takes the remainder in `O(log n)` per turn.

Differs: no trim: heapify the negated stones once, then pop the two heaviest and push back their difference when it is not zero, until at most one stone is left; return it, or 0.

LC 2034 · Stock Price Fluctuation

Problem: Records `(timestamp, price)` arrive out of order, and a later record for a timestamp corrects the earlier one. Support `update`, `current` (price at the latest timestamp), `maximum` and `minimum`: update(1,10), update(2,5), update(1,3); maximum is 5.

Intuition: A correction changes an old record buried in the heap, which `heapq` cannot reach. A stale record only misleads when it reaches the top, and the map says whether it is still true; each stale record is popped once, so reads are `O(log n)` amortized.

Differs: a map `timestamp -> price` holds the truth, and every `update` pushes to a max-heap and a min-heap; `maximum` and `minimum` pop tops whose price no longer matches the map. `current` reads the map at the largest timestamp.

```
while self.prices[self.max_heap[0][1]] != -self.max_heap[0][0]:
    heapq.heappop(self.max_heap)                # corrected since: stale
```

LC 692 · Top K Frequent Words

Problem: Given `words` and k, return the k most frequent words, by frequency high to low and alphabetically on ties: `["i","love","leetcode","i","love","coding"]`, `k = 2` gives `["i","love"]`.

Intuition: Keep the k strongest words in a size-k heap whose top is the weakest: fewer occurrences, or on a tie the alphabetically later word. A string cannot be negated, so this order lives in `__lt__`; the heap gives the `O(n log k)` of the follow-up.

Differs: count the words first, then keep the best k with the custom order. Pop the k survivors, weakest first, and reverse them.

```python
class Entry:
    def __init__(self, count, word):
        self.count, self.word = count, word

    def __lt__(self, other):              # True when self is weaker
        return (self.count, other.word) < (other.count, self.word)
```

Branch

The call stack drives the loop.

The question. What do the smaller pieces say, and which way does the information flow?

Picture. A tree of calls. Information goes down through the arguments, the answers of the smaller pieces come back up through the return values, and the stack remembers the way home.

Shared skeleton.

```
def solve(piece, down):                  # down: what the caller already knows
    if is_base(piece):                   # dial: when a piece is small enough
        return base(piece)
    ups = []
    for sub in split(piece):             # dial: choices, children or halves
        ups.append(solve(sub, refine(down, sub)))   # dial: what goes down
    return combine(piece, ups)           # dial: what comes up
```

Dials. What a piece is (a partial choice, a cell, a subtree, a half of an array, a node of a graph), how it splits, what travels down in the arguments, what travels up in the return value, and what shared state is changed and then undone. The five methods are five answers to *which way does the information flow*.

```
Branch: the call stack drives the loop
├── state is shared and undone on the way back
│   ├── Enumerate    choices made and unmade; record the path
│   └── Path search  cells or squares marked and unmarked
├── return values come up
│   └── Combine      the work happens after the calls
├── arguments go down
│   └── Pass down    the work happens before the calls
└── marks on entry and exit
    └── Stamp        gray while open, black when finished
```

Method	Mnemonic	What it sets	Book entries
Enumerate	Rechexun	record when, where the loop starts, the duplicate rule	#30, #31, #73, #103, #137
Path search	Marexun	the mark, the moves, the prune	#74, #85, #113
Combine	Balerico	what a call returns, what it records globally	#17, #33, #34, #107, #148
Pass down	Bachepa	the constraint each call receives, the shared cursor	#35, #36, #75, #76
Stamp	Grabla	what gray and black mean, what is recorded on exit	#81, #82, #116

Branch is told apart from its neighbours by repetition. When the same piece is never solved twice (a tree, a list of choices, a graph whose nodes are stamped once), plain recursion is the whole answer. When the same smaller input comes back again and again, add a memo to *combine* and you are in **Fill**: dynamic programming is recursion without repetition. The family-wide trap is mixing the two flows: state that should travel in an argument kept in a shared variable that is never undone, or an answer that should come up in a return value written into a global that the siblings overwrite.

Enumerate · Rechexun

REcord, CHoose, EXplore, UNchoose: one shared path grows on the way down and shrinks on the way up.

Picture. A decision tree drawn depth first: each call stands on a node, the shared `path` spells the route from the root, and every branch is a choice appended, explored and popped again.

Invariant. At the top of every call, `path` holds exactly the choices on the route from the root to this node, and every other piece of shared state (`used`, counts, the sum left) agrees with it.

Use when. all subsets, all combinations, all permutations, every arrangement, every way to split, list them all; n up to about 10 to 20, because the answer itself is exponential.

Book entries. #30 Subsets · #31 Permutations · #73 Combination Sum · #103 Next Permutation · #137 Bitmask Subset Enumeration

GENERAL SOLUTION

Problem. Given an array of distinct integers, return all its subsets, the empty one included: `[1, 2, 3]` gives eight, `[]`, `[1]`, `[1, 2]`, `[1, 2, 3]`, `[1, 3]`, `[2]`, `[2, 3]` and `[3]`.

Intuition. Draw the decision tree in which every node is one subset. A call stands on a node and records it first; the loop then extends the path only with numbers to the right of the last choice, so each subset is built once, in index order. Popping after each branch hands the next sibling the path exactly as it was.

```
[]                  start 0: try 1, 2, 3
  [1]               start 1: try 2, 3
    [1, 2]          start 2: try 3
      [1, 2, 3]     start 3: nothing left
    [1, 3]          start 3: nothing left
  [2]               start 2: try 3
    [2, 3]          start 3: nothing left
  [3]               start 3: nothing left
```

```python
def subsets(nums):
    out = []
    path = []

    def backtrack(start):
        out.append(path[:])               # Record: copy, path keeps moving
        for i in range(start, len(nums)): # dial: where the loop starts
            path.append(nums[i])          # Choose nums[i]
            backtrack(i + 1)              # Explore: dial, what goes down
            path.pop()                    # Unchoose: path is as it was

    backtrack(0)
    return out
```

Example.

```python
print(subsets([1, 2, 3]))   # [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
```

Dials. Four dials make every member of the family, and the table is the whole answer to *what changes between subsets, combinations and permutations*:

Problem	Record when	Loop starts at	Pass down	Duplicate rule
LC 78 Subsets	every call	`start`	`i + 1`	none (distinct)
LC 90 Subsets II	every call	`start`	`i + 1`	sort; skip `i > start and nums[i] == nums[i-1]`
LC 77 Combinations	`len(path) == k`	`start` over 1..n	`i + 1`	none; prune when too few numbers remain
LC 39 Combination Sum	`remaining == 0`	`start`	`i` (reuse)	none; sort, `break` when `c[i] > remaining`
LC 40 Combination Sum II	`remaining == 0`	`start`	`i + 1`	sort; skip `i > start and c[i] == c[i-1]`
LC 216 Combination Sum III	`len == k` and `remaining == 0`	`start` over 1..9	`i + 1`	none (digits distinct)
LC 46 Permutations	`len(path) == n`	`0`	`used[i] = True`	skip `used[i]`
LC 47 Permutations II	`len(path) == n`	`0`	`used[i] = True`	sort; also skip `i > 0 and nums[i] == nums[i-1] and not used[i-1]`

The one-line rule: **subsets record every node and look forward; combinations record at depth k or at a hit sum and look forward; permutations record at depth n and look everywhere, with `used[]`.** Next permutation (#103) is the iterative alternative to the permutation tree, bitmask counting (#137) the iterative subsets. Counting the ways instead of listing them (LC 377, LC 518) belongs to Fill.

Traps. `out.append(path)` without the copy stores one list many times, and it ends empty. Passing `start = 0` to a combination lets the path move left, so `[2, 3]` and `[3, 2]` both come out. The duplicate skip is `i > start`, not `i > 0`: the first copy of a value at each level must still be taken.

PROBLEMS

LC 78 · Subsets

Problem: Given 1 to 10 distinct integers `nums`, return every subset in any order, with no subset twice: `[0]` gives `[[],[0]]`, and `[1,2,3]` gives eight subsets.

Intuition: Every node of the forward-looking tree is a different subset and every subset is a node, so recording at the top of each call lists each one exactly once. Ten numbers give at most 1,024 subsets, so listing them all is cheap.

Differs: nothing: this is the general solution.

LC 90 · Subsets II

Problem: As LC 78, but `nums` may contain duplicates and the answer must not repeat a subset: `[1,2,2]` gives `[[],[1],[1,2],[1,2,2],[2],[2,2]]`.

Intuition: After sorting, equal values sit side by side. At one level of the tree a second copy would open the very same subtree as the first, so only the first of a run may branch there; a deeper level may still take the next copy, which is how `[2,2]` appears.

Differs: sort `nums` first; inside the loop skip `nums[i]` when `i > start` and it equals `nums[i - 1]`. Record and pass down as in LC 78.

```
for i in range(start, len(nums)):
    if i > start and nums[i] == nums[i - 1]:
        continue                    # same value, same level: same subtree
```

LC 77 · Combinations

Problem: Given `n` and `k`, return all combinations of `k` numbers chosen from 1 to `n`, in any order: `n = 4, k = 2` gives `[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]`.

Intuition: A combination is a subset of fixed size, so only the nodes at depth k are answers. A branch that cannot reach depth k with the numbers left is dead before it starts, so the loop stops while enough numbers remain to fill the path.

Differs: choose from 1..n, start at 1; record only when `len(path) == k` and return. Prune: the last value the loop tries is `n - (k - len(path)) + 1`.

```
for i in range(start, n - (k - len(path)) + 2):   # room left to reach k
    path.append(i)
    backtrack(i + 1)
    path.pop()
```

LC 39 · Combination Sum

Problem: Given distinct positive `candidates` and a `target`, return every unique combination that sums to `target`; a candidate may be used any number of times: `[2,3,6,7]` with target 7 gives `[[2,2,3],[7]]`.

Intuition: Give each combination one canonical order: the path never moves left, and staying on the same index allows a repeat. All values are positive, so in sorted order the first candidate that overshoots proves every later one overshoots too.

Differs: pass `remaining` down and record when it hits 0. Explore with `i`, not `i + 1`, so a candidate can be chosen again. Sort and `break` at the first candidate larger than `remaining`.

```
if candidates[i] > remaining:
    break                           # sorted: the rest are bigger
path.append(candidates[i])
backtrack(i, remaining - candidates[i])   # i: reuse allowed
```

LC 40 · Combination Sum II

Problem: Given `candidates` that may repeat and a `target`, return every unique combination that sums to `target`, each candidate used at most once: `[10,1,2,7,6,1,5]` with target 8 gives `[[1,1,6],[1,2,5],[1,7],[2,6]]`.

Intuition: Used once means the path looks strictly forward. Repeated values bring back the Subsets II problem: without a skip, `[1,7]` is built once from each `1`. Sorted, only the first copy of a value may branch at each level.

Differs: as LC 39, but explore with `i + 1` and skip `candidates[i]` when `i > start` and it equals `candidates[i - 1]`.

LC 216 · Combination Sum III

Problem: Find all combinations of `k` distinct digits from 1 to 9 that sum to `n`: `k = 3, n = 7` gives `[[1,2,4]]`, and `k = 3, n = 9` gives `[[1,2,6],[1,3,5],[2,3,4]]`.

Intuition: Two conditions define an answer, exactly k numbers and the sum n. The digits are distinct and already sorted, so no duplicate rule is needed and a digit larger than what is left ends the loop.

Differs: the candidates are the digits 1 to 9, each used at most once (`i + 1`). Record only when `len(path) == k` and `remaining == 0`; return at depth k either way; `break` when `d > remaining`.

LC 46 · Permutations

Problem: Given 1 to 6 distinct integers, return all their orderings, in any order: `[0,1]` gives `[[0,1],[1,0]]`, and `[1,2,3]` gives six.

Intuition: Order matters, so at every level any unused value may come next, even one left of the last choice. A `used` flag per index is the only thing that keeps a value from appearing twice, and only full-length paths are orderings.

Differs: no `start`: the loop runs from 0 over all indices and skips those whose `used[i]` is set; set it before exploring, clear it after. Record when `len(path) == n`.

```
for i in range(len(nums)):
    if used[i]:
        continue
    used[i] = True
    path.append(nums[i])
    backtrack()
```

LC 47 · Permutations II

Problem: Given 1 to 8 numbers that may repeat, return all distinct orderings, in any order: `[1,1,2]` gives `[[1,1,2],[1,2,1],[2,1,1]]`.

Intuition: Sort so copies sit together, then force the copies of a value into the path from left to right: a copy may be used only once its left twin is already on the path. Each distinct ordering is then built once, not once per way to shuffle its copies.

Differs: sort first; skip `i` when `used[i]`, or when `i > 0`, `nums[i] == nums[i - 1]` and `used[i - 1]` is false. After exploring, pop and clear `used[i]` as in LC 46.

LC 31 · Next Permutation

Problem: Rearrange `nums` in place into the next larger ordering in dictionary order, or into ascending order if none is larger, with constant extra memory: `[1,2,3]` becomes `[1,3,2]`, and `[3,2,1]` becomes `[1,2,3]`.

Intuition: A suffix that never rises is already its own largest order, so it cannot grow. The smallest growth raises the value right before it, the pivot, by the least step, a swap with the smallest larger value in the suffix, then puts the suffix in ascending order.

Differs: no tree, the iterative alternative (#103): from the right find the pivot, the first `nums[i] < nums[i + 1]`; swap it with the rightmost larger value; reverse the suffix. With no pivot, reverse the whole array.

```
i = len(nums) - 2
while i >= 0 and nums[i] >= nums[i + 1]:
    i -= 1                          # walk left over the falling suffix
j = len(nums) - 1
while i >= 0 and nums[j] <= nums[i]:
    j -= 1                          # rightmost value above the pivot
```

LC 22 · Generate Parentheses

Problem: Given `n` pairs of parentheses, return every well-formed string of length `2n`: `n = 3` gives `["((()))","(()())","(())()","()(())","()()()"]`.

Intuition: A prefix can still grow into a well-formed string exactly when it uses at most n opens and never closes more than it has opened. Two guards keep the tree inside that region, so every leaf at length 2n is an answer and no branch is wasted.

Differs: two choices per level instead of a loop: `(` while `opened < n`, `)` while `closed < opened`. Pass both counts down; record when the length is `2 * n`.

```
if opened < n:
    path.append("(")
    backtrack(opened + 1, closed)
    path.pop()
if closed < opened:
    path.append(")")
```

LC 17 · Letter Combinations of a Phone Number

Problem: Given up to four digits from 2 to 9, return every string their phone-keypad letters can spell (2 is abc, 7 is pqrs, 9 is wxyz): `"23"` gives `["ad","ae","af","bd","be","bf","cd","ce","cf"]`.

Intuition: Each digit contributes exactly one letter in its own position, so the depth of the tree is the digit index and the choices at a level are that key's three or four letters. Every leaf is an answer; four digits give at most 256.

Differs: one level per digit: the call at `index` loops over the letters of `digits[index]` and explores `index + 1`; record at `index == len(digits)`. An empty `digits` returns `[]`, not `[""]`.

LC 131 · Palindrome Partitioning

Problem: Given a string `s`, return every way to cut it into pieces that are all palindromes: `"aab"` gives `[["a","a","b"],["aa","b"]]`.

Intuition: A partition is a sequence of cuts from left to right, and each piece must be a palindrome on its own. Testing a piece before choosing it prunes every bad partition at its first bad piece, so the tree holds only valid prefixes.

Differs: a choice is the next cut: from `start`, loop `end` over `start + 1 .. len(s)`, choose `s[start:end]` only if it is a palindrome, explore from `end`. Record when `start == len(s)`.

Path search · Marexun

MARk the cell, EXplore, UNmark on the way back: the path may not cross itself, but other paths may use the cell later.

Picture. A snake grows through the grid: its body is the marked cells, its head tries each neighbour, and when the head is stuck the snake pulls back one cell and wipes the mark.

Invariant. At the top of every call, exactly the cells (or columns, diagonals, digits) on the current path are marked, and nothing else is.

Use when. a path of adjacent cells, each cell used at most once; place pieces so that none attack; fill a board so every row, column and box is valid; all paths from a source to a target.

Book entries. #74 Word Search · #85 Word Search II · #113 N-Queens

GENERAL SOLUTION

Problem. Given a grid of letters and a `word`, return true if the word can be spelled along a path of cells joined up, down, left or right, each cell used at most once. In `ABCE / SFCS / ADEE`, "ABCCED" is spelled and "ABCB" is not.

Intuition. Try every cell as a start and grow a snake one matching letter at a time. A cell is blocked only while it is part of the snake's body, so overwrite it with `#` on entry and write the letter back on the way out: the next start, or the next branch, finds the board clean.

```
"ABCB": A(0,0) -> B(0,1) -> C(0,2), all three now '#'
  from C: down C(1,2) not B, up off the board, right E not B,
          left (0,1) is '#': on the path, refused
back out: C, B, A written back
start A(2,0): S above, D right, no B -> False
```

```python
def exist(board, word):
    rows, cols = len(board), len(board[0])

    def dfs(r, c, k):
        if k == len(word):
            return True                  # every letter matched
        if r < 0 or r >= rows or c < 0 or c >= cols:
            return False
        if board[r][c] != word[k]:       # wrong letter, or '#' on path
            return False
        board[r][c] = "#"                # Mark: on the path now
        found = False
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            if dfs(r + dr, c + dc, k + 1):    # Explore each neighbour
                found = True
                break
        board[r][c] = word[k]            # Unmark: free for other paths
        return found

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True
    return False
```

Example.

```python
grid = [list("ABCE"), list("SFCS"), list("ADEE")]
print(exist(grid, "ABCCED"))             # True
```

Dials. The mark (a `#` in the cell, a set of columns and diagonals, a set of digits per row, the path list itself), the moves (four neighbours, the next row, the next empty cell, the edges out of a node), the prune, and whether the first success stops the search.

Traps. Forgetting to unmark leaves `#` behind, so later starts fail on cells they should use. Returning before the unmark (inside the loop) leaves the board dirty. DFS that marks forever is reachability; backtracking unmarks, because it asks about paths, not places.

PROBLEMS

LC 79 · Word Search

Problem: Given a board of letters (at most 6 × 6) and a word of 1 to 15 letters, return whether the word runs along adjacent cells, none reused: on `ABCE / SFCS / ADEE`, “SEE” is true and “ABCB” false.

Intuition: A cell may not be reused within one path but may be used by another path. That is exactly a mark that lives as long as the call that set it: set on entry, cleared on exit.

Differs: nothing: this is the general solution.

LC 212 · Word Search II

Problem: Given a board of letters (at most 12 × 12) and up to 3×10^4 words, return every word that can be spelled as in LC 79: board `oaan / etae / ihkr / iflv` with `["oath","pea","eat","rain"]` gives `["eat","oath"]`.

Intuition: The words share prefixes, so put them in a trie and walk the board and the trie together: one walk tests every word at once, and a letter with no trie child ends the branch. Clearing a found word records it once; pruning emptied branches stops later starts walking them.

Differs: carry a trie node with the cell and step only onto a letter that is a child. At a node holding a word, record it and clear it; after the call, delete a child left with no children.

```python
node = parent.children[ch]
if node.word is not None:
    found.append(node.word)
    node.word = None                 # each word once
```

LC 51 · N-Queens

Problem: Place `n` queens on an `n × n` board (n from 1 to 9) so that none attacks another; return every board, drawn with `Q` and `.`: `n = 4` gives `[".Q..","...Q","Q...","..Q."]` and `["..Q.","Q...","...Q",".Q.."]`.

Intuition: Two queens never share a row, so the rows are the depth of the tree and each call places one queen. Every diagonal has a constant name, `row - col` or `row + col`, so three sets answer "is this square attacked?" in three lookups.

Differs: the call for `row` loops over columns and skips any whose column, `row - col` or `row + col` is in its set. Mark all three, explore `row + 1`, unmark. Record a drawn board at `row == n`.

```
if col in cols or row - col in diags or row + col in anti_diags:
    continue
```

LC 52 · N-Queens II

Problem: As LC 51, but return only the number of distinct placements: `n = 4` gives 2, and `n = 1` gives 1.

Intuition: Every leaf at depth n is a distinct placement and nothing else is, so counting leaves answers the question; no board needs drawing. With n at most 9, the pruned tree is small.

Differs: as LC 51, but at `row == n` add 1 to a count instead of drawing the board; return the count.

LC 37 · Sudoku Solver

Problem: Fill the empty cells (`'.'`) of a 9 × 9 Sudoku board in place so that every row, column and 3 × 3 box holds each digit 1 to 9 exactly once. The puzzle has exactly one solution: in the LeetCode example the first row `53..7....` becomes `534678912`.

Intuition: Fill empty cells one at a time, trying every digit still legal there, and back up when a cell has none. A digit is legal exactly when its row, column and box do not hold it yet. One solution exists, so the first full board is the answer.

Differs: the marks are three digit sets per row, column and box (`r // 3 * 3 + c // 3`). Each call fills the next empty cell with each allowed digit; return `True` on the first full board, left in place.

```
for d in "123456789":
    if d in rows[r] or d in cols[c] or d in boxes[b]:
        continue
```

LC 797 · All Paths From Source to Target

Problem: A directed acyclic graph of `n` nodes (2 to 15) is given as `graph[i]`, the nodes `i` points to. Return every path from 0 to `n - 1`: `[[1,2],[3],[3],[]]` gives `[[0,1,3],[0,2,3]]`.

Intuition: The graph has no cycle, so no path can come back to a node already on it. Marks exist only to stop cycles, so here the path list itself is the only state to grow and undo, and a node may appear on many different paths.

Differs: no visited set: start with `[0]`, append each successor, explore, pop; record a copy when the node is `n - 1`.

Combine · Balerico

BAse, LEft, RIght, COmbine: solve the parts first, do the work after the calls.

Picture. Answers flow upward like water: each leaf hands a small answer to its parent, and each parent fuses its children's answers into its own before handing it on.

Invariant. When a call returns, its return value is the complete answer for its own piece, computed from nothing but its children's return values and the piece itself.

Use when. sort, count pairs across halves, height, diameter, the best path anywhere in a tree, the lowest common ancestor, a value per subtree, range sums with updates, split a square into four.

Book entries. #17 Merge Sort · #33 Diameter and Max Path Sum · #34 Lowest Common Ancestor · #107 Segment Tree · #148 Quadtree

GENERAL SOLUTION

Problem. Sort a list of numbers with merge sort (#17), returning a new sorted list: `[5, 2, 4, 1]` gives `[1, 2, 4, 5]`.

Intuition. A list of one element is sorted already. Anything longer is cut in half, each half comes back sorted from its own call, and the combine step merges them by taking the smaller front each time. All the work happens after the calls, on the way up.

```
[5, 2, 4, 1]
cut    [5, 2] | [4, 1]
cut    [5] [2] | [4] [1]       bases: one element each
merge  [2, 5] | [1, 4]
merge  1 right, 2 left, 4 right, rest [5]  ->  [1, 2, 4, 5]
```

```python
def merge_sort(nums):
    if len(nums) <= 1:                    # Base: sorted already
        return list(nums)
    mid = len(nums) // 2
    left = merge_sort(nums[:mid])         # Left: its answer comes up
    right = merge_sort(nums[mid:])        # Right
    merged = []                           # Combine: the work after calls
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:           # a tie takes the left: stable
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    return merged + left[i:] + right[j:]  # one side ran out: the rest
```

Example.

```python
print(merge_sort([5, 2, 4, 1]))           # [1, 2, 4, 5]
```

Dials. What a call returns (a sorted list, a height, a gain, a found node, a sum, a pair), what the combine step computes from the parts, and whether a call also records a candidate in a global `best` that is not the value it returns. The quadtree (#148) splits a square into four the same way.

Traps. Returning what the parent cannot extend: the bent path `left + right` goes into `best`, but only one arm, `1 + max(left, right)`, goes up. Forgetting the leftovers after the merge loop loses the largest values.

PROBLEMS

LC 148 · Sort List

Problem: Given the head of a linked list of up to 5×10^4 nodes, return it sorted ascending: `4 -> 2 -> 1 -> 3` gives `1 -> 2 -> 3 -> 4`. The follow-up asks for O(n log n) time and O(1) memory.

Intuition: A list has no random access, but it splits at the middle in one pass and merges by changing pointers, so merge sort keeps O(n log n) with no array and no copied values. Only the O(log n) recursion stack is left; merging runs of length 1, 2, 4, ... bottom up removes it.

Differs: the piece is a linked list: find the middle with slow and fast pointers, cut `slow.next`, sort both halves, merge by relinking nodes behind a dummy head instead of building a list.

```python
slow, fast = head, head.next
while fast and fast.next:
    slow, fast = slow.next, fast.next.next
second = slow.next
slow.next = None                      # cut into two lists
```

LC 315 · Count of Smaller Numbers After Self

Problem: Given `nums` (up to 10^5 values), return `counts` where `counts[i]` is the number of smaller elements to the right of `nums[i]`: `[5,2,6,1]` gives `[2,1,1,0]`.

Intuition: During a merge, every right-half element sits after every left-half element in the array. A right element taken out before `left[i]` is smaller than it and stood after it, and each such pair meets in different halves at exactly one merge, so each is counted once.

Differs: sort indices, not values; when a left index is taken, add `j`, the number of right elements already taken, to its count. Ties take the left first, so equal values are not counted.

```python
if j == len(right) or (i < len(left)
                       and nums[left[i]] <= nums[right[j]]):
    counts[left[i]] += j              # right ones already out are smaller
```

LC 543 · Diameter of Binary Tree

Problem: Given the root of a binary tree, return the number of edges on the longest path between any two nodes; it need not pass through the root: `[1,2,3,4,5]` gives 3, the path 4-2-1-3.

Intuition: The longest path bends at exactly one node, its highest, and there its length is the two arm heights added. A parent can continue only one arm, so each call records the bent path as a candidate and passes only its longer arm up.

Differs: return the height (nodes on the longest downward path, 0 for `None`); combine records `left + right` in `best` and returns `1 + max(left, right)`. The answer counts edges.

LC 124 · Binary Tree Maximum Path Sum

Problem: Node values may be negative. Return the largest sum of a non-empty path between any two nodes, each node used at most once: `[1,2,3]` gives 6, and `[-10,9,20,null,null,15,7]` gives 42 (15 + 20 + 7).

Intuition: As in LC 543 the best path bends at one node, so each call records "node plus both arms" and returns "node plus one arm". A negative arm only lowers a sum, so it is cut to zero; a path holds a node, so `best` starts at a real value.

Differs: return the gain `node.val + max(left, right)`, each child gain cut to `max(gain, 0)`; record `node.val + left + right` in `best`, seeded with `root.val`.

```
left = max(gain(node.left), 0)        # a negative arm is cut off
right = max(gain(node.right), 0)
best[0] = max(best[0], node.val + left + right)
return node.val + max(left, right)
```

LC 236 · Lowest Common Ancestor of a Binary Tree

Problem: Given a binary tree and two of its nodes `p` and `q`, return their lowest common ancestor, a node being its own ancestor: in `[3,5,1,6,2,0,8,null,null,7,4]`, p = 5, q = 1 give 3; p = 5, q = 4 give 5.

Intuition: Each call reports up whether it saw `p`, `q` or both. The first node that hears from both sides is the deepest one holding both. Both nodes are promised to exist, so a target that sits above the other returns itself and is the answer.

Differs: return what was found: `None`, `p`, `q` or the answer. Base: `None`, `p` or `q` itself. Combine: both sides non-`None` returns the node; otherwise pass up the one that is.

LC 307 · Range Sum Query - Mutable

Problem: Build a class over `nums` with `update(index, val)` and `sumRange(left, right)`, the inclusive sum: on `[1,3,5]`, `sumRange(0,2)` is 9; after `update(1,2)` it is 8.

Intuition: A range sum is the sum of the sums of its halves, so a segment tree (#107) stores the sum of every segment. A query is covered by O(log n) stored pieces, and an update changes only the sums on one leaf-to-root path; up to 3×10^4 calls interleave.

Differs: each node stores its segment's sum, combine is `tree[2 * i] + tree[2 * i + 1]`; `update` walks to one leaf and redoes the sums on the way up; `sumRange` returns 0 for no overlap and the stored sum for full overlap.

LC 337 · House Robber III

Problem: Houses form a binary tree; robbing two houses joined by an edge sets off the alarm. Return the most money that can be robbed: `[3,2,3,null,3,null,1]` gives 7 (3 + 3 + 1).

Intuition: Robbing a node forbids robbing its children, so a parent needs two answers from each child: the best with the child robbed and the best without. A pair carries both up in one pass, with no memo, because each subtree is solved once.

Differs: return a pair `(rob, skip)` for each subtree: `rob = val + left_skip + right_skip`, `skip = max(left) + max(right)`; the answer is `max` of the root's pair.

```
def go(node):
    if node is None:
        return (0, 0)
    left, right = go(node.left), go(node.right)
    return (node.val + left[1] + right[1], max(left) + max(right))
```

Pass down · Bachepa

BAse, CHEck, PAss the constraints down: the work happens before the calls.

Picture. A funnel narrowing as it descends: every ancestor tightens the range a node must fit, and each call checks itself against what came down before it hands a tighter promise to its children.

Invariant. On entry, the arguments hold every constraint the ancestors impose on this subtree (a value range, an index range, a key to follow), and nothing below needs to look back up.

Use when. validate a BST, a property that depends on all ancestors and not only the parent, build a tree from traversals, read a serialized tree back, walk a BST by key, in-order position (k-th smallest, the swapped pair).

Book entries. #35 Validate a Binary Search Tree · #36 Serialize and Deserialize a Binary Tree · #75 Construct Binary Tree from Preorder and Inorder · #76 BST Search, Insert, Delete

GENERAL SOLUTION

Problem. Given the root of a binary tree, return true if it is a valid binary search tree: every value in a node's left subtree is smaller than the node and every value in its right subtree is larger. `[2,1,3]` is valid; `[5,4,6,null,null,3,7]` is not, because 3 sits right of 5.

Intuition. A node is bounded by all its ancestors, not only its parent: going left caps it from above, going right raises its floor. Each call receives the open range its ancestors allow, checks itself, and hands each child a tighter range. Nothing below ever needs to look back up.

```
fits(5, -inf, inf)    ok
  fits(4, -inf, 5)    ok
  fits(6, 5, inf)     ok
    fits(3, 5, 6)     3 is not above 5: False
```

```python
import math


def is_valid_bst(root):
    def fits(node, low, high):
        if node is None:                  # Base: an empty tree is valid
            return True
        if not low < node.val < high:     # Check against all ancestors
            return False
        return (fits(node.left, low, node.val)         # Pass down:
                and fits(node.right, node.val, high))  # tighter ranges

    return fits(root, -math.inf, math.inf)
```

Example.

```python
print(is_valid_bst(build_tree([2, 1, 3])))                # True
```

Dials. What goes down (a value range, an inorder index range, a key, a shared cursor or counter), what the check rejects or does, and whether the walk follows one path (a BST by key) or visits every node.

Traps. Comparing a node only with its parent passes `[5, 4, 6, null, null, 3, 7]`, where 3 sits in the right subtree of 5. A cursor passed as a plain integer is copied into each call, so the caller never learns how far its left subtree read; share it in a box or an iterator.

PROBLEMS

LC 98 · Validate Binary Search Tree

Problem: Given the root of a binary tree of 1 to 10^4 nodes, return whether it is a valid BST, with strict order on both sides: `[2,1,3]` gives true, and `[5,1,4,null,null,3,6]` gives false.

Intuition: A BST promises that every node of the left subtree is smaller and every node of the right one larger, which is a range set by all ancestors at once. Passing that range down lets each node check itself in O(1).

Differs: nothing: this is the general solution. Values span the whole 32-bit range, so the seeds are `-inf` and `inf`, and an equal value is invalid.

LC 230 · Kth Smallest Element in a BST

Problem: Given the root of a BST and `k` (1 ≤ k ≤ n), return the k-th smallest value, counting from 1: `[5,3,6,2,4,null,null,1]` with k = 3 gives 3.

Intuition: An in-order walk (left, node, right) visits a BST in increasing order, so the k-th node visited is the k-th smallest. The count is the shared state passed along the walk, and stopping at k costs O(h + k) instead of O(n).

Differs: walk in order and count `k` down at every visit; the node at which it reaches 0 is the answer. An explicit stack makes stopping there a plain `return`.

```
while True:
    while node:
        stack.append(node)
        node = node.left
    node = stack.pop()
    k -= 1                              # then return at 0, else go right
```

LC 99 · Recover Binary Search Tree

Problem: Exactly two nodes of a BST had their values swapped by mistake. Fix the tree without changing its structure: `[3,1,4,null,null,2]` becomes `[2,1,4,null,null,3]`.

Intuition: In order, a BST reads as a rising sequence. Swapping two values makes it drop once, when they are neighbours in that order, or twice. The larger misplaced value starts the first drop and the smaller one ends the last, so carrying the previous node finds both.

Differs: walk in order carrying `prev`; at each drop `prev.val > node.val`, the first time set `first = prev`, every time set `second = node`. Swap the two values at the end.

```
if prev and prev.val > node.val:
    if first is None:
        first = prev
    second = node
```

LC 297 · Serialize and Deserialize Binary Tree

Problem: Design `serialize(root)`, which turns a binary tree into a string, and `deserialize(data)`, which rebuilds the same tree from it; the format is yours: `[1,2,3,null,null,4,5]` must come back unchanged.

Intuition: With a marker for every missing child, a preorder fixes the tree on its own: each subtree ends exactly where its markers close it. On reading back, one shared cursor over the tokens tells the right subtree where the left one stopped.

Differs: write preorder with `#` for every missing child, joined by commas. Read back with one shared iterator: take a token; `#` returns `None`, otherwise build the node, then its left, then its right.

```
token = next(tokens)                    # inside read(tokens), an iterator
if token == "#":
    return None
node = TreeNode(int(token))
node.left, node.right = read(tokens), read(tokens)   # left reads first
return node
```

LC 105 · Construct Binary Tree from Preorder and Inorder Traversal

Problem: Given the preorder and inorder traversals of a binary tree with unique values, build the tree: preorder `[3,9,20,15,7]` and inorder `[9,3,15,20,7]` give `[3,9,20,null,null,15,7]`.

Intuition: Preorder names each root before its subtrees, and in the inorder list that root splits its range into exactly its left and right subtrees. Each call owns an inorder range; the left subtree comes first in preorder, so building left before right keeps one shared cursor in step.

Differs: pass the inorder range `lo..hi` down and share a preorder cursor. Each call takes the next preorder value as its root, finds it in a value-to-index map, builds `lo..mid-1`, then `mid+1..hi`.

LC 450 · Delete Node in a BST

Problem: Given the root of a BST and a `key`, delete the node with that key, if any, and return the root of the resulting BST: `[5,3,6,2,4,null,7]` with key 3 can give `[5,4,6,2,null,null,7]`.

Intuition: The key picks the side at every node, so only one path is walked, O(h). Each call returns the new root of its subtree, which lets the parent rewire its link without parent pointers. A node with two children takes its successor's value, and the successor is deleted below.

Differs: walk down by key and reattach on the way back: `root.left = delete(root.left, key)`. At the key: no left child returns the right, no right returns the left; with two, copy the successor's value and delete it on the right.

```
if root.left is None:
    return root.right
if root.right is None:
    return root.left
```

LC 235 · Lowest Common Ancestor of a Binary Search Tree

Problem: Given a BST and two of its nodes `p` and `q`, return their lowest common ancestor, a node being its own ancestor: in `[6,2,8,0,4,7,9,null,null,3,5]`, p = 2, q = 8 give 6; p = 2, q = 4 give 2.

Intuition: The BST order says which side holds each node, so the root-to-node paths of `p` and `q` agree until the node where they part, or until one of them is reached. That node is their lowest common ancestor, found in one walk down, O(h).

Differs: no recursion over both sides: walk from the root, left while `p` and `q` are both smaller, right while both are larger; the first node that separates them, or equals one, is the answer.

```
node = root
while not min(p.val, q.val) <= node.val <= max(p.val, q.val):
    if p.val < node.val:
        node = node.left              # both smaller: go left
    else:
        node = node.right             # both larger: go right
```

Stamp · Grabla

GRAy on entry, BLAck on exit: every node gets two stamps, and the time between them is its open call.

Picture. A DFS walks a directed graph with a stamp in each hand: a node turns gray when the walk enters it and black when the walk has left everything below it; the gray nodes are exactly the current path.

Invariant. At every moment the gray nodes form the path from the root of this DFS to the current node, and a black node is finished: everything reachable from it is black too, and its result is final.

Use when. prerequisites, dependencies, "is it possible to finish all", an order consistent with rules, a cycle in a directed graph, nodes that never reach a cycle, the longest path in a DAG, which edge disconnects the network.

Book entries. #81 Cycle Detection in a Directed Graph · #82 Topological Sort with DFS · #116 Critical Connections

GENERAL SOLUTION

Problem. Given `n` nodes and directed edges `[u, v]`, return an order of all nodes that puts `u` before `v` for every edge, or `[]` if the edges form a cycle (#82): `n = 3` with `[[0,1],[1,2],[0,2]]` gives `[0, 1, 2]`.

Intuition. A node turns gray on entry and black when everything below it is finished, so the gray nodes are exactly the current path. An edge into a gray node goes back up the path and closes a cycle. A node turns black only after all its successors did, so the reversed finish order puts every node before its successors.

```
enter 0: gray
  enter 1: gray
    enter 2: gray, no edges out; black, finished [2]
  1 black, finished [2, 1]
  edge 0 -> 2: 2 is black, finished already, skip
0 black, finished [2, 1, 0]
reversed: [0, 1, 2]
```

```python
def topo_order(n, edges):
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
    color = [0] * n                        # 0 white, 1 gray, 2 black
    finished = []

    def dfs(u):
        color[u] = 1                       # Gray: u is on the path
        for v in graph[u]:
            if color[v] == 1:
                return False               # back edge: a cycle
            if color[v] == 0 and not dfs(v):
                return False
        color[u] = 2                       # Black: all below u is done
        finished.append(u)                 # dial: what is recorded on exit
        return True

    for u in range(n):
        if color[u] == 0 and not dfs(u):
            return []
    return finished[::-1]
```

Example.

```python
print(topo_order(3, [[0, 1], [1, 2], [0, 2]]))              # [0, 1, 2]
```

Dials. What gray means (a cycle when hit), what black means (safe, finished, memoized), what is recorded on exit (the finish order, a cached answer, a low-link), and whether the graph is directed or undirected (then the stamp is a discovery time and the parent edge is skipped).

Traps. A plain visited set calls the diamond a cycle: a second route into a finished node is fine, only a route into an open one is not. Recording at entry instead of exit breaks the order. Recursion deeper than about a thousand frames needs `sys.setrecursionlimit` or an explicit stack.

PROBLEMS

LC 207 · Course Schedule

Problem: There are `numCourses` courses; a pair `[a, b]` means b must be taken before a. Return whether all courses can be finished: 2 courses with `[[1,0]]` give true, and with `[[1,0],[0,1]]` give false.

Intuition: All courses can be taken exactly when the prerequisite graph has no cycle. In a DFS a cycle shows itself as an edge back into a node still on the path, that is, a gray node; a black node reached again is only a second route, never a cycle.

Differs: the edge for `[a, b]` is `b -> a`; return `False` at the first gray hit and `True` if every DFS finishes. Nothing is recorded on exit.

LC 802 · Find Eventual Safe States

Problem: A directed graph is given as `graph[i]`, the nodes `i` points to. A node is safe if every path from it ends at a node with no outgoing edge. Return the safe nodes in ascending order: `[[1,2],[2,3],[5],[0],[5],[],[]]` gives `[2,4,5,6]`.

Intuition: One path into a cycle makes a node unsafe, so a node is safe exactly when its DFS finishes black, with no gray hit below it. A node left gray by a failed search leads into a cycle for good, so every later visit to it fails at once.

Differs: a node is safe exactly when it ends black. On a gray hit return `False` and leave the node gray; the colour doubles as the memo. List the black nodes in index order.

```python
if color[u]:                           # inside dfs(u)
    return color[u] == 2               # black: safe; gray: unsafe
color[u] = 1
if all(dfs(v) for v in graph[u]):
    color[u] = 2                       # else u stays gray: unsafe for good
return color[u] == 2
```

LC 269 · Alien Dictionary

Problem: Given words sorted in an unknown alien alphabet, return a string of their letters in an order consistent with the sorting, any one, or `""` if none exists: `["wrt","wrf","er","ett","rftt"]` gives `"wertf"`.

Intuition: The list is sorted, so only the first difference between two neighbouring words says anything: that letter comes before the other. Those facts are edges, and a valid alphabet is a topological order of the letters. A word followed by its own proper prefix breaks any order.

Differs: build the graph first: every letter is a node, each adjacent pair of words gives one edge at its first differing letter. Return the reversed finish order as a string, or `""` on a cycle or a bad prefix.

```
for a, b in zip(words, words[1:]):
    diff = next(((x, y) for x, y in zip(a, b) if x != y), None)
    if diff:
        graph[diff[0]].add(diff[1])
    elif len(a) > len(b):
        return ""                       # "abc" before "ab": no order works
```

LC 1203 · Sort Items by Groups Respecting Dependencies

Problem: Each of `n` items is in one of `m` groups or none (`group[i] == -1`); `beforeItems[i]` lists items that must precede item i. Return any order keeping each group's items together and every rule, or `[]`: `group = [0,-1,0]` and `beforeItems = [[],[0],[]]` allow `[0,2,1]`.

Intuition: Items of one group must be contiguous, so the groups need an order of their own, and inside a group the items keep the item order: a two-level topological order. An ungrouped item is a group of one, and a rule between items of different groups orders their groups.

Differs: give each ungrouped item a new group; build an item graph and a group graph (a cross-group edge also links the groups). Topologically sort both; list the items group by group in group order. Either cycle returns `[]`.

LC 329 · Longest Increasing Path in a Matrix

Problem: Given an `m × n` integer matrix (up to 200 × 200), return the length of the longest strictly increasing path moving up, down, left or right: `[[9,9,4],[6,6,8],[2,1,1]]` gives 4, the path 1, 2, 6, 9.

Intuition: A strictly increasing step can never return to a cell, so the moves form a DAG and no gray check is needed. The answer of a cell is final once its call finishes: it is black, and every later path into that cell reuses it instead of walking on.

Differs: no gray: from each cell step to the neighbours with a larger value; the answer of a cell is `1 + max` over those steps, cached when its call finishes (black is the memo). Return the largest.

```
best = 1
for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
    if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] > matrix[r][c]:
        best = max(best, 1 + dfs(nr, nc))
memo[(r, c)] = best                     # black: final
```

LC 1192 · Critical Connections in a Network

Problem: `n` servers (up to 10^5) are joined by undirected `connections`. Return every connection whose removal disconnects some servers, in any order: `n = 4` with `[[0,1],[1,2],[2,0],[1,3]]` gives `[[1,3]]`.

Intuition: Stamp every server with its discovery time. An edge `u-v` of the DFS tree is critical exactly when nothing below `v` reaches `u` or above by another edge. The lowest stamp a subtree can reach comes up as `low`; a deep path needs a raised recursion limit or a stack.

Differs: the entry stamp is `disc[u]`; skip the parent edge; after a child `v` returns, `low[u] = min(low[u], low[v])`, and `u-v` is a bridge when `low[v] > disc[u]`. A back edge gives `low[u] = min(low[u], disc[v])`.

```
if disc[v] == -1:                       # tree edge
    dfs(v, u)
    low[u] = min(low[u], low[v])
    if low[v] > disc[u]:
        bridges.append([u, v])
```

Fill

Answers to smaller inputs, in dependency order.

The question. What did the smaller inputs already answer?

Picture. A pyramid built from the bottom up: the base row is written from the definition, and every brick above rests on bricks already laid. The last brick is the answer.

Shared skeleton. Tasefire: TAble, SEed, FIll, REad.

```
dp = [0] * (n + 1)                      # TAble: the shape is a dial
dp[0] = base                            # SEed: the answers known outright
for s in range(1, n + 1):               # FIll: the dependency order is a dial
    dp[s] = best_or_sum(dp[t] for t in smaller(s))
answer = dp[n]                          # REad: the goal cell
```

Dials. The table's shape (one index, a few states per step, items times capacity, two prefixes, an interval, the best earlier element) and its fill order, which must visit every cell after the cells it reads. Then the combine (`max`, `min`, `+`, `or`) and the seed, which is the answer for the empty or one-element input.

Method	Mnemonic	What it sets	Book entries
Chain	Tasefire	`dp[i]` from a few earlier cells, often two variables	#28, #90
State vector	Costep	several states per step, stepped from a copy	#29, #117
Knapsack	Tasefire	items times capacity; loop direction picks 0/1 or unbounded	classic
2-D table	Tasefire	`dp[i][j]` from up, left and diagonal	classic
Interval	Tasefire	`dp[l][r]` by increasing length	#87
Subsequence	Tasefire	`dp[i]` from the best earlier compatible `j`; sorted tails	#58

Fill is Branch's *combine* with a memo, turned upside down: dynamic programming is recursion without repetition, and the bottom-up loop visits the subproblems in the order the recursion would finish them. Use it when the subproblems overlap; when they do not (merge sort, a tree's height), plain recursion already solves each subproblem once. The family-wide trap is a fill order that reads a cell before it is final: in place where a copy was needed, a capacity loop running the wrong way, an interval filled by left end instead of by length. Say the recurrence and which cells it reads before choosing the loops.

Chain · Tasefire

TAble, SEed, FIll, REad: dp[i] from a few earlier cells.

Picture. A row of cells filled left to right; each new cell looks back a few cells, usually one or two, and writes the best (or the sum) of what it sees.

Invariant. At the top of step `i`, `dp[0..i-1]` hold the final answers for the first `0..i-1` elements, and `dp[i]` reads only those.

Use when. number of ways to reach step n, no two adjacent, decode a string of digits, can the string be segmented, contiguous subarray with the largest sum or product; the answer for a prefix follows from the answers for slightly shorter prefixes.

Book entries. #28 Kadane's Algorithm · #90 Fibonacci

GENERAL SOLUTION

Problem. Given a row of non-negative values, return the largest sum of values picked with no two adjacent ones: `[2, 7, 9, 3, 1]` gives 12, from 2 + 9 + 1.

Intuition. Look at the last value. Either it is skipped, and the answer is the best over one fewer value, or it is taken, and its neighbour is out, so the answer is the best over two fewer plus it. Those two smaller answers are already in the row, so each cell costs two reads.

```
i  value  skip = dp[i-1]  take = dp[i-2] + value  dp[i]
1  2      (seed)                                  2
2  7      2               0 + 7 = 7               7
3  9      7               2 + 9 = 11              11
4  3      11              7 + 3 = 10              11
5  1      11              11 + 1 = 12             12
```

```python
def chain(values):
    n = len(values)
    dp = [0] * (n + 1)              # TAble: dp[i] = best over the first i
    if n:
        dp[1] = values[0]           # SEed: dp[0] = 0 picks nothing
    for i in range(2, n + 1):       # FIll: left to right
        skip = dp[i - 1]
        take = dp[i - 2] + values[i - 1]
        dp[i] = max(skip, take)
    return dp[n]                    # REad: the whole row
```

Example.

```python
print(chain([2, 7, 9, 3, 1]))             # 12
```

Dials. How far back a cell reads (one, two, every earlier cut), the combine (`max`, `+`, `or`), the guards on each option, and where to read the answer (the last cell, or the best cell anywhere). When a cell reads only the last two, the table collapses to two variables.

Traps. Updating `prev2` and `prev1` one after the other, so the second assignment reads a value already overwritten: assign both in one tuple assignment. Seeding `best = 0` in Kadane, which answers `0` for an all-negative array. Forgetting that `'0'` alone decodes to nothing.

PROBLEMS

LC 70 · Climbing Stairs

Problem: You climb a staircase of `n` steps (`1 <= n <= 45`), one or two steps at a time. Return the number of distinct ways to reach the top: `n = 3` gives 3 (1+1+1, 1+2, 2+1).

Intuition: The last move is a 1-step or a 2-step: two disjoint cases, so the ways to reach `n` are the ways to reach `n - 1` plus the ways to reach `n - 2`. That is Fibonacci shifted by one, `ways(n) = F(n + 1)`.

Differs: the combine is `+` and both options are always open: `ways[i] = ways[i-1] + ways[i-2]`, seeded `ways[0] = ways[1] = 1`, collapsed to two variables.

```python
prev, curr = 1, 1
for _ in range(n - 1):
    prev, curr = curr, prev + curr
```

LC 198 · House Robber

Problem: `nums[i]` is the money in house `i` along a street; robbing two adjacent houses alerts the police. Return the most money you can rob: `[2,7,9,3,1]` gives 12 (houses 0, 2 and 4).

Intuition: This is the anchor problem. The last house is skipped or taken, and taking it rules out its neighbour, so the best for a prefix reads only the best for the two shorter prefixes; two variables carry the whole row.

Differs: the general solution with the table collapsed to two variables, `prev2` and `prev1`, starting at `0, 0`; each house does `prev2, prev1 = prev1, max(prev1, prev2 + x)`.

```python
prev2, prev1 = 0, 0
for x in nums:
    prev2, prev1 = prev1, max(prev1, prev2 + x)
```

LC 213 · House Robber II

Problem: LC 198, but the houses stand in a circle, so the first and the last are neighbours. Return the most money you can rob: `[2,3,2]` gives 3, since houses 0 and 2 touch.

Intuition: No valid plan robs both the first and the last house, so every plan fits inside one of two straight streets: without the first house, or without the last. The better of the two straight answers is the circular answer.

Differs: run LC 198 twice, on `nums[1:]` and on `nums[:-1]`, and return the larger; a single house returns `nums[0]`.

LC 91 · Decode Ways

Problem: Letters map to digits, A to "1" up to Z to "26". Given a string of digits, return the number of ways to decode it: "226" gives 3 (2 2 6, 22 6, 2 26); "06" gives 0.

Intuition: The last letter uses either the last digit or the last two, two disjoint cases whose counts add. An option is open only if its digits form a code: `'0'` alone has no letter, and a pair must lie between "10" and "26", so "06" is not a code.

Differs: the combine is `+` with a guard on each option: add `dp[i-1]` if `s[i-1] != '0'`, add `dp[i-2]` if `s[i-2:i]` is between `"10"` and `"26"`. Seed `dp[0] = 1`.

```
one = curr if s[i - 1] != "0" else 0
two = prev if "10" <= s[i - 2:i] <= "26" else 0
prev, curr = curr, one + two
```

LC 139 · Word Break

Problem: Given a string `s` and a list `wordDict`, return true if `s` splits into a sequence of dictionary words, reuse allowed: "leetcode" with `["leet","code"]` gives true.

Intuition: A prefix splits if some shorter prefix splits and the piece after it is a word. The last word can have any length, so every earlier cut is a candidate, not only one or two; words have at most 20 letters, so only the last 20 cuts can work.

Differs: a cell reads every earlier cut: `dp[i]` is true if some `j < i` has `dp[j]` and `s[j:i]` in the word set. The combine is `or`; seed `dp[0] = True`.

```
for i in range(1, len(s) + 1):
    dp[i] = any(dp[j] and s[j:i] in words
                for j in range(max(0, i - 20), i))
```

LC 53 · Maximum Subarray

Problem: Given an integer array `nums`, return the largest sum of a non-empty contiguous subarray: `[-2,1,-3,4,-1,2,1,-5,4]` gives 6, from `[4,-1,2,1]`.

Intuition: Let the cell be the best sum of a subarray ending exactly here. Contiguity leaves two choices: extend the best one ending one place earlier, or start fresh at this element, whichever is larger. The answer may end anywhere, so keep the best cell seen.

Differs: Kadane: the cell is the best sum ending exactly at `i`, `here = max(x, here + x)`, and REad takes the best cell anywhere. Seed both at `nums[0]`.

```
here = best = nums[0]
for x in nums[1:]:
    here = max(x, here + x)
    best = max(best, here)
```

LC 152 · Maximum Product Subarray

Problem: Given an integer array `nums`, return the largest product of a non-empty contiguous subarray: `[2,3,-2,4]` gives 6, from `[2,3]`; `[-2,0,-1]` gives 0.

Intuition: A negative factor flips order: the most negative product ending before it becomes the largest after it. So carry both the largest and the smallest product ending here, each built from the old pair or restarted at `x`; a zero collapses both, and the restart recovers.

Differs: two cells per index, the largest and smallest product ending here, both from the old pair: `hi, lo = max(x, hi*x, lo*x), min(x, hi*x, lo*x)`. REad the best `hi` anywhere.

```
hi = lo = best = nums[0]
for x in nums[1:]:
    hi, lo = max(x, hi * x, lo * x), min(x, hi * x, lo * x)
    best = max(best, hi)
```

State vector · Costep

COpy the previous states, STEP every state at once.

Picture. A short column of numbers, one per state (holding, sold today, resting), marches across the days; every number of the new column is computed from the whole old column.

Invariant. At the top of day `i`, each state holds the best result of days `0..i-1` that ends in that state, and nothing in the vector has seen day `i` yet.

Use when. buy and sell with a fee, a cooldown, at most k transactions; at most k stops or k edges; at each step the item can be in one of a few modes, and the mode decides what is allowed next.

Book entries. #29 Best Time to Buy and Sell Stock · #117 Bellman-Ford

GENERAL SOLUTION

Problem. Given daily prices, a fee paid on every sale and a cooldown flag, return the most profit from unlimited trades of one share at a time; with the cooldown on, the day after a sale allows no buy. `[1, 2, 3, 0, 2]` with cooldown and no fee gives 3: buy, sell, rest, buy, sell.

Intuition. Each evening you are in one of three states: `hold` a share, `sold` today, or `rest` without a share and free to buy. Tomorrow's best value for each state depends only on today's three values, so carry the column and step all of it at once from a copy. The cooldown is a missing arrow: a buy may read `rest` but not `sold`.

```
price  hold  sold  rest
seed   -inf  -inf  0
1      -1    -inf  0
2      -1    1     0
3      -1    2     1
0      1     -1    2
2      1     3     2      answer max(sold, rest) = 3
```

```
import math


def costep(prices, fee, cooldown):
    hold, sold, rest = -math.inf, -math.inf, 0   # seed: no share, cash 0
    for p in prices:
        old_hold, old_sold, old_rest = hold, sold, rest  # COpy the vector
        free = old_rest if cooldown else max(old_rest, old_sold)
        hold = max(old_hold, free - p)           # STEP: keep, or buy today
        sold = old_hold + p - fee                # sell today
        rest = max(old_rest, old_sold)           # no share, not sold today
    return max(sold, rest)
```

Example.

```
print(costep([1, 2, 3, 0, 2], 0, True))   # 3
```

Dials. The states (holding or not, trades used, rounds of edges), which old states each new state may read (the cooldown reads `rest` only), the cost on a move (the fee, an edge weight), and the number of steps (days, or `k + 1` rounds).

Traps. Stepping in place, so a state reads a value already updated today: a sale and a buy on one day, or several flights in one Bellman-Ford round. Seeding `hold = 0`, which sells a share that was never bought; use `-inf`.

PROBLEMS

LC 121 · Best Time to Buy and Sell Stock

Problem: Given `prices[i]` for each day, buy one share on one day and sell it on a later day. Return the largest profit, or 0 if none is possible: `[7,1,5,3,6,4]` gives 5 (buy at 1, sell at 6).

Intuition: Only one transaction, so a buy always spends the starting cash, never an earlier profit. `hold` is the best cash after buying, which is minus the cheapest price so far (#29), and today's best sale pairs with it.

Differs: two states and one trade: `hold = max(hold, -p)` buys from the starting cash `0`, never from a profit, and `done = max(done, hold + p)`.

```
hold, done = -math.inf, 0
for p in prices:
    hold, done = max(hold, -p), max(done, hold + p)
```

LC 122 · Best Time to Buy and Sell Stock II

Problem: Given daily prices, hold at most one share at a time, but buy and sell as often as you like, even on the same day. Return the largest total profit: `[7,1,5,3,6,4]` gives 7 (4 + 3).

Intuition: With unlimited trades and no waiting, a sale can be followed by a buy at once, so every positive day-to-day rise can be collected; the answer is the sum of those rises. The vector finds the same number with two states, `hold` and `cash`.

Differs: `fee = 0` and cooldown off, so two states suffice: `hold` and `cash`, with `cash = max(cash, hold + p)` and `hold = max(hold, cash - p)` from the old pair.

LC 714 · Best Time to Buy and Sell Stock with Transaction Fee

Problem: LC 122, but every transaction pays `fee` once. Return the largest profit: `prices = [1,3,2,8,4,9]`, `fee = 2` gives 8 (buy 1, sell 8, buy 4, sell 9).

Intuition: The fee belongs on exactly one move of each trade; put it on the sale. It makes a small rise not worth a trade, and the `max` with "keep what you have" decides that by itself, with no special case.

Differs: cooldown off; the sale pays the fee: `cash = max(cash, hold + p - fee)`.

LC 309 · Best Time to Buy and Sell Stock with Cooldown

Problem: LC 122, but after selling you cannot buy on the next day. Return the largest profit: `[1,2,3,0,2]` gives 3 (buy, sell, cooldown, buy, sell).

Intuition: After a sale the next day is a forced rest. The third state `sold` carries "sold today" for one day, and the buy reads only `rest`, so a sale on day `i` cannot fund a buy before day `i + 2`.

Differs: `fee = 0`, cooldown on: the general solution as written; the buy reads `rest` only, never `sold`.

LC 123 · Best Time to Buy and Sell Stock III

Problem: Given daily prices, complete at most two transactions, never holding two shares. Return the largest profit: `[3,3,5,0,0,3,1,4]` gives 6 (buy 0, sell 3, buy 1, sell 4).

Intuition: The state must count trades used: four states, one after each move of the two trades. The second buy spends the first trade's profit, so each state reads the one before it in the chain buy1, sell1, buy2, sell2.

Differs: four states stepped from the old vector: `buy1 = max(buy1, -p)`, `sell1 = max(sell1, buy1 + p)`, `buy2 = max(buy2, sell1 - p)`, `sell2 = max(sell2, buy2 + p)`. Return `sell2`.

```
buy1, sell1, buy2, sell2 = (max(buy1, -p), max(sell1, buy1 + p),
                            max(buy2, sell1 - p), max(sell2, buy2 + p))
```

LC 188 · Best Time to Buy and Sell Stock IV

Problem: LC 123 with at most `k` transactions (`1 <= k <= 100`). Return the largest profit: `k = 2`, `prices = [3,2,6,5,0,3]` gives 7 (buy 2, sell 6, buy 0, sell 3).

Intuition: LC 123 generalised: the state is the number of trades started. Trade `j` buys with the cash left after trade `j - 1`, so the vector is two arrays of length `k + 1`, stepped from a copy of the old sales.

Differs: `buy[j]` and `sell[j]` for `j = 1..k`, `sell[0] = 0`; each day copies `sell`, then `buy[j] = max(buy[j], old_sell[j-1] - p)` and `sell[j] = max(sell[j], buy[j] + p)`.

```
old_sell = sell[:]
for j in range(1, k + 1):
    buy[j] = max(buy[j], old_sell[j - 1] - p)
    sell[j] = max(sell[j], buy[j] + p)
```

LC 787 · Cheapest Flights Within K Stops

Problem: `n` cities, `flights[i] = [from, to, price]`. Return the cheapest price from `src` to `dst` with at most `k` stops, or -1: flights 0→1 (100), 1→2 (100), 0→2 (500), from 0 to 2 with `k = 1` give 200 (0→1→2).

Intuition: At most `k` stops means at most `k + 1` flights. One Bellman-Ford round extends every known route by one flight, so after round `r` the vector of cities holds the cheapest routes of at most `r` flights, provided each round reads only the previous round's copy.

Differs: the vector is `dist` over cities; run exactly `k + 1` rounds, each relaxing every flight from a copy of the previous round. Return `dist[dst]`, or `-1` if still `inf`.

```
for _ in range(k + 1):
    new = dist[:]
    for u, v, w in flights:
        if dist[u] + w < new[v]:
            new[v] = dist[u] + w
    dist = new
```

LC 1334 · Find the City With the Smallest Number of Neighbors at a Threshold Distance

Problem: `n` cities (`n <= 100`), weighted two-way `edges`. Return the city with the fewest other cities reachable within `distanceThreshold`, ties to the largest number: `n = 4`, edges 0-1 (3), 1-2 (1), 1-3 (4), 2-3 (1), threshold 4 give 3.

Intuition: Floyd-Warshall steps a whole matrix once per middle city `m`: after step `m`, `dist[i][j]` is the shortest path whose middles are all `<= m`. In place is safe because row and column `m` do not change during step `m`; n^3 is a million steps.

Differs: the vector is the matrix `dist[i][j]`, stepped once per `m` with `dist[i][j] = min(dist[i][j], dist[i][m] + dist[m][j])`. Then count cities within the threshold per city; ties go to the greatest index.

```
for m in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][m] + dist[m][j])
```

Knapsack · Tasefire

Items times capacity; the loop order is the dial.

Picture. One row of capacities `0..C` is swept once per item; sweeping right to left uses the item at most once, left to right lets it pile up again and again.

Invariant. After the first `k` items, `dp[c]` is the best (or the count) over those items with total size `c` or at most `c`; cells not yet swept for item `k + 1` still hold the old row.

Use when. a subset that sums to a target, split into two equal halves, assign + or -, fewest coins, number of ways to make an amount, items with a budget in one or two resources.

Book entries. none: knapsack is a classic problem outside the book.

GENERAL SOLUTION

Problem. Given items as `(weight, value)` pairs and a weight budget, return the most value that fits; a flag says whether each item is used at most once (0/1) or as often as you like (unbounded). Items `(1, 15), (3, 20), (4, 30)` with budget 4 give 35 as 0/1 and 60 unbounded.

Intuition. Sweep one row of budgets `0..C` once per item: `dp[c]` either ignores the item or takes it on top of `dp[c - weight]`. The direction decides what `dp[c - weight]` already contains. Going down, it is still the row without this item, so the item is used at most once; going up, it may already hold the item, so the item piles up.

```
budget            0   1   2   3   4
no items          0   0   0   0   0
(1, 15), down     0  15  15  15  15
(3, 20), down     0  15  15  20  35
(4, 30), down     0  15  15  20  35   0/1 answer 35
(1, 15), up       0  15  30  45  60   unbounded: weight 1 four times
```

```python
def knapsack(items, capacity, unbounded):
    dp = [0] * (capacity + 1)       # TAble + SEed: no items, value 0
    for weight, value in items:     # FIll: items outer
        if unbounded:               # up: dp[c - weight] may hold this item
            caps = range(weight, capacity + 1)
        else:                       # down: dp[c - weight] is the old row
            caps = range(capacity, weight - 1, -1)
        for c in caps:
            dp[c] = max(dp[c], dp[c - weight] + value)
    return dp[capacity]             # REad: the full budget
```

Example.

```python
items = [(1, 15), (3, 20), (4, 30)]
print(knapsack(items, 4, False))          # 35
```

Dials. The combine (`max`, `min` with `inf` seeds, `or`, `+` for counts), exact size (seed only `dp[0]`) or at most, the direction of the capacity loop, the number of capacities, and for counts which loop is outer.

Traps. Looping a 0/1 capacity upward, which reuses the item. Swapping the loops of a count, which turns combinations into ordered sequences. Seeding a minimum with `0` instead of `inf`.

PROBLEMS

LC 416 · Partition Equal Subset Sum

Problem: Given an array of positive integers `nums` (at most 200, each at most 100), return true if it splits into two subsets with equal sums: `[1,5,11,5]` gives true (`[1,5,5]` and `[11]`).

Intuition: Two equal halves exist exactly when some subset sums to half the total, so the question is a 0/1 knapsack of booleans: which sums are reachable. Each number lands in one half once, so the budget loop runs down; half the total is at most 10,000, so the row is short.

Differs: booleans, exact size: the capacity is `total // 2` (an odd total returns `False`), `dp[0] = True`, and `dp[c] = dp[c] or dp[c - x]`, capacity loop down.

```python
dp = [True] + [False] * half
for x in nums:
    for c in range(half, x - 1, -1):
        dp[c] = dp[c] or dp[c - x]
```

LC 494 · Target Sum

Problem: Put a `+` or `-` before every number of `nums` and add them up. Return the number of sign choices that give `target`: `[1,1,1,1,1]` with `target = 3` gives 5 (one minus, five places).

Intuition: With `P` the sum signed `+` and `M` the rest, `P - M = target` and `P + M = total`, so `P = (total + target) / 2`: a sign choice is a subset choice. A `0` takes either sign, and `dp[c] += dp[c - 0]` doubles the count by itself.

Differs: a count with `+`, exact size, budget `goal = (total + target) // 2`; return `0` if `total + target` is odd or `abs(target) > total`. 0/1, loop down to `x` inclusive.

```
dp = [1] + [0] * goal
for x in nums:
    for c in range(goal, x - 1, -1):
        dp[c] += dp[c - x]
```

LC 474 · Ones and Zeroes

Problem: Given binary strings `strs` and integers `m` and `n`, return the size of the largest subset with at most `m` zeros and `n` ones in total: `["10","0001","111001","1","0"]`, `m = 5`, `n = 3` gives 4.

Intuition: Each string is an item used at most once, and its weight is in two resources at the same time, its zeros and its ones. The row of budgets becomes a grid of budgets, and both loops run down so that every cell still reads the grid without this string.

Differs: two capacities: `dp[i][j]` is the largest subset within `i` zeros and `j` ones, each string costs `(zeros, ones)` and has value 1, and both capacity loops run down.

```
for i in range(m, zeros - 1, -1):
    for j in range(n, ones - 1, -1):
        dp[i][j] = max(dp[i][j], dp[i - zeros][j - ones] + 1)
```

LC 322 · Coin Change

Problem: Given coin values `coins`, each available without limit, and an `amount`, return the fewest coins that make exactly `amount`, or -1: coins `[1,2,5]` and amount 11 give 3 (5 + 5 + 1).

Intuition: Coins repeat, so this is the unbounded knapsack with `min`: the upward loop lets `dp[c - coin]` already contain the same coin. Exact size means only `dp[0]` starts at 0; every other amount starts at `inf`, which marks "no coins make this".

Differs: unbounded, `min`, exact size: `dp[0] = 0`, every other cell `inf`, loop up, `dp[c] = min(dp[c], dp[c - coin] + 1)`. Return `-1` if `dp[amount]` is still `inf`.

LC 518 · Coin Change II

Problem: Given unlimited coins of each value in `coins` and an `amount`, return the number of combinations that make it, order ignored: amount 5 with `[1,2,5]` gives 4: 5, 2+2+1, 2+1+1+1 and 1+1+1+1+1.

Intuition: With coins in the outer loop, every combination is built in one fixed order of coin kinds: all 1s first, then 2s, then 5s. So `1+2` and `2+1` are the same build and are counted once, which is what "combinations" asks.

Differs: unbounded count with `+`: coins outer, amounts inner going up, `dp[0] = 1`, `dp[c] += dp[c - coin]`.

LC 377 · Combination Sum IV

Problem: Given distinct positive integers `nums` and a `target`, return the number of sequences of them, repeats allowed, that sum to `target`; different orders count apart: `[1,2,3]` with target 4 gives 7.

Intuition: Order matters here, so a sequence is decided by its last number: the count for `a` sums the counts for `a - x` over every `x`. With the amount outer, any number can follow any other, which is exactly what the coins-outer order of LC 518 forbids.

Differs: the loops swap: amounts outer, numbers inner, `dp[a] += dp[a - x]` for every `x <= a`, `dp[0] = 1`.

```
for a in range(1, target + 1):
    for x in nums:
        if x <= a:
            dp[a] += dp[a - x]
```

LC 279 · Perfect Squares

Problem: Given `n` (`1 <= n <= 10^4`), return the fewest perfect squares (1, 4, 9, ...) that sum to `n`: 12 gives 3 (4 + 4 + 4); 13 gives 2 (4 + 9).

Intuition: The squares up to `n` are coins that may repeat, so this is LC 322 with a generated coin list. There are at most 100 squares, so about a million cell updates, and every amount is reachable because 1 is a square.

Differs: LC 322 with the items being the squares `1, 4, 9, ...` up to `n`; unbounded `min`, loop up. No `-1` case.

```
squares = [k * k for k in range(1, int(n ** 0.5) + 1)]
```

2-D table · Tasefire

dp[i][j] from up, left and diagonal.

Picture. A grid filled row by row, left to right, with a border row and column seeded first; every cell reads its upper, left and upper-left neighbours.

Invariant. When cell `(i, j)` is written, every cell in the rows above and to its left in row `i` is final; `dp[i][j]` answers the grid up to `(i, j)`, or the prefixes `a[:i]` and `b[:j]`.

Use when. paths moving only right or down, the largest square of ones, two strings: common subsequence, edit distance, interleaving, wildcard or regular expression matching.

Book entries. none: grid and two-string tables are classic problems outside the book.

GENERAL SOLUTION

Problem. Given two strings `a` and `b`, return the length of the shortest string that has both as subsequences: “abac” and “cab” give 5, for example “cabac”.

Intuition. Look at the last letters of the two prefixes. If they match, the answer writes that letter once and comes from the diagonal cell; if not, one of the two is written last, so the answer is one more than the better of the cells above and to the left. The border is an empty prefix, which costs a copy of the other one.

```
      ""  c  a  b
  ""   0  1  2  3
  a    1  2  2  3
  b    2  3  3  3
  a    3  4  4  4
  c    4  4  5  5      answer: bottom-right cell, 5
```

```
def table_2d(a, b):
    rows, cols = len(a) + 1, len(b) + 1
    dp = [[0] * cols for _ in range(rows)]  # TAble: a[:i] against b[:j]
    for i in range(rows):
        dp[i][0] = i                        # SEed: b empty, copy a[:i]
    for j in range(cols):
        dp[0][j] = j                        # SEed: a empty, copy b[:j]
    for i in range(1, rows):                # FIll: row by row
        for j in range(1, cols):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1       # diagonal
            else:
                dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1
    return dp[-1][-1]                       # REad: both strings whole
```

Example.

```
print(table_2d("abac", "cab"))            # 5
```

Dials. What a cell means (paths, a cost, a side length, a yes or no), the border seeds, which of the three neighbours a cell reads and under which test, and the combine. A cell that reads only the row above and its left neighbour needs one row of memory.

Traps. Off by one between `dp` indices and string indices: `dp[i][j]` looks at `a[i - 1]` and `b[j - 1]`. Forgetting the border, which carries the cost of an empty prefix. Building rows with `[[0] * cols] * rows`, which aliases one row.

PROBLEMS

LC 62 · Unique Paths

Problem: A robot starts at the top-left cell of an `m x n` grid and moves only right or down. Return the number of paths to the bottom-right cell: `m = 3`, `n = 7` gives 28; `m = 3`, `n = 2` gives 3.

Intuition: The last move into a cell came from above or from the left, two disjoint cases, so the counts add; the first row and column have one way in each. A cell reads only the row above and its left neighbour, so one row, updated in place, holds the table.

Differs: cells count paths: `dp[i][j] = dp[i-1][j] + dp[i][j-1]`, the first row and column all `1`, no strings and no test. One row suffices: `row[j] += row[j - 1]`.

```
row = [1] * n
for _ in range(1, m):
    for j in range(1, n):
        row[j] += row[j - 1]
```

LC 64 · Minimum Path Sum

Problem: Given an `m x n` grid of non-negative numbers, move only right or down from the top-left to the bottom-right cell. Return the smallest sum along such a path: `[[1,3,1],[1,5,1],[4,2,1]]` gives 7 (1, 3, 1, 1, 1).

Intuition: The cheapest route into a cell ends with a step from the cheaper of its two predecessors, above or left, plus the cell itself. Border cells have only one way in, so the first row and column are running sums.

Differs: cells hold the cheapest sum: `grid[i][j] + min(up, left)`; the border is the running sum along the first row and the first column. The grid itself can serve as the table.

LC 221 · Maximal Square

Problem: Given an `m x n` matrix of characters `'0'` and `'1'`, return the area of the largest square containing only `'1'`: `[["0","1"],["1","0"]]` gives 1.

Intuition: Let a cell be the side of the largest all-ones square whose bottom-right corner is here. A square of side `k` ends here exactly when squares of side `k - 1` end above, to the left and on the diagonal, so the smallest of those three, plus one, is the side.

Differs: `min(up, left, diagonal) + 1` on a `'1'`, `0` on a `'0'`, with a zero border. REad the largest cell anywhere, squared.

```
if matrix[i - 1][j - 1] == "1":
    dp[i][j] = min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]) + 1
    best = max(best, dp[i][j])
```

LC 1143 · Longest Common Subsequence

Problem: Given strings `text1` and `text2`, return the length of their longest common subsequence, letters kept in order but not necessarily adjacent, or 0: “abcde” and “ace” give 3 (“ace”).

Intuition: If the last letters of the two prefixes match, some longest common subsequence ends with that pair, so take the diagonal plus one. If not, at least one of the two letters is unused, and dropping it loses nothing: take the better of up and left.

Differs: border `0`; on a match the diagonal plus one, otherwise `max(up, left)` with no `+ 1`.

LC 72 · Edit Distance

Problem: Given `word1` and `word2`, return the fewest single-letter inserts, deletes and replacements that turn `word1` into `word2`: “horse” to “ros” gives 3 (replace h, delete r, delete e).

Intuition: The last operation deletes the last letter of `word1` (up), inserts the last letter of `word2` (left), or replaces one by the other (diagonal); matching last letters cost nothing and copy the diagonal. The border is the cost of building a prefix from nothing.

Differs: border `i` and `j` as in the general solution; a match copies the diagonal, a mismatch is `1 + min(up, left, diagonal)`.

```
if word1[i - 1] == word2[j - 1]:
    dp[i][j] = dp[i - 1][j - 1]
else:
    dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
```

LC 97 · Interleaving String

Problem: Given `s1`, `s2` and `s3`, return true if `s3` is formed by merging all letters of `s1` and `s2`, each keeping its own order: “aabcc”, “dbbca” and “aadbbcbcac” give true.

Intuition: A cell asks whether `s1[:i]` and `s2[:j]` interleave into `s3[:i + j]`. The last letter of that prefix of `s3` came from the end of the `s1` part or of the `s2` part; whichever matches, the cell inherits the answer of the prefix without it.

Differs: booleans, `dp[0][0] = True`; a cell reads up when `s1[i-1]` equals `s3[i+j-1]` and left when `s2[j-1]` does. Lengths that do not add up return `False` before the fill.

```
c = s3[i + j - 1]
dp[i][j] = ((i > 0 and dp[i - 1][j] and s1[i - 1] == c) or
            (j > 0 and dp[i][j - 1] and s2[j - 1] == c))
```

LC 10 · Regular Expression Matching

Problem: Given a string `s` and a pattern `p` where `'.'` matches any letter and `'*'` matches zero or more of the element before it, return true if `p` matches all of `s`: “aa” with “a” gives false, with “a*” true.

Intuition: `x*` is one unit: it is dropped whole, which looks two pattern positions back, or it eats one more letter of `s` and stays in the same column, which looks up. A plain letter or `'.'` consumes one letter of each and looks at the diagonal.

Differs: booleans over `s[:i]` and `p[:j]`; a plain letter or `'.'` reads the diagonal. A `'*'` reads two columns back or, if `s[i-1]` matches `p[j-2]`, up. The top row lets `a*b*` match “”.

```
if p[j - 1] == "*":
    dp[i][j] = dp[i][j - 2] or (
        i > 0 and p[j - 2] in (s[i - 1], ".") and dp[i - 1][j])
```

Interval · Tasefire

dp[l][r] filled by increasing length.

Picture. The triangle above the diagonal of an `n x n` grid, filled one diagonal at a time from the main diagonal outward; the top-right corner is the whole input.

Invariant. When `dp[l][r]` is written, every interval shorter than `r - l + 1` is final, and the cell reads only those: the inner `(l + 1, r - 1)`, the trims `(l + 1, r)` and `(l, r - 1)`, or the two sides of a split.

Use when. palindromic substring or subsequence, take from either end, merge or burst neighbours, the answer for a range is built from the answers for its sub-ranges.

Book entries. #87 Longest Palindromic Substring

GENERAL SOLUTION

Problem. Two players take turns removing a number from either end of a row and keep it. Return the largest lead, own total minus the opponent’s, that the player to move can secure when both play best: `[1, 5, 2]` gives -2, because whichever end the first player takes, the 5 is left exposed.

Intuition. Taking an end leaves the opponent the same game on a shorter interval, so the lead on `nums[l..r]` is the better of `nums[l]` minus the opponent's lead on `l+1..r` and `nums[r]` minus the lead on `l..r-1`. Both are shorter, so filling by increasing length makes every read final.

```
length 1   dp[0][0] = 1    dp[1][1] = 5    dp[2][2] = 2
length 2   dp[0][1] = max(1 - 5, 5 - 1) = 4
           dp[1][2] = max(5 - 2, 2 - 5) = 3
length 3   dp[0][2] = max(1 - 3, 2 - 4) = -2
```

```python
def interval(nums):
    n = len(nums)
    if n == 0:
        return 0
    dp = [[0] * n for _ in range(n)]        # TAble: lead on nums[l..r]
    for i in range(n):
        dp[i][i] = nums[i]                  # SEed: length 1
    for length in range(2, n + 1):          # FIll: by increasing length
        for l in range(n - length + 1):
            r = l + length - 1
            take_left = nums[l] - dp[l + 1][r]
            take_right = nums[r] - dp[l][r - 1]
            dp[l][r] = max(take_left, take_right)
    return dp[0][n - 1]                     # REad: the whole row
```

Example.

```python
print(interval([1, 5, 2]))              # -2
```

Dials. What a cell means (a lead, a palindrome flag, a length, a count), which inner cells it reads (the inner pair, the two trims, or every split point), and the fill order: by length, or `l` going down with `r` going up, which also finishes every shorter interval first.

Traps. Filling by `l` going up, which reads `dp[l + 1][...]` before it is written. Choosing the first balloon of a range to burst, whose neighbours are still changing. Using an interval table where the centers of #87 do the same in `O(1)` space.

PROBLEMS

LC 5 · Longest Palindromic Substring

Problem: Given a string `s` (length at most 1000), return its longest substring that reads the same backwards: "babad" gives "bab" ("aba" is also accepted); "cbbd" gives "bb".

Intuition: A palindrome is a shorter palindrome with a matching letter added at each end, so a cell reads only its inner cell. Every chain of such cells starts at one of the `2n - 1` centers, a letter or a gap, so walking outward from each center finds every palindrome.

Differs: #87 drops the table: from each center two arms move outward while the letters match; the longest window wins. The table form, by length, is `pal[l][r] = s[l] == s[r] and (r - l < 2 or pal[l + 1][r - 1])`.

LC 647 · Palindromic Substrings

Problem: Given a string `s`, return how many of its substrings are palindromes, counting substrings at different positions separately: "abc" gives 3; "aaa" gives 6.

Intuition: Each successful step of an expansion from a center uncovers exactly one new palindrome, the window it now covers, and no palindrome is reached from two centers. So count the steps instead of keeping the longest window.

Differs: count instead of keeping the longest: add one for every successful expansion step from every center, or count the true cells of `pal`.

```python
while left >= 0 and right < n and s[left] == s[right]:
    count += 1
    left, right = left - 1, right + 1
```

LC 516 · Longest Palindromic Subsequence

Problem: Given a string `s`, return the length of its longest subsequence (letters kept in order, gaps allowed) that is a palindrome: “bbbab” gives 4 (“bbbb”); “cbbd” gives 2.

Intuition: A subsequence may skip letters, so a mismatched end is dropped, not fatal: take the better of the two trims. Matching ends can always be the outer pair of a longest one, so they add 2 to the inner interval.

Differs: lengths, seed `dp[i][i] = 1`: matching ends give `dp[l + 1][r - 1] + 2`, otherwise `max(dp[l + 1][r], dp[l][r - 1])`.

```
if s[l] == s[r]:
    dp[l][r] = dp[l + 1][r - 1] + 2
else:
    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1])
```

LC 1312 · Minimum Insertion Steps to Make a String Palindrome

Problem: Given a string `s`, return the fewest letters to insert anywhere so that it becomes a palindrome: “mbadm” gives 2 (to “mbdadbm”); “zzazz” gives 0.

Intuition: The letters of a longest palindromic subsequence already pair up around a center. Every other letter needs one inserted mirror, and no plan can do with fewer, so the answer is the length minus LC 516’s answer.

Differs: return `len(s)` minus LC 516’s answer; or fill directly with `dp[l + 1][r - 1]` on matching ends and `1 + min(dp[l + 1][r], dp[l][r - 1])` otherwise.

LC 132 · Palindrome Partitioning II

Problem: Given a string `s` (length at most 2000), return the fewest cuts that split it into pieces that are all palindromes: “aab” gives 1 (“aa” and “b”); “a” gives 0.

Intuition: The last piece of a best split is some palindrome `s[l..r]`, and before it lies a best split of `s[:l]`. So first fill the interval table of which ranges are palindromes, then run a chain over end points that tries every palindromic last piece; n^2 cells fit.

Differs: two tables: the interval table `pal[l][r]` by length, then a chain over end points, `cuts[r] = min(cuts[l - 1] + 1)` over every `l` with `pal[l][r]`, and `0` when `pal[0][r]`.

```
for r in range(n):
    cuts[r] = 0 if pal[0][r] else min(
        cuts[l - 1] + 1 for l in range(1, r + 1) if pal[l][r])
```

LC 312 · Burst Balloons

Problem: Balloons `nums` (at most 300) stand in a row. Bursting balloon `i` earns `nums[i-1] * nums[i] * nums[i+1]` with its current neighbours, a missing one counting as 1. Return the most coins: `[3,1,5,8]` gives 167.

Intuition: Choose the *last* balloon `k` to burst in an open range `(l, r)`: at that moment its neighbours are the walls `l` and `r`, which are fixed, and the two sides were emptied independently. The first balloon to burst has neighbours that keep changing, so it does not split the range.

Differs: pad `nums` with `1` at both ends; `dp[l][r]` is the best over the open interval, the max over every `k` in `(l, r)` of `dp[l][k] + dp[k][r] + v[l] * v[k] * v[r]`. n^3 fits.

```
for k in range(l + 1, r):
    gain = v[l] * v[k] * v[r]
    dp[l][r] = max(dp[l][r], dp[l][k] + dp[k][r] + gain)
```

Subsequence · Tasefire

Best earlier compatible element; sorted tails speed it up.

Picture. Each element looks back over the earlier elements it may follow and hangs itself on the end of the longest chain among them; with sorted tails, the look back becomes one binary search.

Invariant. At the top of step `i`, `dp[j]` is the longest chain ending at element `j` for every `j < i`; in the fast form, `tails[k]` is the smallest last value of a chain of length `k + 1`, and `tails` is increasing.

Use when. longest increasing, longest chain, fits inside, nested, non-overlapping jobs with the most profit; the order is fixed (or a sort fixes it) and the chain may skip elements.

Book entries. #58 Longest Increasing Subsequence

GENERAL SOLUTION

Problem. Given a sequence and a test `fits(a, b)`, true when `b` may follow `a`, return the length of the longest chain of elements kept in their order, each fitting after the one before. With `a < b` it is the longest strictly increasing subsequence: `[2, 5, 3, 7, 4, 1, 8]` gives 4, from 2, 3, 4, 8.

Intuition. The longest chain ending at `i` is `i` hung on the longest chain ending at an earlier `j` it fits after, and left to right every such `dp[j]` is already final. For `a < b` keep only the smallest end of each length, since whatever extends a larger end extends a smaller one; those ends rise with the length, so one binary search places `x`.

```
x   dp[i]  hangs on   tails after
2   1      -          [2]
5   2      2          [2, 5]
3   2      2          [2, 3]         3 is a smaller end for length 2
7   3      5 or 3     [2, 3, 7]
4   3      3          [2, 3, 4]
1   1      -          [1, 3, 4]      not a real chain; only the length counts
8   4      7 or 4     [1, 3, 4, 8]   answer 4
```

```
from bisect import bisect_left


def subsequence(items, fits):
    dp = [1] * len(items)               # TAble + SEed: each item alone
    for i in range(len(items)):         # FIll: left to right
        for j in range(i):              # every earlier item i may follow
            if fits(items[j], items[i]):
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp, default=0)           # REad: the best end anywhere


def lis_tails(nums):
    tails = []                          # TAble + SEed: no lengths yet
    for x in nums:                      # FIll: one binary search per x
        k = bisect_left(tails, x)       # first tail >= x
        if k == len(tails):
            tails.append(x)             # x extends the longest chain
        else:
            tails[k] = x                # a smaller end for length k + 1
    return len(tails)                   # REad: the number of lengths
```

Example.

```
less = lambda a, b: a < b
print(subsequence([2, 5, 3, 7, 4, 1, 8], less))             # 4
print(lis_tails([2, 5, 3, 7, 4, 1, 8]))                     # 4
```

Dials. The compatibility test, the sort that fixes the order first, what a cell carries (a length, a count, a profit), and the search over tails: `bisect_left` for strictly increasing, `bisect_right` for non-decreasing.

Traps. Reading `dp[-1]` instead of the best cell anywhere: the chain may end at any element. Treating `tails` as a real subsequence; only its length is meaningful. Swapping `bisect_left` and `bisect_right`, which lets equal values chain or forbids it wrongly.

PROBLEMS

LC 300 · Longest Increasing Subsequence

Problem: Given an integer array `nums` (length at most 2500), return the length of its longest strictly increasing subsequence, values kept in order but not necessarily adjacent: `[10,9,2,5,3,7,101,18]` gives 4 (2, 3, 7, 101); `[7,7,7,7]` gives 1.

Intuition: A smaller end is the better state for its length: whatever extends a larger end extends it too. So keep the smallest end of each length; they rise strictly, and a new value replaces the first end at least as large, or extends the longest chain. An equal value replaces its twin.

Differs: nothing: `lis_tails` as written, where `bisect_left` is the lower bound of #58. The quadratic `subsequence` with `a < b` also passes, since `n <= 2500` is about three million pairs.

LC 354 · Russian Doll Envelopes

Problem: `envelopes[i] = [w, h]` (up to `10^5` of them); one fits inside another when both its width and height are strictly smaller, with no rotation. Return the most envelopes nested one in another: `[[5,4],[6,4],[6,7],[2,3]]` gives 3 ([2,3] in [5,4] in [6,7]).

Intuition: Sorting by width leaves only the heights to test: a chain in width order needs rising heights. Equal widths cannot nest, so sort them by height descending; their heights then fall, and a strictly rising run takes at most one of them. `10^5` envelopes need the tails form.

Differs: sort by width ascending and, for equal widths, by height descending; then `lis_tails` on the heights alone.

```
envelopes.sort(key=lambda e: (e[0], -e[1]))
heights = [h for w, h in envelopes]
```

LC 673 · Number of Longest Increasing Subsequence

Problem: Given an integer array `nums` (length at most 2000), return how many strictly increasing subsequences have the longest length, different index choices counting apart: `[1,3,5,4,7]` gives 2 (1, 3, 4, 7 and 1, 3, 5, 7); `[2,2,2,2,2]` gives 5.

Intuition: A longest chain ending at `i` is a longest chain ending at a compatible `j`, plus `i`; different `j` give different chains, so the counts of the `j` tied for the best length add, and a longer `j` restarts the sum. Tails forget counts; `n <= 2000` lets the n^2 table run.

Differs: the quadratic table carries `count[i]` beside `length[i]`, both seeded 1: a longer chain through `j` resets `count[i] = count[j]`, an equally long one adds `count[j]`. REad: sum the counts at the maximum length.

```
if nums[j] < nums[i]:
    if length[j] + 1 > length[i]:
        length[i], count[i] = length[j] + 1, count[j]
    elif length[j] + 1 == length[i]:
        count[i] += count[j]
```

LC 1048 · Longest String Chain

Problem: A word is a predecessor of another if inserting exactly one letter anywhere makes it the other: “ba” precedes “bda”. Given `words` (at most 1000, each at most 16 letters), return the longest chain of predecessors: `["a","b","ba","bca","bda","bdca"]` gives 4 (a, ba, bda, bdca).

Intuition: A predecessor is exactly one letter shorter, so in length order every predecessor is filled first. Instead of testing every earlier word, delete each letter in turn: the at most 16 results are the only candidates, and a map from word to best chain looks each one up.

Differs: sort words by length; the scan over earlier `j` becomes a lookup: `best[w]` is one more than the best `best` among the one-letter deletions of `w`, a missing word counting 0. REad the largest value.

```
for w in sorted(words, key=len):
    best[w] = 1 + max((best.get(w[:i] + w[i + 1:], 0)
                       for i in range(len(w))), default=0)
```

LC 1964 · Find the Longest Valid Obstacle Course at Each Position

Problem: Given obstacle heights `obstacles` (length up to `10^5`), return for every `i` the length of the longest non-decreasing subsequence of `obstacles[0..i]` that ends with obstacle `i`: `[1,2,3,2]` gives `[1,2,3,3]`; `[2,2,1]` gives `[1,2,1]`.

Intuition: The slot where `x` lands is the length of the longest course it ends: every tail before the slot is an end `x` may follow, and the tail in it is too tall. Non-decreasing lets an equal height follow its twin, so the slot comes after every tail `<= x`.

Differs: `bisect_right` instead of `bisect_left`, and every position records `k + 1`, the length of the course the obstacle ends, before writing `x` into the slot.

```
k = bisect_right(tails, x)
answer.append(k + 1)
```

LC 1235 · Maximum Profit in Job Scheduling

Problem: Job `i` runs from `startTime[i]` to `endTime[i]` and pays `profit[i]` (`n <= 5 * 10^4`). Return the largest total profit of jobs with no overlap; a job may start the moment another ends: starts `[1,2,3,3]`, ends `[3,4,5,6]`, profits `[50,10,40,70]` give 120.

Intuition: In end order, the best over the first `i` jobs skips job `i` or takes it on top of the best over the jobs ending by its start. Those form a prefix of the order, found by one binary search, and a prefix-best table holds a prefix's best in one cell.

Differs: sort by end time; the cell is the best over the first `i` jobs, not the best chain ending at `i`: `dp[i] = max(dp[i - 1], dp[k] + profit)`, with `k = bisect_right(ends, start)` the jobs ending by this start.

```
k = bisect_right(ends, start)
dp[i] = max(dp[i - 1], dp[k] + profit)       # i counts from 1
```

Serve

The caller drives the loop; an invariant survives every call.

The question. What must stay true between calls?

Picture. A switchboard. Calls come in one at a time, from a loop the caller drives and may stop at any moment. Each call is routed straight to its line, changes it, and puts the board back in order before hanging up, so the next call, whatever it is, finds everything where it should be.

Invariant. At every return to the caller, the structures agree with each other and the next call can be answered from them in one cheap step: no stale entry, no missing one.

Shared skeleton.

```
class Service:
    def __init__(self):
        self.index = {}          # dial: a map, slots, parents, a trie, counts
        self.order = []          # dial: a list, buckets, an array, nothing

    def op(self, key):           # one call is one turn of the caller's loop
        node = self.locate(key)  # dial: look up, hash, find the root, walk
        self.change(node)        # the work the call asks for
        self.repair()            # restore the invariant before returning
        return node
```

Dials. How a call locates its key, and what order structure is kept beside the lookup. A map can point straight into a linked list, a packed array or a set of buckets; a hash picks a slot and a probe settles collisions; parent pointers climb to the root that names a group; a trie walks one letter at a time; a tally keeps one counter per key and files the keys by count. The repair is whatever the change broke: a node moved to the front, a last element moved into a hole, a table rebuilt twice as wide, a small tree hung under a big one.

Method	Mnemonic	What it sets	Book entries
Map and order	OpMapSync	a map into a list, buckets or an array; both kept in sync	#6, #44, #91, #93, #141
Hash to slots	Haprop	a slot from the hash, a probe past collisions, a load bound	#51, #101, #139, #146, #147
Roots	FiFiSaSu	parent pointers; the root names the group	#41, #118
Trie	WaCreStep	one node per prefix, a flag or a count at each node	#43, #97, #143
Tally	Cobuco	a counter per key or line, buckets indexed by count	#48, #61, #71, #108

In the other six families the code owns its loop and reads the answer once, at the end. Here the caller owns the loop, so every call must be cheap and must leave the structures right, because any call may be the last. Some problems in this family are one function, not a class: top k frequent, redundant connection, repeated DNA sequences. Read them as a service whose caller is a `for` loop over the input, one call per item (count, union, insert), then one query. The family-wide trap is two structures that drift apart: a map entry whose node was never unlinked, a moved value whose index was not corrected, an empty bucket left in the list. Say the invariant in one sentence, then check that every method restores it on every path, the update of an existing key and the removal of the last element included.

Map and order · OpMapSync

List the OPerations, keep a MAP plus an order structure, SYNC both in every method.

Picture. A card index beside a shelf. The index finds any card's place on the shelf at once, and every call that moves, adds or removes a card fixes the shelf and the index before it returns.

Invariant. Every key in the map points at its one live place in the order structure, and every place in the order structure belongs to a key in the map.

Use when. design a cache, `get` and `put` in O(1), least recently used, least frequently used, insert, delete and get random in O(1), most frequent with ties broken by recency, a deep copy of nodes with extra pointers, reverse a list or groups of it.

Book entries. #6 Reverse a Linked List · #44 LRU Cache · #91 Insert Delete GetRandom O(1) · #93 LFU Cache · #141 All O(1) Data Structure

GENERAL SOLUTION

Problem. Design a cache of fixed capacity: `get(key)` returns the value or -1, `put(key, value)` inserts or updates, both in O(1), and a put that overflows the capacity evicts the least recently used key. Capacity 2: `put(1, 10)`, `put(2, 20)`, `get(1)` gives 10, `put(3, 30)` evicts 2, then `get(2)` gives -1.

Intuition. Two questions need two structures. "Where is key k?" is a map's job; "who was used least recently?" is the job of a doubly linked list from newest to oldest, whose nodes unlink themselves in O(1) and whose arrows the map holds. Every use unlinks a node and splices it in after `head`, so the node before `tail` is always the one to evict. Sentinels at both ends mean no splice ever meets an empty list.

```
call          list, newest first   map keys
put(1, 10)    1                    {1}
put(2, 20)    2 1                  {1, 2}
get(1) → 10   1 2                  {1, 2}      a read moves 1 to the front
put(3, 30)    3 1                  {1, 3}      over capacity: 2 leaves both
get(2) → -1   3 1                  {1, 3}      a miss moves nothing
```

```python
class Node:
    def __init__(self, key=0, value=0):
        self.key, self.value = key, value  # the key lets eviction clean the map
        self.prev = self.next = None


class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.nodes = {}                    # MAP: key -> its node in the list
        self.head, self.tail = Node(), Node()   # sentinels at both ends
        self.head.next, self.tail.prev = self.tail, self.head

    def unlink(self, node):                # the two neighbours close the gap
        node.prev.next, node.next.prev = node.next, node.prev

    def push_front(self, node):            # the newest sits right after head
        node.prev, node.next = self.head, self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):                    # OP 1: a read is a use
        if key not in self.nodes:
            return -1
        node = self.nodes[key]
        self.unlink(node)                  # SYNC: move it to the front
        self.push_front(node)
        return node.value

    def put(self, key, value):             # OP 2: a write is a use too
        if key in self.nodes:
            node = self.nodes[key]         # update in place: never a 2nd node
            node.value = value
            self.unlink(node)
        else:
            node = Node(key, value)
            self.nodes[key] = node
        self.push_front(node)              # SYNC: newest after head
        if len(self.nodes) > self.capacity:
            oldest = self.tail.prev        # the least recently used
            self.unlink(oldest)
            del self.nodes[oldest.key]     # SYNC: list and map both forget it
```

Example.

```
cache = LRUCache(2)
cache.put(1, 10)
cache.put(2, 20)
print(cache.get(1))                    # 10
cache.put(3, 30)                       # over capacity: 2 is the oldest
print(cache.get(2))                    # -1
cache.put(1, 11)                       # an update is a use, not a 2nd node
cache.put(4, 40)                       # evicts 3
one = LRUCache(1)
one.put(5, 50)
one.put(6, 60)
```

Dials. What the map points at (a list node, an array index, a node inside a bucket, a copy) and what the order structure is: a recency list, one recency list per use count plus the smallest count (#93), a packed array (#91), a stack per count, a linked list of count buckets (#141). The order alone, with no map, is the relinking of #6.

Traps. A `put` to an existing key that makes a second node leaves a ghost in the list whose eviction later deletes the live key. Forgetting that `get` is a use. Deleting a key from the map before correcting the value that moved into its place.

PROBLEMS

LC 206 · Reverse Linked List

Problem: Given the head of a singly linked list, reverse it and return the new head: `[1,2,3,4,5]` gives `[5,4,3,2,1]`.

Intuition: Two chains: the reversed part headed by `prev` and the rest headed by `curr`. Each step lifts the front node off the rest and lays it on the reversed part. An arrow can be turned the moment you stand on its node, provided you first saved where it pointed.

Differs: no map and no caller: the order structure alone, and the relinking primitive every linked design reuses: save next, flip, step. Return `prev`, since `curr` ends as `None`.

```
prev, curr = None, head
while curr:
    nxt = curr.next                    # save the rest before the cut
    curr.next = prev                   # flip the arrow
    prev, curr = curr, nxt             # step: both move one node on
return prev
```

LC 25 · Reverse Nodes in k-Group

Problem: Given a linked list and k, reverse its nodes k at a time, leaving a last group shorter than k as it is; only links may change, not values: `[1,2,3,4,5]` with k = 2 gives `[2,1,4,3,5]`.

Intuition: A group is reversed by the save, flip, step of LC 206 run exactly k times. Start `prev` at the node after the group, so the group's old first node, now its last, already points on to the rest. Then the node before the group points at the old kth node.

Differs: a dummy before the head is `group_prev`; walk k steps to `kth`, and stop if the list ends first. Reverse k nodes, stitch, and move `group_prev` to the old first node.

```
prev, curr = kth.next, group_prev.next    # prev: the node after the group
for _ in range(k):                        # LC 206 for exactly k nodes
    nxt = curr.next
    curr.next = prev
    prev, curr = curr, nxt
group_prev.next, group_prev = kth, group_prev.next   # stitch, move on
```

LC 146 · LRU Cache

Problem: Design `LRUCache(capacity)` with `get(key)`, the value or -1, and `put(key, value)`; when the keys exceed the capacity, evict the least recently used; both in O(1) average: capacity 2, `put(1,1)`, `put(2,2)`, `get(1)` = 1, `put(3,3)` evicts 2, `get(2)` = -1.

Intuition: The general solution: the map answers where a key is, the doubly linked list answers who is oldest, and every call that touches a key moves its node to the front, so the tail is always the least recently used.

Differs: none. `OrderedDict` is this same structure, a hash map threaded with a linked list, but interviewers usually want the list built by hand.

```
self.data.move_to_end(key)                # a use: the newest sits last
if len(self.data) > self.capacity:
    self.data.popitem(last=False)         # the oldest sits first
```

LC 460 · LFU Cache

Problem: Design `LFUCache(capacity)` with O(1) `get` and `put`, each a use of its key; when full, a new key evicts the least frequently used key, the least recent on a tie: capacity 2, `put(1,1)`, `put(2,2)`, `get(1)`, then `put(3,3)` evicts 2.

Intuition: Group the keys by use count, each group an LRU list: the victim is the oldest key of the lowest group. Counts rise one at a time, so the lowest count `min_freq` moves only two ways: up one when its last key moves up, back to 1 when a new key arrives.

Differs: the order structure is a map count → LRU list, plus `min_freq`. A use moves the node from list f to the front of list f + 1. When full, evict the tail of list `min_freq` before inserting.

```
bucket = self.buckets[node.freq]          # one LRU list per use count
bucket.unlink(node)
if bucket.is_empty() and self.min_freq == node.freq:
    self.min_freq += 1                    # its last key moved up
node.freq += 1
self.add(node)                            # front of the list for freq + 1
```

LC 380 · Insert Delete GetRandom O(1)

Problem: Design `RandomizedSet` with `insert(val)` and `remove(val)`, each returning whether the set changed, and `getRandom()`, every element equally likely, all in average O(1): `insert(1)` is true, `remove(2)` false, `insert(2)` true, then `getRandom()` gives 1 or 2.

Intuition: A random index of a packed array is a random element, and a map value → index finds any value. A removal would leave a hole, so the last value jumps into it and the array shrinks from the right; a set has no order to keep, which makes the jump legal.

Differs: the order structure is an array and the map holds indices. `remove` moves the last value into the hole, corrects its index, pops, and only then deletes `val`, which may be the last.

```
i, last = self.index_of[val], self.values[-1]
self.values[i] = last                     # the last value fills the hole
self.index_of[last] = i                   # and learns its new slot
self.values.pop()
del self.index_of[val]                    # only now: last may be val
```

LC 895 · Maximum Frequency Stack

Problem: Design `FreqStack`: `push(val)`, and `pop()`, which removes and returns the most frequent value, the one closest to the top on a tie: push 5, 7, 5, 7, 4, 5, then four pops give 5, 7, 5, 4.

Intuition: The f-th copy of a value goes on stack f, so the top of the highest stack is a most frequent value and, among ties, the one pushed last. A value on stack f also sits on stack f - 1, so when the top stack empties, the one below it is not empty.

Differs: a map value → count, a map count → stack, and `max_freq`. Push raises the count and appends to that count's stack; pop takes the top of stack `max_freq`, lowers the count, and lowers `max_freq` when that stack empties.

```
def pop(self):
    val = self.stacks[self.max_freq].pop()    # most frequent, then newest
    self.count[val] -= 1
    if not self.stacks[self.max_freq]:
        self.max_freq -= 1                    # its copies wait one stack down
    return val
```

LC 138 · Copy List with Random Pointer

Problem: Each node of a linked list has `next` and `random`, which points at any node or is null. Return a deep copy of brand new nodes: `[[7,null],[13,0],[11,4],[10,2],[1,0]]`, as pairs of value and random index, gives a new list with the same shape.

Intuition: A `random` may point forward to a node not copied yet, so copy first and wire later. A map from each old node to its copy is the sync between the two lists: any old pointer, followed through the map, lands on the matching new node.

Differs: the map is old node → copy, and the order structure is the new list. Walk the old list twice (`nodes` below): make every copy, then wire `next` and `random` through the map; seed it with `None: None`.

```
copies = {None: None}                     # old node -> its copy
for old in nodes:                         # pass 1: make every copy
    copies[old] = Node(old.val)
for old in nodes:                         # pass 2: wire through the map
    copies[old].next = copies[old.next]
    copies[old].random = copies[old.random]
```

Hash to slots · Haprop

HAsh the key to its home slot, PRObe past the other keys there, Place or read it.

Picture. A wall of pigeonholes. A key's hash names its hole; when other keys are already there, the key walks the hole's chain, or on to the next holes, until it finds itself or a free place. When the wall gets too full it is rebuilt twice as wide.

Invariant. Every key sits in the chain of its home slot `hash(key) % width`, or on the unbroken run of taken slots that starts there, and `size / width` stays under the load bound.

Use when. design a hash map or a hash set without built-in libraries, short codes for long keys, seen before, repeated substrings of a fixed length, which server owns a key, a membership test over a huge set where a rare false yes is fine, approximate counts over a stream.

Book entries. #51 Hash Map with Separate Chaining · #101 Hash Map with Open Addressing · #139 Consistent Hashing · #146 Bloom Filter · #147 Count-Min Sketch

GENERAL SOLUTION

Problem. Design a hash map without a built-in dictionary: `put(key, value)` inserts or overwrites, `get(key)` returns the value or -1, `remove(key)` deletes the key. With 8 slots and integers that hash to themselves, keys 1 and 9 share slot 1, and after `put(1, 10)` and `put(9, 90)` the call `get(9)` must still give 90.

Intuition. One long list of pairs costs O(n) per call. The hash cuts it into many short chains: a key always lands in slot `hash(key) % width`, so a call computes one index and scans one chain, comparing keys, because different keys can share a slot. The load `size / width` is the average chain length; doubling the width whenever it passes 0.75 keeps every call O(1) on average.

```
call          slot   the chain afterwards     size / width
put(1, 10)    1      [1:10]                   1 / 8
put(9, 90)    1      [1:10, 9:90]             2 / 8    a collision
put(1, 11)    1      [1:11, 9:90]             2 / 8    an overwrite
get(9) → 90   1      scan: 1 is not 9, 9 is
remove(1)     1      [9:90]                   1 / 8
```

```python
class ChainedMap:
    def __init__(self, width=8):
        self.buckets = [[] for _ in range(width)]   # one chain per slot
        self.size = 0

    def locate(self, key):                # HAsh the key, PRObe its chain
        chain = self.buckets[hash(key) % len(self.buckets)]
        for i, pair in enumerate(chain):
            if pair[0] == key:            # compare keys, not hashes
                return chain, i
        return chain, -1

    def put(self, key, value):            # Place: overwrite or append
        chain, i = self.locate(key)
        if i >= 0:
            chain[i][1] = value
            return
        chain.append([key, value])
        self.size += 1
        if self.size > 0.75 * len(self.buckets):    # repair the load
            pairs = [pair for bucket in self.buckets for pair in bucket]
            self.buckets = [[] for _ in range(2 * len(self.buckets))]
            for pair in pairs:            # rehash: the slot depends on width
                chain, _ = self.locate(pair[0])
                chain.append(pair)

    def get(self, key):
        chain, i = self.locate(key)
        return chain[i][1] if i >= 0 else -1

    def remove(self, key):
        chain, i = self.locate(key)
        if i >= 0:
            chain.pop(i)
            self.size -= 1
```

Example.

```python
table = ChainedMap(8)
table.put(1, 10)
table.put(9, 90)                          # 9 % 8 == 1: the same slot as key 1
print(table.get(9))                       # 90
table.put(1, 11)                          # an overwrite, not a second pair
table.remove(1)
for k in range(20):
    table.put(k, k * k)
```

Dials. What a slot holds: a chain of pairs (#51), one key with a probe to the next slot and a tombstone on removal (#101), k bits set to 1 by k hashes (#146), a counter in each of d rows whose minimum is read (#147), a server's point on a sorted ring found by bisect (#139). What the key is: the caller's key, a counter id, a window of the input. And the repair: double at load 0.75 for chains, at 0.5 counting tombstones for probing.

Traps. Copying each chain to the same index of the wider array instead of rehashing: key 9 stays in slot 1, but `get(9)` looks in slot 9. A probing `remove` that writes EMPTY cuts a run and strands every key behind it. Python's `hash()` of a string changes between runs, so a hash that must survive a restart needs a fixed function.

PROBLEMS

LC 706 · Design HashMap

Problem: Design `MyHashMap` without built-in hash tables: `put(key, value)` inserts or updates, `get(key)` returns the value or -1, `remove(key)` deletes; keys and values lie in 0 .. 10^6: `put(1, 1)`, `put(2, 2)`, `get(1)` = 1, `get(3)` = -1, `put(2, 1)`, `get(2)` = 1.

Intuition: The general solution as it stands. An integer key is its own hash, so a call scans the one chain `key % width`; the load bound keeps that chain a few pairs long however many keys arrive.

Differs: none in the steps; the slot is `key % len(self.buckets)`. With at most 10^4 calls, a fixed prime width such as 10007 can replace the resize; a prime spreads keys that share a stride.

LC 705 · Design HashSet

Problem: Design `MyHashSet` without built-in hash tables: `add(key)`, `remove(key)` and `contains(key)`, keys in 0 .. 10^6: `add(1)`, `add(2)`, `contains(1)` is true, `contains(3)` false, `add(2)`, `remove(2)`, then `contains(2)` is false.

Intuition: A set is a map without values, and open addressing drops the chains: a taken slot sends the key on to the next, so a key sits on the unbroken run that starts at its home. A search stops at the first empty slot, so a removal leaves a tombstone that searches step over.

Differs: slots hold keys: Probe walks `i = (i + 1) % width` until the key or EMPTY; `remove` writes DELETED; `add` searches first, then takes the first DELETED or EMPTY slot on its path. Resize when `(size + tombstones) / width` passes 0.5.

```
i = key % self.width                    # HAsh: the home slot
while self.slots[i] is not EMPTY:       # a run ends only at EMPTY
    if self.slots[i] == key:
        return True                     # a tombstone never equals a key
    i = (i + 1) % self.width            # PRObe: the next slot
return False
```

LC 535 · Encode and Decode TinyURL

Problem: Design `encode(longUrl)`, which returns a short URL, and `decode(shortUrl)`, which returns the original; any scheme works if decode undoes encode: “https://leetcode.com/problems/design-tinyurl” may become “http://tinyurl.com/4e9iAk”.

Intuition: A short code needs a key that never collides. A counter hands out ids 1, 2, 3, ..., and the id written in base 62, digits and both cases of letters, is the code: six characters cover 62^6, about 57 billion URLs. A map from code to URL decodes in one lookup.

Differs: Hash is a counter, so Probe never runs: no two URLs share an id. `ALPHABET` holds the 62 characters; Place is `long_of[code] = long_url`. A second map URL → code returns the same code when a URL comes back.

```
n, code = len(self.long_of) + 1, ""     # the next id: 1, 2, 3, ...
while n:
    n, digit = divmod(n, 62)            # peel one base-62 digit
    code = ALPHABET[digit] + code
self.long_of[code] = long_url
return "http://tinyurl.com/" + code
```

LC 187 · Repeated DNA Sequences

Problem: A DNA string `s` is made of the letters A, C, G and T. Return every 10-letter substring that occurs more than once, in any order: "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT" gives ["AAAAACCCCC","CCCCCAAAAA"].

Intuition: Slide a 10-letter window across `s` and drop each window into a seen set; a window whose place is already taken has occurred before. The answer is a set too, so a third sighting is not reported twice. Two bits per letter make a window a 20-bit integer that rolls in O(1).

Differs: the keys are windows of the input, not a caller's calls. Place is `seen.add(window)`, and a window that finds its place taken goes into `repeated`. No class, no removal.

```
seen, repeated = set(), set()
for i in range(len(s) - 9):
    window = s[i:i + 10]                  # HAsh the window into the set
    if window in seen:
        repeated.add(window)              # its place was already taken
    seen.add(window)
```

Roots · FiFiSaSu

FInd one root, FInd the other, SAme? stop; else hang the Small Under the big.

Picture. Each group is a tree of arrows pointing up to its root, and the root names the group. A union hangs one root under the other with a single arrow; a find climbs to the root and, on the way back, points the whole path straight at it.

Invariant. Two nodes are connected by the edges seen so far exactly when they have the same root, and `size[root]` is the size of that root's group.

Use when. connected, same group, merge accounts that share something, number of provinces or components, redundant edge, a cycle in an undirected graph, equivalences that arrive one at a time, connect everything at minimum cost.

Book entries. #41 Union-Find · #118 Kruskal's Minimum Spanning Tree

GENERAL SOLUTION

Problem. Nodes 0 .. n-1 receive undirected edges one at a time. Report how many groups remain and which edges joined two nodes that were already connected. Five nodes with the edges `[0,1]`, `[2,3]`, `[0,2]`, `[3,1]` end in 2 groups, {0, 1, 2, 3} and {4}, and `[3,1]` is redundant.

Intuition. Each group is a tree of parent pointers whose root names it, so two nodes are connected exactly when their roots match. An edge either joins two roots, one pointer write, or finds a single root and closes a cycle. Hanging the smaller tree under the bigger keeps every tree at most log n tall, and path compression flattens each path a find climbs.

```
edge    roots    action                  groups
[0,1]   0, 1     1 under 0               4
[2,3]   2, 3     3 under 2               3
[0,2]   0, 2     sizes tie: 2 under 0    2
[3,1]   0, 0     same root: redundant    2
```

```python
def find(parent, x):                        # FInd: climb to the root
    if parent[x] != x:
        parent[x] = find(parent, parent[x])   # point the path at the root
    return parent[x]


def union(parent, size, a, b):
    root_a, root_b = find(parent, a), find(parent, b)   # FInd, FInd
    if root_a == root_b:
        return False                        # SAme? stop: already one group
    if size[root_a] < size[root_b]:
        root_a, root_b = root_b, root_a   # root_a is the bigger tree
    parent[root_b] = root_a               # Small Under big
    size[root_a] += size[root_b]
    return True


def merge_all(n, edges):
    parent, size = list(range(n)), [1] * n
    groups, redundant = n, []
    for a, b in edges:                      # the caller: one union per edge
        if union(parent, size, a, b):
            groups -= 1                     # two groups became one
        else:
            redundant.append([a, b])      # both ends already connected
    return groups, redundant
```

Example.

```python
print(merge_all(5, [[0, 1], [2, 3], [0, 2], [3, 1]]))      # (2, [[3, 1]])
```

Dials. What a node is (an index, a label 1 .. n, an account, a value or a prime factor), where the edges come from (a list, the 1s of a matrix, a shared email, a shared factor, sorted by cost), and what is read at the end (the group count, the first union that fails, the size at each root, a total weight).

Traps. Comparing parents instead of roots: before the last union `parent[3]` is 2 and `parent[1]` is 0, so `[3,1]` would look like a join. Labels 1 .. n need n + 1 slots. A forest is not connected: check that n - 1 unions succeeded before calling the result a tree.

PROBLEMS

LC 684 · Redundant Connection

Problem: A tree on nodes 1 .. n got one extra edge, and `edges` lists all n edges. Return an edge whose removal leaves a tree, the one that comes last in `edges` if several do: `[[1,2],[1,3],[2,3]]` gives `[2,3]`.

Intuition: One extra edge makes exactly one cycle. Fed in order, every edge of that cycle but the last joins two groups, and the last one finds both ends already connected. It is the edge that closes the cycle, and so the cycle edge that comes last.

Differs: labels run 1 .. n, so `parent` gets n + 1 slots; return the first edge whose union fails instead of collecting them.

LC 547 · Number of Provinces

Problem: `isConnected` is an n x n matrix with `isConnected[i][j] = 1` when cities i and j are directly connected. A province is a group connected directly or through other cities; return how many there are: `[[1,1,0],[1,1,0],[0,0,1]]` gives 2.

Intuition: Every 1 is an edge. Start with n provinces, one per city; each union that joins two roots turns two provinces into one, so the count after the last edge is the answer. The matrix is symmetric, so its upper triangle holds every edge once.

Differs: the edges come from the matrix: union i and j for every `i < j` with a 1, and read only `groups`.

```python
for i in range(n):
    for j in range(i + 1, n):             # the upper triangle: each edge once
        if is_connected[i][j] and union(parent, size, i, j):
            groups -= 1
```

LC 323 · Number of Connected Components in an Undirected Graph

Problem: Given n nodes 0 .. n-1 and a list of undirected edges, return the number of connected components: n = 5 with `[[0,1],[1,2],[3,4]]` gives 2.

Intuition: The components start as n single nodes, and an edge lowers the count exactly when it joins two different roots; an edge inside a component changes nothing.

Differs: none: the answer is the general solution's `groups`.

LC 721 · Accounts Merge

Problem: Each account is a name, then emails. Accounts that share an email belong to one person; equal names alone do not. Return each person as the name and their sorted emails: two "John" accounts sharing johnsmith@mail.com merge, while a third John stays apart.

Intuition: A shared email is an edge between two accounts. Remember the first account that listed each email; a later account listing it is unioned with that one. When all accounts are in, the emails grouped by the root of their owner are exactly one person's.

Differs: nodes are account indices, and edges come from a map email → first owner. Afterwards group every email under `find(parent, owner[email])` and output that root's name with the sorted emails.

```
for i, account in enumerate(accounts):
    for email in account[1:]:
        if email in owner:
            union(parent, size, i, owner[email])   # a shared email
        else:
            owner[email] = i
```

LC 261 · Graph Valid Tree

Problem: Given n nodes 0 .. n-1 and a list of undirected edges, return true if the edges form a valid tree: n = 5 with `[[0,1],[0,2],[0,3],[1,4]]` is true, and with `[[0,1],[1,2],[2,3],[1,3],[1,4]]` false.

Intuition: A tree on n nodes has exactly n - 1 edges and no cycle. With n - 1 edges and no cycle, every edge merged two groups, so the n groups fell to one: the graph is connected.

Differs: return False unless `len(edges) == n - 1`, then return False at the first union that finds a single root; otherwise True.

LC 1135 · Connecting Cities With Minimum Cost

Problem: Cities 1 .. n and `connections[i] = [x, y, cost]`, undirected. Return the minimum total cost that leaves every pair of cities connected, or -1 if that is impossible: n = 3 with `[[1,2,5],[1,3,6],[2,3,1]]` gives 6.

Intuition: Kruskal: take the edges cheapest first and keep one only if it joins two groups. Every cheaper edge has already been seen, so a joining edge is the cheapest one leaving its group, and by the cut property some minimum tree uses it. An edge inside a group would only close a cycle.

Differs: the caller's loop runs over the connections sorted by cost, and a successful union adds the cost. A tree needs n - 1 edges; fewer means some city is cut off: -1.

```
for x, y, cost in sorted(connections, key=lambda c: c[2]):   # cheapest first
    if union(parent, size, x, y):
        total += cost
        used += 1
return total if used == n - 1 else -1
```

LC 952 · Largest Component Size by Common Factor

Problem: Given unique positive integers `nums`, two numbers are joined by an edge when they share a factor greater than 1. Return the size of the largest connected component: `[4,6,15,35]` gives 4, through the shared factors 2, 3 and 5.

Intuition: Testing every pair costs n^2 gcds. Make the factors nodes too: union each number with its divisors, so two numbers that share a prime meet at that prime's node. The components of the numbers are then the groups, and a tally of roots over `nums` gives their sizes.

Differs: nodes are the values 0 .. max(nums), factors included. Union x with f and x // f for every divisor f from 2 up to √x, then count `find(parent, x)` over `nums` only: factor nodes are not in the array.

```
f = 2
while f * f <= x:                          # every divisor pair f, x // f
    if x % f == 0:
        union(parent, size, x, f)
        union(parent, size, x, x // f)
    f += 1
```

Trie · WaCreStep

WAlk the letters, CREate a missing child, STEP down; flag the end.

Picture. Keys are spelled downward from a root, one letter per edge. Keys that share a prefix share its path, and a flag marks the node where a key stops; a question about a prefix walks down its letters and looks at what hangs below.

Invariant. The path from the root to any node spells a prefix of some inserted key, and a node is flagged exactly when that prefix is itself an inserted key.

Use when. prefix, starts with, autocomplete, suggestions as you type, the shortest root of a word, wildcard search over a dictionary, keys that are paths, how many words share a prefix, many words searched at once.

Book entries. #43 Trie · #97 Design Search Autocomplete System · #143 In-Memory File System

GENERAL SOLUTION

Problem. Build a prefix tree with `insert(word)`; `search(word)`, whether that exact word was inserted; and `starts_with(prefix)`, whether some inserted word begins with it. After inserting "apple", "app", "apt" and "bat", `search("ap")` is false but `starts_with("ap")` is true.

Intuition. A set of words answers `search`, but a prefix question would compare against every word. In a trie "app", "apple" and "apt" all hang below the one path a → p, so any question costs one step per letter, however many words are stored. A query walks the same way an insert does but never creates; the flag tells a stored word from a prefix of one.

```
call               walk                          result
insert("apple")    a p p l e created, e flagged
insert("app")      a p p reused, p flagged
search("ap")       a p: the node has no flag     False
starts_with("ap")  a p: the node exists          True
search("apl")      a p, then no child l          False
```

```python
class TrieNode:
    def __init__(self):
        self.children = {}                # letter -> child node
        self.is_end = False               # an inserted word stops here


class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:                   # WAlk the letters
            if ch not in node.children:
                node.children[ch] = TrieNode()   # CREate if missing
            node = node.children[ch]      # STEP down
        node.is_end = True                # flag the end

    def walk(self, prefix):               # the same walk, creating nothing
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None               # the path breaks off
            node = node.children[ch]
        return node

    def search(self, word):               # the path and the flag
        node = self.walk(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):        # the path alone
        return self.walk(prefix) is not None
```

Example.

```python
trie = Trie()
for word in ["apple", "app", "apt", "bat"]:
    trie.insert(word)
print(trie.search("ap"), trie.starts_with("ap"))           # False True
```

Dials. What a step is (a letter, a name in a path, every child at a `.`), what a node stores (the end flag, the whole word, a count of the words that pass through, the three best below it, a file's content), when the walk stops (the end of the key, the first flag, a broken path), and where it starts: at the root on every call, or at a cursor kept between calls (#97).

Traps. A `search` that returns true when the path exists finds "ap" in a trie holding only "apple"; check the flag. A query that creates nodes as it walks makes later prefix questions lie. A cursor that falls off the trie must stay off until the key ends, even if later letters exist below the root.

PROBLEMS

LC 208 · Implement Trie (Prefix Tree)

Problem: Implement `Trie` with `insert(word)`, `search(word)` and `startsWith(prefix)`: `insert("apple")`, then `search("apple")` is true, `search("app")` false and `startsWith("app")` true; after `insert("app")`, `search("app")` is true.

Intuition: The general solution. Words that share a prefix share its path, so every call costs one step per letter, and the flag tells a stored word from a mere prefix of one.

Differs: none; LeetCode spells the last method `startsWith`.

LC 211 · Design Add and Search Words Data Structure

Problem: Design `WordDictionary` with `addWord(word)` and `search(word)`, where a `.` in the searched word matches any letter: after adding “bad”, “dad” and “mad”, “pad” is false, “bad” true, “.ad” true and “b..” true.

Intuition: A letter steps to one child, but a dot may be any letter, so the walk splits and tries every child. The search becomes a depth-first search over the trie that succeeds as soon as one branch reaches the end of the pattern on a flagged node.

Differs: insert is unchanged. `search` becomes recursive: `match(node, i)` asks whether `word[i:]` matches below `node`, and a dot branches over every child.

```python
def match(node, i):                       # does word[i:] match below node?
    if i == len(word):
        return node.is_end
    if word[i] == ".":                    # any letter: try every child
        return any(match(c, i + 1) for c in node.children.values())
    return word[i] in node.children and match(node.children[word[i]], i + 1)
```

LC 1268 · Search Suggestions System

Problem: Given `products` and `searchWord`, after each typed letter of `searchWord` suggest up to three products that start with the typed prefix, the lexicographically smallest: mobile, mouse, moneypot, monitor, mousepad with “mouse” give [mobile, moneypot, monitor] twice, then [mouse, mousepad] three times.

Intuition: The products with the typed prefix are the words below that prefix’s node. Insert the products in sorted order and let each node keep the first three words that pass through it: they are its three smallest. Typing steps one child down per letter and reads the node’s list.

Differs: each node stores `suggest`, filled at insert up to three words. The query keeps one cursor across the letters and gives `[]` for every letter after the path breaks.

```python
for word in sorted(products):
    node = root
    for ch in word:
        node = node.children.setdefault(ch, TrieNode())
        if len(node.suggest) < 3:
            node.suggest.append(word)     # sorted order: the first 3 win
```

LC 648 · Replace Words

Problem: Given a dictionary of roots and a sentence, replace every word by the shortest root that is a prefix of it, and keep words with no root: roots cat, bat, rat turn “the cattle was rattled by the battery” into “the cat was rat by the bat”.

Intuition: Walk each word down the trie of roots. The first flagged node met is the shortest root, so the walk stops there; a path that breaks first means no root is a prefix, and the word stays as it is.

Differs: the walk stops early: at the first flagged node it returns the prefix so far, and on a broken path the whole word.

```python
node, replaced = root, word               # no root met: the word stays
for i, ch in enumerate(word):
    node = node.children.get(ch)
    if node is None or node.is_end:       # a broken path, or the first flag
        replaced = word[:i + 1] if node else word
        break
```

LC 642 · Design Search Autocomplete System

Problem: Design `AutocompleteSystem(sentences, times)`: `input(c)` returns the three most typed past sentences starting with the text typed so far, ties in ASCII order; `#` records the sentence. With "i love you" 5, "island" 3, "iroman" 2, "i love leetcode" 2, `input('i')` gives ["i love you","island","i love leetcode"].

Intuition: The sentences that match the typed text all end below its node, so a cursor stays on that node between calls and a keystroke is one step down, never a fresh walk. Ranking by `(-count, sentence)` puts "i love leetcode" before "iroman": a space sorts before every letter.

Differs: the walk is spread over calls: the cursor lives in `self.node` and stays `None` from a broken path until `#`. An end node stores its sentence, a map holds the counts, and `#` inserts the typed text once more and resets the cursor.

```
found = []
self.collect(self.node, found)            # every sentence below the cursor
ranked = sorted((-self.counts[s], s) for s in found)
return [s for _, s in ranked[:3]]
```

LC 588 · Design In-Memory File System

Problem: Design `FileSystem` with `ls(path)`, `mkdir(path)`, `addContentToFile(filePath, content)` and `readContentFromFile(filePath)` on absolute paths; `ls` of a file gives [its name], of a directory its sorted entries: after `mkdir("/a/b/c")` and `addContentToFile("/a/b/c/d", "hello")`, `ls("/")` gives ["a"].

Intuition: A directory tree is a trie whose edges are names instead of letters, so walking a path is the trie walk over its parts. `mkdir` and `addContentToFile` create missing nodes on the way; `ls` and the read only walk. A directory's children are its listing.

Differs: a step is a path part, `[p for p in path.split("/") if p]`, so "/" is the empty walk. The end flag becomes `is_file` with `content`, appended to on every add; `ls` returns `[parts[-1]]` for a file, else `sorted(node.children)`.

LC 2416 · Sum of Prefix Scores of Strings

Problem: For each string in `words`, add up the scores of all its non-empty prefixes, where the score of a prefix is how many strings in `words` start with it: `["abc","ab","bc","b"]` gives `[5,4,3,2]`, since "abc" scores 2 + 2 + 1.

Intuition: The words that start with a prefix are exactly the words whose insert walked through that prefix's node. Count the passages at every node during the inserts; a word's answer is then the sum of the counts along its own path.

Differs: no end flag: Step increments `node.count` at every node it enters, and a second walk per word sums the counts on its path.

```
for ch in word:                            # insert: count every passage
    node = node.children.setdefault(ch, TrieNode())
    node.count += 1
```

Tally · Cobuco

COunt every key, BUcket the keys by their count, COllect from the right end.

Picture. Tally marks on a board. Each item adds a mark under its key; the keys are then filed into numbered bins by how many marks they carry, and the answer is read off from the fullest bin down, or from the emptiest up.

Invariant. `count[key]` is how many times the key has been seen so far, and bucket c holds exactly the keys whose count is c.

Use when. most frequent, top k, sort by frequency, a row, column or diagonal filled by one player, a cooldown between repeats, no two equal neighbours, at least h papers with h citations, values in a small range.

Book entries. #48 Design Tic-Tac-Toe · #61 Bucket Sort · #71 Task Scheduler · #108 Counting Sort

GENERAL SOLUTION

Problem. Given an array and an integer k, return the k most frequent values in any order; the answer is unique. `[1, 1, 1, 2, 2, 3]` with k = 2 gives `[1, 2]`.

Intuition. A count is a whole number from 1 to n, so it can serve as an array index: drop each value into `buckets[count]` and read the buckets from n down. The bucket index already is the order, so nothing is compared or sorted, and the whole job is O(n) instead of the O(n log n) of a sort by count.

```
step      what it holds
COunt     {1: 3, 2: 2, 3: 1}
BUcket    buckets[3] = [1], buckets[2] = [2], buckets[1] = [3]
COllect   c = 6, 5, 4 are empty; c = 3 takes 1; c = 2 takes 2: k reached
```

```python
def top_k_frequent(nums, k):
    count = {}
    for x in nums:                          # COunt each value
        count[x] = count.get(x, 0) + 1
    buckets = [[] for _ in range(len(nums) + 1)]   # a count is at most n
    for x in count:                         # BUcket: the index is the count
        buckets[count[x]].append(x)
    result = []
    for c in range(len(nums), 0, -1):       # COllect from the full end
        for x in buckets[c]:
            result.append(x)
            if len(result) == k:
                return result
    return result
```

Example.

```python
print(top_k_frequent([1, 1, 1, 2, 2, 3], 2))               # [1, 2]
```

Dials. What is counted (values, letters, a signed mark per line, papers per citation count), the bucket index (the count itself, a count capped at n, a value shifted by the minimum as in #108), and how to collect: from the full end or the empty end, stopping at k, writing each key as many times as it was counted, or one formula read from the top bucket alone (#71).

Traps. n buckets instead of n + 1: a value that fills the whole array has count n. `elif` for the anti-diagonal misses the centre of an odd board, which lies on both diagonals. In the scheduler, adding 1 for the last row instead of the number of tasks tied at the top count.

PROBLEMS

LC 348 · Design Tic-Tac-Toe

Problem: Design `TicTacToe(n)`: `move(row, col, player)` marks an empty cell for player 1 or 2 and returns the player if the move fills a whole row, column or diagonal with their marks, else 0. On 3 x 3, if player 1 holds (2, 0) and (2, 2), their move at (2, 1) returns 1.

Intuition: A line wins only when all n of its cells hold one player's marks. Player 1 adds +1 and player 2 adds -1 to every line through the cell, so a counter hits n or -n exactly then; a line marked by both can reach neither, which is the rule that a mixed line is dead.

Differs: Count is one signed counter per row, per column and per long diagonal, 2n + 2 in all, with no Bucket; Collect tests `abs(counter) == n` on the at most four lines the move touches.

```python
self.rows[row] += sign                  # sign: +1 for player 1, -1 for 2
self.cols[col] += sign
if row == col:
    self.diagonal += sign
if row + col == self.n - 1:             # not elif: the centre is on both
    self.anti_diagonal += sign
```

LC 347 · Top K Frequent Elements

Problem: Given an integer array `nums` and k, return the k most frequent elements in any order; the answer is unique: `[1,1,1,2,2,3]` with k = 2 gives `[1,2]`.

Intuition: The general solution. LeetCode asks for better than O(n log n); a frequency is at most n, so buckets indexed by count order the values without a single comparison, in O(n).

Differs: none.

LC 451 · Sort Characters By Frequency

Problem: Given a string `s`, sort it in decreasing order of character frequency and return it; equally frequent characters may come in any order: “tree” gives “eert”, and “eetr” is also right.

Intuition: The same buckets with no k: collect every character from the full end, written as many times as it occurs. Copies of one character stay together because each is written as one run.

Differs: Collect writes `ch * c` for every character in bucket c, from c = n down to 1, and joins the runs; there is no early stop.

```
runs = []
for c in range(len(s), 0, -1):            # the most frequent first
    for ch in buckets[c]:
        runs.append(ch * c)               # the whole run at once
return "".join(runs)
```

LC 621 · Task Scheduler

Problem: Tasks are letters A to Z, one interval each; two runs of the same letter need at least n intervals between them, and the CPU may idle. Return the fewest intervals: `["A","A","A","B","B","B"]` with n = 2 gives 8, as A B idle A B idle A B.

Intuition: The most frequent task frames the schedule: `max_count - 1` full rows of width n + 1, then a last row holding every task tied at `max_count`. The other tasks fill the gaps; if they overflow, the rows widen and no idle is left, so the answer is at least `len(tasks)`.

Differs: Count, then no buckets: Collect reads only the top, `max_count` and `tied`, the number of tasks that reach it, into one formula.

```
max_count = max(count.values())
tied = sum(1 for c in count.values() if c == max_count)
frame = (max_count - 1) * (n + 1) + tied  # full rows, then the last row
return max(frame, len(tasks))             # overflow: rows widen, no idles
```

LC 767 · Reorganize String

Problem: Rearrange the characters of `s` so that no two adjacent characters are equal and return any such string, or “” if none exists: “aab” gives “aba”, and “aaab” gives “”.

Intuition: Even slots 0, 2, 4, … are never adjacent. The most frequent letter goes there first, and fits exactly when its count is at most (n + 1) // 2, which every answer needs. The others follow along the evens, then the odds, with too few copies to land beside their own.

Differs: Count, return “” if the top count exceeds (n + 1) // 2, then Collect into slots rather than a list: letters most frequent first, written to the even indices and then the odd ones.

```
slots = list(range(0, len(s), 2)) + list(range(1, len(s), 2))
out, k = [""] * len(s), 0
for ch in sorted(count, key=count.get, reverse=True):   # most frequent first
    for _ in range(count[ch]):
        out[slots[k]] = ch                # the evens first, then the odds
        k += 1
```

LC 274 · H-Index

Problem: Given `citations[i]`, the citations of paper i, return the h-index: the largest h such that at least h papers have at least h citations each: `[3,0,6,1,5]` gives 3, since three papers have 3 or more.

Intuition: h never exceeds n, the number of papers, so a paper cited more than n times counts as n. Bucket the papers by capped citations and walk from n down, adding up the papers with at least c citations; the first c whose total reaches c is the largest h.

Differs: the bucket index is `min(citation, n)` and a bucket holds a number of papers: the counting sort of #108, capped at n. Collect is a suffix sum from n down that stops at the first `papers >= h`.

```
papers = 0
for h in range(n, -1, -1):
    papers += buckets[h]                  # papers cited at least h times
    if papers >= h:
        break                             # the largest such h
```

LC 1636 · Sort Array by Increasing Frequency

Problem: Sort `nums` in increasing order of how often each value occurs; values with the same frequency go in decreasing order: `[1,1,2,2,2,3]` gives `[3,1,1,2,2,2]`, and `[2,3,1,3,2]` gives `[1,3,3,2,2]`.

Intuition: The order is by count first and by value second, so the buckets are read from the empty end up, and inside a bucket the larger value comes first. Each value is written as many times as it occurs, so equal values stay together.

Differs: Collect walks the buckets from c = 1 up, each sorted in decreasing order, and writes each value c times; in one line, `sorted(nums, key=lambda x: (count[x], -x))`.

```
result = []
for c in range(1, len(nums) + 1):         # the least frequent first
    for x in sorted(buckets[c], reverse=True):   # a tie: larger value first
        result.extend([x] * c)
```

Index of problems

LC 1 Two Sum · *Lookback*
LC 3 Longest Substring Without Repeating Characters · *Window*
LC 4 Median of Two Sorted Arrays · *Bisect the answer*
LC 5 Longest Palindromic Substring · *Interval*
LC 10 Regular Expression Matching · *2-D table*
LC 11 Container With Most Water · *Pincers*
LC 14 Longest Common Prefix · *Runs*
LC 15 3Sum · *Pincers*
LC 17 Letter Combinations of a Phone Number · *Enumerate*
LC 20 Valid Parentheses · *Nesting*
LC 22 Generate Parentheses · *Enumerate*
LC 23 Merge k Sorted Lists · *Cheapest first*
LC 25 Reverse Nodes in k-Group · *Map and order*
LC 28 Find the Index of the First Occurrence in a String · *Fold*
LC 31 Next Permutation · *Enumerate*
LC 32 Longest Valid Parentheses · *Nesting*
LC 33 Search in Rotated Sorted Array · *Bisect the answer*
LC 34 Find First and Last Position of Element in Sorted Array · *Bisect stored order*
LC 35 Search Insert Position · *Bisect stored order*
LC 37 Sudoku Solver · *Path search*
LC 38 Count and Say · *Runs*
LC 39 Combination Sum · *Enumerate*
LC 40 Combination Sum II · *Enumerate*
LC 42 Trapping Rain Water · *Pincers*
LC 43 Multiply Strings · *Peel*
LC 45 Jump Game II · *Reach*
LC 46 Permutations · *Enumerate*
LC 47 Permutations II · *Enumerate*
LC 48 Rotate Image · *Two passes*
LC 49 Group Anagrams · *Lookback*
LC 50 Pow(x, n) · *Peel*
LC 51 N-Queens · *Path search*
LC 52 N-Queens II · *Path search*
LC 53 Maximum Subarray · *Chain*
LC 54 Spiral Matrix · *Pincers*
LC 55 Jump Game · *Reach*
LC 56 Merge Intervals · *Sorted sweep*
LC 57 Insert Interval · *Sorted sweep*
LC 62 Unique Paths · *2-D table*
LC 64 Minimum Path Sum · *2-D table*
LC 67 Add Binary · *Peel*
LC 68 Text Justification · *Runs*
LC 70 Climbing Stairs · *Chain*
LC 72 Edit Distance · *2-D table*
LC 73 Set Matrix Zeroes · *Two passes*
LC 74 Search a 2D Matrix · *Bisect stored order*
LC 75 Sort Colors · *Partition*
LC 76 Minimum Window Substring · *Window*
LC 77 Combinations · *Enumerate*
LC 78 Subsets · *Enumerate*
LC 79 Word Search · *Path search*
LC 84 Largest Rectangle in Histogram · *Ratchet*
LC 85 Maximal Rectangle · *Ratchet*
LC 88 Merge Sorted Array · *Chase and zip*
LC 90 Subsets II · *Enumerate*
LC 91 Decode Ways · *Chain*
LC 97 Interleaving String · *2-D table*
LC 98 Validate Binary Search Tree · *Pass down*
LC 99 Recover Binary Search Tree · *Pass down*
LC 102 Binary Tree Level Order Traversal · *Wave*
LC 105 Construct Binary Tree from Preorder and Inorder Traversal · *Pass down*
LC 121 Best Time to Buy and Sell Stock · *State vector*
LC 122 Best Time to Buy and Sell Stock II · *State vector*
LC 123 Best Time to Buy and Sell Stock III · *State vector*
LC 124 Binary Tree Maximum Path Sum · *Combine*
LC 127 Word Ladder · *Wave*
LC 128 Longest Consecutive Sequence · *Lookback*
LC 130 Surrounded Regions · *Diver*
LC 131 Palindrome Partitioning · *Enumerate*
LC 132 Palindrome Partitioning II · *Interval*
LC 133 Clone Graph · *Wave*
LC 134 Gas Station · *Fold*
LC 135 Candy · *Two passes*
LC 136 Single Number · *Fold*
LC 138 Copy List with Random Pointer · *Map and order*
LC 139 Word Break · *Chain*
LC 141 Linked List Cycle · *Chase and zip*
LC 142 Linked List Cycle II · *Chase and zip*
LC 143 Reorder List · *Chase and zip*
LC 146 LRU Cache · *Map and order*
LC 148 Sort List · *Combine*
LC 149 Max Points on a Line · *Lookback*
LC 150 Evaluate Reverse Polish Notation · *Nesting*
LC 151 Reverse Words in a String · *Two passes*
LC 152 Maximum Product Subarray · *Chain*
LC 153 Find Minimum in Rotated Sorted Array · *Bisect the answer*
LC 155 Min Stack · *Nesting*
LC 162 Find Peak Element · *Bisect the answer*
LC 167 Two Sum II - Input Array Is Sorted · *Pincers*
LC 168 Excel Sheet Column Title · *Peel*
LC 169 Majority Element · *Fold*
LC 173 Binary Search Tree Iterator · *Diver*
LC 187 Repeated DNA Sequences · *Hash to slots*
LC 188 Best Time to Buy and Sell Stock IV · *State vector*
LC 191 Number of 1 Bits · *Peel*
LC 198 House Robber · *Chain*
LC 199 Binary Tree Right Side View · *Wave*
LC 200 Number of Islands · *Diver*
LC 202 Happy Number · *Chase and zip*
LC 204 Count Primes · *Two passes*
LC 206 Reverse Linked List · *Map and order*
LC 207 Course Schedule · *Stamp*
LC 208 Implement Trie (Prefix Tree) · *Trie*
LC 209 Minimum Size Subarray Sum · *Window*
LC 210 Course Schedule II · *Wave*
LC 211 Design Add and Search Words Data Structure · *Trie*
LC 212 Word Search II · *Path search*
LC 213 House Robber II · *Chain*
LC 215 Kth Largest Element in an Array · *Partition*
LC 216 Combination Sum III · *Enumerate*
LC 221 Maximal Square · *2-D table*
LC 224 Basic Calculator · *Nesting*
LC 229 Majority Element II · *Fold*
LC 230 Kth Smallest Element in a BST · *Pass down*
LC 232 Implement Queue using Stacks · *Cursor stacks*
LC 235 Lowest Common Ancestor of a Binary Search Tree · *Pass down*
LC 236 Lowest Common Ancestor of a Binary Tree · *Combine*
LC 238 Product of Array Except Self · *Two passes*
LC 239 Sliding Window Maximum · *Ratchet*
LC 240 Search a 2D Matrix II · *Pincers*
LC 253 Meeting Rooms II · *Sorted sweep*
LC 261 Graph Valid Tree · *Roots*
LC 269 Alien Dictionary · *Stamp*
LC 271 Encode and Decode Strings · *Runs*
LC 274 H-Index · *Tally*
LC 278 First Bad Version · *Bisect stored order*
LC 279 Perfect Squares · *Knapsack*
LC 287 Find the Duplicate Number · *Chase and zip*
LC 289 Game of Life · *Two passes*
LC 295 Find Median from Data Stream · *Keep the best*
LC 297 Serialize and Deserialize Binary Tree · *Pass down*
LC 300 Longest Increasing Subsequence · *Subsequence*
LC 307 Range Sum Query - Mutable · *Combine*
LC 309 Best Time to Buy and Sell Stock with Cooldown · *State vector*
LC 312 Burst Balloons · *Interval*
LC 315 Count of Smaller Numbers After Self · *Combine*
LC 322 Coin Change · *Knapsack*
LC 323 Number of Connected Components in an Undirected Graph · *Roots*
LC 329 Longest Increasing Path in a Matrix · *Stamp*
LC 337 House Robber III · *Combine*
LC 338 Counting Bits · *Peel*
LC 340 Longest Substring with At Most K Distinct Characters · *Window*
LC 341 Flatten Nested List Iterator · *Diver*
LC 346 Moving Average from Data Stream · *Time ring*
LC 347 Top K Frequent Elements · *Tally*
LC 348 Design Tic-Tac-Toe · *Tally*
LC 353 Design Snake Game · *Time ring*
LC 354 Russian Doll Envelopes · *Subsequence*
LC 355 Design Twitter · *Cheapest first*
LC 359 Logger Rate Limiter · *Time ring*
LC 362 Design Hit Counter · *Time ring*
LC 373 Find K Pairs with Smallest Sums · *Cheapest first*
LC 377 Combination Sum IV · *Knapsack*
LC 380 Insert Delete GetRandom O(1) · *Map and order*
LC 382 Linked List Random Node · *Fold*
LC 394 Decode String · *Nesting*
LC 407 Trapping Rain Water II · *Cheapest first*
LC 410 Split Array Largest Sum · *Bisect the answer*
LC 415 Add Strings · *Peel*
LC 416 Partition Equal Subset Sum · *Knapsack*
LC 424 Longest Repeating Character Replacement · *Window*

LC 435 Non-overlapping Intervals · *Sorted sweep*
LC 438 Find All Anagrams in a String · *Window*
LC 443 String Compression · *Runs*
LC 450 Delete Node in a BST · *Pass down*
LC 451 Sort Characters By Frequency · *Tally*
LC 452 Minimum Number of Arrows to Burst Balloons · *Sorted sweep*
LC 459 Repeated Substring Pattern · *Fold*
LC 460 LFU Cache · *Map and order*
LC 474 Ones and Zeroes · *Knapsack*
LC 480 Sliding Window Median · *Keep the best*
LC 494 Target Sum · *Knapsack*
LC 502 IPO · *Keep the best*
LC 503 Next Greater Element II · *Ratchet*
LC 516 Longest Palindromic Subsequence · *Interval*
LC 518 Coin Change II · *Knapsack*
LC 525 Contiguous Array · *Lookback*
LC 528 Random Pick with Weight · *Bisect stored order*
LC 535 Encode and Decode TinyURL · *Hash to slots*
LC 543 Diameter of Binary Tree · *Combine*
LC 547 Number of Provinces · *Roots*
LC 560 Subarray Sum Equals K · *Lookback*
LC 587 Erect the Fence · *Ratchet*
LC 588 Design In-Memory File System · *Trie*
LC 621 Task Scheduler · *Tally*
LC 622 Design Circular Queue · *Time ring*
LC 642 Design Search Autocomplete System · *Trie*
LC 647 Palindromic Substrings · *Interval*
LC 648 Replace Words · *Trie*
LC 673 Number of Longest Increasing Subsequence · *Subsequence*
LC 684 Redundant Connection · *Roots*
LC 692 Top K Frequent Words · *Keep the best*
LC 695 Max Area of Island · *Diver*
LC 703 Kth Largest Element in a Stream · *Keep the best*
LC 704 Binary Search · *Bisect stored order*
LC 705 Design HashSet · *Hash to slots*
LC 706 Design HashMap · *Hash to slots*
LC 714 Best Time to Buy and Sell Stock with Transaction Fee · *State vector*
LC 715 Range Module · *Sorted sweep*
LC 721 Accounts Merge · *Roots*
LC 739 Daily Temperatures · *Ratchet*
LC 743 Network Delay Time · *Cheapest first*
LC 752 Open the Lock · *Wave*
LC 763 Partition Labels · *Reach*
LC 767 Reorganize String · *Tally*
LC 769 Max Chunks To Make Sorted · *Reach*
LC 785 Is Graph Bipartite? · *Wave*
LC 787 Cheapest Flights Within K Stops · *State vector*
LC 797 All Paths From Source to Target · *Path search*
LC 802 Find Eventual Safe States · *Stamp*
LC 841 Keys and Rooms · *Diver*
LC 844 Backspace String Compare · *Cursor stacks*
LC 846 Hand of Straights · *Sorted sweep*
LC 875 Koko Eating Bananas · *Bisect the answer*
LC 876 Middle of the Linked List · *Chase and zip*
LC 881 Boats to Save People · *Pincers*
LC 895 Maximum Frequency Stack · *Map and order*
LC 901 Online Stock Span · *Ratchet*
LC 905 Sort Array By Parity · *Partition*
LC 912 Sort an Array · *Partition*
LC 933 Number of Recent Calls · *Time ring*
LC 952 Largest Component Size by Common Factor · *Roots*
LC 973 K Closest Points to Origin · *Partition*
LC 974 Subarray Sums Divisible by K · *Lookback*
LC 981 Time Based Key-Value Store · *Bisect stored order*
LC 994 Rotting Oranges · *Wave*
LC 1004 Max Consecutive Ones III · *Window*
LC 1011 Capacity To Ship Packages Within D Days · *Bisect the answer*
LC 1024 Video Stitching · *Reach*
LC 1046 Last Stone Weight · *Keep the best*
LC 1048 Longest String Chain · *Subsequence*
LC 1135 Connecting Cities With Minimum Cost · *Roots*
LC 1143 Longest Common Subsequence · *2-D table*
LC 1192 Critical Connections in a Network · *Stamp*
LC 1203 Sort Items by Groups Respecting Dependencies · *Stamp*
LC 1235 Maximum Profit in Job Scheduling · *Subsequence*
LC 1249 Minimum Remove to Make Valid Parentheses · *Nesting*
LC 1268 Search Suggestions System · *Trie*
LC 1312 Minimum Insertion Steps to Make a String Palindrome · *Interval*
LC 1326 Minimum Number of Taps to Open to Water a Garden · *Reach*
LC 1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance · *State vector*
LC 1381 Design a Stack With Increment Operation · *Cursor stacks*
LC 1472 Design Browser History · *Cursor stacks*
LC 1514 Path with Maximum Probability · *Cheapest first*
LC 1584 Min Cost to Connect All Points · *Cheapest first*
LC 1592 Rearrange Spaces Between Words · *Runs*
LC 1631 Path With Minimum Effort · *Cheapest first*
LC 1636 Sort Array by Increasing Frequency · *Tally*
LC 1964 Find the Longest Valid Obstacle Course at Each Position · *Subsequence*
LC 1971 Find if Path Exists in Graph · *Diver*
LC 2034 Stock Price Fluctuation · *Keep the best*
LC 2161 Partition Array According to Given Pivot · *Partition*
LC 2296 Design a Text Editor · *Cursor stacks*
LC 2416 Sum of Prefix Scores of Strings · *Trie*

Index of book entries

The 150 entries of *150 Algorithms*, each under the one method its code uses.

#1 Two Sum · *Lookback*
#2 Prefix Sum + Hash Map · *Lookback*
#3 Product of Array Except Self · *Two passes*
#4 Longest Consecutive Sequence · *Lookback*
#5 Group Anagrams · *Lookback*
#6 Reverse a Linked List · *Map and order*
#7 3Sum · *Pincers*
#8 Container With Most Water · *Pincers*
#9 Trapping Rain Water · *Pincers*
#10 Floyd's Tortoise and Hare · *Chase and zip*
#11 Longest Substring Without Repeating Characters · *Window*
#12 Find All Anagrams in a String · *Window*
#13 Minimum Window Substring · *Window*
#14 Binary Search · *Bisect stored order*
#15 Binary Search on the Answer · *Bisect the answer*
#16 Search in a Rotated Sorted Array · *Bisect the answer*
#17 Merge Sort · *Combine*
#18 Quick Sort · *Partition*
#19 Top-K Elements · *Keep the best*
#20 Merge K Sorted Arrays · *Cheapest first*
#21 Find Median from Data Stream · *Keep the best*
#22 Valid Parentheses · *Nesting*
#23 Monotonic Stack · *Ratchet*
#24 Min Stack · *Nesting*
#25 Decode String · *Nesting*
#26 Merge Intervals · *Sorted sweep*
#27 Meeting Rooms II · *Sorted sweep*
#28 Kadane's Algorithm · *Chain*
#29 Best Time to Buy and Sell Stock · *State vector*
#30 Subsets · *Enumerate*
#31 Permutations · *Enumerate*
#32 Binary Tree Level-Order Traversal · *Wave*
#33 Diameter and Max Path Sum · *Combine*
#34 Lowest Common Ancestor · *Combine*
#35 Validate a Binary Search Tree · *Pass down*
#36 Serialize and Deserialize a Binary Tree · *Pass down*
#37 Depth-First Search on a Graph · *Diver*
#38 Number of Islands · *Diver*
#39 BFS Shortest Path · *Wave*
#40 Kahn's Algorithm · *Wave*
#41 Union-Find · *Roots*
#42 Dijkstra's Algorithm · *Cheapest first*
#43 Trie · *Trie*
#44 LRU Cache · *Map and order*
#45 Design Hit Counter · *Time ring*
#46 Design Twitter · *Cheapest first*
#47 Snapshot Array · *Bisect stored order*
#48 Design Tic-Tac-Toe · *Tally*
#49 Flatten Nested List Iterator · *Diver*
#50 Key-Value Store with Transactions · *Cursor stacks*
#51 Hash Map with Separate Chaining · *Hash to slots*
#52 Dutch National Flag · *Partition*
#53 Longest Repeating Character Replacement · *Window*
#54 Minimum Size Subarray Sum · *Window*
#55 Median of Two Sorted Arrays · *Bisect the answer*
#56 Search a 2D Matrix · *Pincers*
#57 Find Peak Element · *Bisect the answer*
#58 Longest Increasing Subsequence · *Subsequence*
#59 Heapify · *Keep the best*
#60 Quickselect · *Partition*
#61 Bucket Sort · *Tally*
#62 Largest Rectangle in Histogram · *Ratchet*
#63 Basic Calculator · *Nesting*
#64 Sliding Window Maximum · *Ratchet*
#65 Insert Interval · *Sorted sweep*
#66 Interval List Intersections · *Chase and zip*
#67 Jump Game · *Reach*
#68 Jump Game II · *Reach*
#69 Gas Station · *Fold*
#70 Interval Scheduling · *Sorted sweep*
#71 Task Scheduler · *Tally*
#72 Partition Labels · *Reach*
#73 Combination Sum · *Enumerate*
#74 Word Search · *Path search*
#75 Construct Binary Tree from Preorder and Inorder · *Pass down*
#76 BST Search, Insert, Delete · *Pass down*
#77 BST Iterator · *Diver*
#78 Multi-Source BFS · *Wave*
#79 Word Ladder · *Wave*
#80 Clone Graph · *Wave*
#81 Cycle Detection in a Directed Graph · *Stamp*
#82 Topological Sort with DFS · *Stamp*
#83 Alien Dictionary · *Wave*
#84 Connected Components · *Diver*
#85 Word Search II · *Path search*
#86 KMP String Matching · *Fold*
#87 Longest Palindromic Substring · *Interval*
#88 Encode and Decode Strings · *Runs*
#89 Game of Life · *Two passes*
#90 Fibonacci · *Chain*
#91 Insert Delete GetRandom O(1) · *Map and order*
#92 Time-Based Key-Value Store · *Bisect stored order*
#93 LFU Cache · *Map and order*
#94 Cache with Expiring Keys · *Keep the best*
#95 Design Browser History · *Cursor stacks*
#96 Design Underground System · *Lookback*
#97 Design Search Autocomplete System · *Trie*
#98 My Calendar I and II · *Bisect stored order*
#99 Design Snake Game · *Time ring*
#100 URL Shortener · *Peel*
#101 Hash Map with Open Addressing · *Hash to slots*
#102 Boyer-Moore Majority Vote · *Fold*
#103 Next Permutation · *Enumerate*
#104 Reorder List · *Chase and zip*
#105 Median of K Sorted Arrays · *Bisect the answer*
#106 Random Pick with Weight · *Bisect stored order*
#107 Segment Tree · *Combine*
#108 Counting Sort · *Tally*
#109 Heap Sort · *Keep the best*
#110 Circular Queue · *Time ring*
#111 Candy · *Two passes*
#112 Hand of Straights · *Sorted sweep*
#113 N-Queens · *Path search*
#114 Bipartite Check · *Wave*
#115 Cycle Detection in an Undirected Graph · *Diver*
#116 Critical Connections · *Stamp*
#117 Bellman-Ford · *State vector*
#118 Kruskal's Minimum Spanning Tree · *Roots*
#119 Prim's Minimum Spanning Tree · *Cheapest first*
#120 Rabin-Karp · *Window*
#121 Longest Common Prefix · *Runs*
#122 Text Justification · *Runs*
#123 String Compression · *Runs*
#124 Reverse Words in a String · *Two passes*
#125 Multiply Strings · *Peel*
#126 Spiral Matrix · *Pincers*
#127 Rotate Image · *Two passes*
#128 GCD and LCM · *Peel*
#129 Sieve of Eratosthenes · *Two passes*
#130 Fast Exponentiation · *Peel*
#131 Reservoir Sampling · *Fold*
#132 Orientation and Segment Intersection · *Ratchet*
#133 Convex Hull · *Ratchet*
#134 Max Points on a Line · *Lookback*
#135 Single Number · *Fold*
#136 Count Set Bits · *Peel*
#137 Bitmask Subset Enumeration · *Enumerate*
#138 Rate Limiter · *Time ring*
#139 Consistent Hashing · *Hash to slots*
#140 Stock Price Tracker · *Keep the best*
#141 All O(1) Data Structure · *Map and order*
#142 Range Module · *Sorted sweep*
#143 In-Memory File System · *Trie*
#144 Design a Text Editor · *Cursor stacks*
#145 Unique ID Generator · *Peel*
#146 Bloom Filter · *Hash to slots*
#147 Count-Min Sketch · *Hash to slots*
#148 Quadtree · *Combine*
#149 Job Scheduler · *Cheapest first*
#150 Inverted Index · *Chase and zip*